

BỘ CÔNG AN
HỌC VIỆN KỸ THUẬT VÀ CÔNG NGHỆ AN NINH



ĐỒ ÁN TỐT NGHIỆP

**NGHIÊN CỨU XÂY DỰNG HỆ THỐNG
PHẦN MỀM TỰ ĐỘNG HÓA CI/CD
TRONG QUẢN LÝ VÀ RÀ QUÉT MÃ NGUỒN BẢO MẬT
TẠI TRƯỜNG ĐẠI HỌC KỸ THUẬT - HẠ CẦN CẦN**

Học viên: Nguyễn Tuấn Anh

Ngành/chuyên ngành: AN TOÀN THÔNG TIN CẦN CẦN

Niên khoá: 2021-2026

Bắc Ninh, năm 2026

BỘ CÔNG AN
HỌC VIỆN KỸ THUẬT VÀ CÔNG NGHỆ AN NINH



ĐỒ ÁN TỐT NGHIỆP

**NGHIÊN CỨU XÂY DỰNG HỆ THỐNG
PHẦN MỀM TỰ ĐỘNG HÓA CI/CD
TRONG QUẢN LÝ VÀ RÀ QUÉT MÃ NGUỒN BẢO MẬT
TẠI TRƯỜNG ĐẠI HỌC KỸ THUẬT - HẬU CẦN CAND**

Ngành/chuyên ngành: An toàn thông tin CAND

Người hướng dẫn: Thạc sĩ Đoàn Ngọc Tú

Học viên thực hiện: Nguyễn Tuấn Anh

Lớp: B1

Khoá: D11 (2021 - 2026)

Hệ: Đại học

Bắc Ninh, năm 2026

LỜI CAM ĐOAN

Tôi xin cam đoan các kết quả được trình bày trong Đồ án này là công trình nghiên cứu của tôi, được thực hiện dưới sự hướng dẫn của thầy ThS. Đoàn Ngọc Tú. Các số liệu, kết quả trong Đồ án là hoàn toàn trung thực và chưa được công bố trong bất kỳ công trình nào trước đây. Tất cả các tài liệu, kết quả được sử dụng để tham khảo trong Đồ án đều đã được trích dẫn đầy đủ và tuân thủ đúng quy định về bản quyền và đạo đức nghiên cứu.

Tôi cũng xin cam đoan rằng trong bài nghiên cứu này không có tài liệu chứa bí mật nhà nước hoặc thông tin nội bộ không được phép công bố./.

Bắc Ninh, ngày ... tháng ... năm 2026

Tác giả

(Ký, ghi rõ họ tên)

Nguyễn Tuấn Anh

LỜI CẢM ƠN

Kính thưa quý thầy cô, với mỗi môn học tại trường, việc làm bài báo cáo chuyên đề, Đồ án tốt nghiệp, tạo sản phẩm ứng dụng thực tế là điều thú vị, hấp dẫn nhất đối với tôi trong quá trình học tập. Và cuối cùng, sản phẩm nghiên cứu chắc chắn là sản phẩm mà bản thân tôi tâm huyết nhất để thực hiện, bằng những kiến thức, kỹ năng đã có được qua suốt thời gian học tập tại Nhà trường.

Để hoàn thành đồ án “Nghiên cứu xây dựng hệ thống phần mềm tự động hóa CI/CD trong quản lý và rà quét mã nguồn bảo mật tại Trường Đại học Kỹ thuật - Hậu cần CAND”, trước hết tôi xin gửi lời cảm ơn sâu sắc tới thầy Thạc sĩ Đoàn Ngọc Tú - Phó Giám đốc Trung tâm T2 và quý thầy, cô giáo Khoa Công nghệ và An toàn thông tin Học viện Kỹ thuật và Công nghệ an ninh lời cảm ơn chân thành đã tạo điều kiện thuận lợi giúp tôi trong suốt thời gian học tập, nghiên cứu và thực hiện hoàn thiện Đồ án tốt nghiệp này.

Tôi xin chân thành cảm ơn đến Học viện Kỹ thuật và Công nghệ an ninh đã tạo điều kiện thuận lợi để tôi hoàn thành nhiệm vụ. Tôi cũng xin cảm ơn Đội Công nghệ thông tin Phòng Tham mưu Công an Tỉnh Bắc Ninh cũng như những đơn vị khác đã tạo điều kiện cho phép tôi được tham gia nghiên cứu trong thời gian làm Đồ án tốt nghiệp.

Cuối cùng, tôi xin bày tỏ lòng cảm ơn đến gia đình, anh em, bạn bè, các đồng chí đã động viên và cổ vũ tôi trong suốt thời gian nghiên cứu.

Xin trân trọng cảm ơn!

MỤC LỤC

DANH MỤC TỪ VIẾT TẮT	iv
DANH MỤC HÌNH ẢNH	v
DANH MỤC BẢNG	vii
MỞ ĐẦU	1
CHƯƠNG 1. TỔNG QUAN VỀ CHUYỂN ĐỔI SỐ VÀ CÁC MÔ HÌNH TỰ ĐỘNG HÓA TRIỂN KHAI LIÊN TỤC	6
1.1. Tổng quan về tình hình chuyển đổi số và yêu cầu đặt ra.....	6
1.2. Khái niệm và quy trình nghiệp vụ trong hoạt động triển khai tự động hóa.....	7
1.2.1. Khái niệm về tự động hóa, triển khai liên tục.....	7
1.2.2. Phân tích các nội hàm khái niệm.	10
1.3. Thực trạng cơ sở vật chất.....	11
1.3.1. Thực trạng.....	11
1.3.2. Cơ sở vật chất kỹ thuật.	12
1.4. Các công nghệ triển khai CI/CD phổ biến.....	12
1.4.1. Công nghệ trên hệ thống quản lý mã nguồn.....	13
1.4.2. Công nghệ tích hợp liên tục sử dụng Jenkins.....	14
1.4.3. Module phân tích và rà quét bảo mật mã nguồn.....	15
1.4.4. Module quản lý thư viện, đóng gói và ảo hóa.	17
1.4.5. Module giám sát và theo dõi hoạt động.....	18
1.5. Ý nghĩa và kết quả mang lại của đồ án	19
1.6. Kết luận chương 1	20
CHƯƠNG 2. PHÂN TÍCH QUÁ TRÌNH XÂY DỰNG TÍCH HỢP VÀ TRIỂN KHAI HỆ THỐNG	22
2.1. Tổng quan về yêu cầu hệ thống.	22
2.2. Sơ đồ hóa quy trình nghiệp vụ.....	23
2.2.1. Danh sách dịch vụ.....	23
2.2.2. Đặc tả usecase CI/CD.	23
2.3. Quy trình cài đặt môi trường CI/CD	32
2.3.1. Cài đặt docker	32
2.3.2. Cài đặt gitlab server	32

2.3.3. Cài đặt Nexus Repository	33
2.3.4. Cài Jenkins	35
2.3.5. Cài Java & Maven cho Jenkins Container.....	37
2.3.6. Cài SonarQube	38
2.3.7. Cài Trivy scan images.....	39
2.3.8. Viết Pipeline.....	40
2.4. Quy trình phân luồng làm việc Gitlab quản lý mã nguồn.....	42
2.4.1. Các bước tạo các nhánh Git trên GitLab UI	42
2.4.2. Tạo tag (gắn nhãn version)	43
2.4.3. Bảo vệ nhánh.....	43
2.4.4. Phân quyền người dùng (Access Control).....	45
2.4.5. Dashboard quản lý an ninh mã nguồn.	46
2.4.6. Cơ chế bảo mật xác thực hai lớp 2FA và giao thức OTP	47
2.5. Kết luận Chương 2	50
CHƯƠNG 3. XÂY DỰNG TRIỂN KHAI VÀ KIỂM THỬ HỆ THỐNG ..	51
3.1. Mô hình triển khai hệ thống.....	51
3.1.1. Mô hình vật lý tổng thể CICD.	51
3.1.2. Mô hình logic CICD.	51
3.2. Xây dựng các kịch bản kiểm thử.	52
3.2.1. Kiểm thử về môi trường khởi tạo	52
3.2.2. Kiểm thử quá trình lưu trữ mã nguồn từ GitLab.	53
3.2.3. Kiểm thử quá trình tự động xây dựng ứng dụng.	54
3.2.4. Kiểm thử phân tích mã nguồn tĩnh với SonarQube.	55
3.2.5. Kiểm thử phân tích động với Trivy.	56
3.2.6. Lưu trữ báo cáo và artifacts	57
3.3. Đánh giá kết quả triển khai.	58
3.3.1. Kết quả đạt được về mặt kỹ thuật	58
3.3.2. Đánh giá định lượng kết quả triển khai	59
3.3.3. So sánh với các mô hình triển khai hệ thống khác	60
3.3.4. Định hướng phát triển trong thời gian tới.....	61

3.4. Kiến nghị đề xuất giải pháp nâng cao trong quá trình tự động hóa hiện nay.....	62
3.4.1. Hoàn thiện mô hình DevSecOps, bảo đảm an ninh ngay từ khâu thiết kế.....	62
3.4.2. Triển khai tự động hóa toàn trình, hạn chế tối đa thao tác thủ công ..	63
3.4.3. Gắn mô hình DevOps với cơ chế “một cửa” trong triển khai và quản lý hệ thống.....	64
3.4.4. Chuẩn hóa hạ tầng và nâng cao năng lực đội ngũ cán bộ kỹ thuật.....	64
3.5. Kết luận chương 3.....	65
KẾT LUẬN	66
DANH MỤC TÀI LIỆU THAM KHẢO	67

DANH MỤC TỪ VIẾT TẮT

STT	Viết tắt	Viết đầy đủ
1	ATTT	An toàn thông tin
2	BCA	Bộ Công an
3	CI	Continuous Integration – Tích hợp liên tục
4	CD	Continuous Deployment – Triển khai liên tục
5	SCM	Source Code Management – Quản lý mã nguồn
6	SAST	Static Application Security Testing – Phân tích bảo mật mã nguồn tĩnh
7	SCA	Software Composition Analysis – Phân tích thư viện và phụ thuộc
8	Pipeline	Quy trình tự động hóa CI/CD
9	OWASP	Open Web Application Security Project
10	CVE	Common Vulnerabilities and Exposures – Lỗ hổng bảo mật công bố

DANH MỤC HÌNH ẢNH

Hình 1.1. Tích hợp liên tục CI.....	8
Hình 1.2. Triển khai liên tục CD	9
Hình 1.3. Mô hình chung.....	10
Hình 1.4. Mô hình gitlab quản lý code.....	13
Hình 1.5. Quá trình quản lý build bằng Jenkins.....	14
Hình 1.6. Mô hình tạo báo cáo bảo mật với sonarqube quét tĩnh source.....	16
Hình 1.7. Kết quả đánh giá từ 3 phần mềm quét.....	17
Hình 1.8. Quản lý thông tin máy chủ server	19
Hình 2.1. Hệ thống tích hợp và triển khai liên tục (CI/CD).....	22
Hình 2.2. Usecase build và triển khai mã nguồn.....	24
Hình 2.3. Tiến trình phân tích mã nguồn tĩnh SonarQube	24
Hình 2.4. Usecase build và tag Docker Image	25
Hình 2.5. Use Case rà quét bảo mật Docker Image với Trivy.	26
Hình 2.6. Biểu đồ đăng nhập và đăng xuất Hệ thống CI/CD.....	27
Hình 2.7. Biểu đồ hoạt động xây dựng mã nguồn.....	28
Hình 2.8. Biểu đồ Phân tích mã nguồn tĩnh	29
Hình 2.9. Biểu đồ build và tag Docker Image.....	30
Hình 2.10. Biểu đồ rà quét bảo mật Docker Image.....	31
Hình 2.11. Tạo docker network.....	32
Hình 2.12. Cài đặt Gitlab thành công.....	33
Hình 2.13. Thành công khi cài Nexus	35
Hình 2.14. Cài Jenkins thành công.....	35
Hình 2.15. Thêm các credentials thành công	37
Hình 2.16. Thành công chạy rà quét dự án.....	37
Hình 2.17. Tải các plugin về gitlab, blue ocean cho giám sát.....	38
Hình 2.18. Log không chứa lỗi.....	39
Hình 2.19. Nếu source không lỗi thì thành công và không tự ngắt tiến trình	41
Hình 2.20. Logic fail nếu ≥ 2 lỗi HIGH/CRITICAL.....	41
Hình 2.21. Kết quả quét đã tự động ngắt khi có kết quả code không sạch ...	42
Hình 2.22. Tạo nhánh Git trên GitLab UI	43
Hình 2.23. Tiến hành thêm các bảo vệ (No one).....	44
Hình 2.24. Pin có giá trị 7 ngày từ khi tạo.	48
Hình 2.25. Kết quả tạo xác thực 2FA thành công	49
Hình 3.1. Mô hình vật lý tổng thể	51
Hình 3.2. Mô hình logic	51
Hình 3.3. Xác nhận môi trường các công cụ đã được cài đặt Java, Maven. ..	52
Hình 3.4. Stage Java và Maven báo cáo thành công	53
Hình 3.5. Kiểm thử quá trình lưu trữ mã nguồn từ GitLab ổn định.....	54
Hình 3.6. Kiểm thử quá trình tự động xây dựng ứng dụng	54
Hình 3.7. Build và gắn thẻ Docker Image (Build & Tag Docker Image)	55

Hình 3.8. Kiểm thử phân tích mã nguồn tĩnh với SonarQube chạy ổn định..	55
Hình 3.9. Báo cáo dạng JSON.....	56
Hình 3.10. Báo cáo dạng HTML	57
Hình 3.11. Lưu trữ báo cáo và artifacts.....	58
Hình 3.12. Kết quả đánh giá triển khai.....	58
Hình 3.13. Kết quả đạt được về mặt kỹ thuật.....	59
Hình 3.14. Kết quả ưu điểm của sản phẩm so với các mô hình khác	61

DANH MỤC BẢNG

Bảng 2.1. Danh sách module và mục đích sử dụng.....	23
Bảng 3.1. Đánh giá định lượng kết quả triển khai.....	59
Bảng 3.2. So sánh với các mô hình triển khai hệ thống khác.....	60

MỞ ĐẦU

1. Lý do lựa chọn đề tài.

Trong bối cảnh chuyển đổi số đang diễn ra mạnh mẽ trên mọi lĩnh vực, yêu cầu về việc phát triển phần mềm nhanh chóng, an toàn và hiệu quả trở thành yếu tố then chốt để bảo đảm thành công trong quản lý, vận hành hệ thống thông tin. Tại Trường Đại học Kỹ thuật - Hậu cần CAND, nơi đào tạo nguồn nhân lực chất lượng cao trong lĩnh vực An toàn thông tin, việc nghiên cứu và triển khai các công cụ tự động hóa hiện đại phục vụ công tác phát triển phần mềm và bảo mật là nhiệm vụ cấp thiết, mang tính chiến lược.

Hệ thống CI/CD (Continuous Integration/Continuous Deployment) kết hợp cùng các công cụ rà quét mã nguồn bảo mật tự động sẽ giúp chuẩn hóa và tự động hóa toàn bộ quy trình phát triển phần mềm, từ giai đoạn viết mã, kiểm thử, tích hợp, đến triển khai. Nhờ đó, các lỗi bảo mật có thể được phát hiện sớm ngay trong vòng đời phát triển, giảm thiểu rủi ro tiềm ẩn khi đưa hệ thống vào vận hành. Đồng thời, CI/CD còn nâng cao tốc độ phát triển, tăng tính ổn định và minh bạch trong quản lý mã nguồn, tạo nền tảng quan trọng cho việc xây dựng quy trình phát triển phần mềm an toàn, hiệu quả.

Việc áp dụng hệ thống tự động hóa CI/CD trong quản lý và rà quét mã nguồn góp phần giúp tối ưu hóa nguồn lực, tiết kiệm thời gian, giảm sai sót thủ công, mà còn góp phần hình thành quy trình phát triển phần mềm và tiếp nhận chuyển giao công nghệ chuyên nghiệp, đáp ứng chuẩn mực an toàn thông tin trong CAND. Hơn nữa, triển khai giải pháp này tại Trường Đại học Kỹ thuật - Hậu cần CAND sẽ tạo môi trường học tập, nghiên cứu hiện đại cho giảng viên và học viên, góp phần nâng cao năng lực thực tiễn, tiệm cận với xu hướng DevSecOps quốc tế, đồng thời đáp ứng yêu cầu bảo mật ngày càng cao trong các hệ thống của lực lượng Công an nhân dân.

2. Các công trình nghiên cứu có liên quan

Chuyển đổi số trong lực lượng Công an nhân dân, đặc biệt trong lĩnh vực công nghệ thông tin và tự động hóa quy trình phát triển phần mềm, đã và đang được quan tâm nghiên cứu rộng rãi. Nhiều công trình đã tập trung vào việc ứng dụng các mô hình tích hợp - triển khai liên tục (CI/CD) nhằm nâng cao hiệu quả quản lý, đảm bảo chất lượng và an toàn mã nguồn trong các hệ thống thông tin

của ngành. Các nghiên cứu này nhấn mạnh vai trò của việc tự động hóa trong kiểm thử, triển khai và giám sát mã nguồn, đồng thời đề xuất các giải pháp đảm bảo tính bảo mật, tính toàn vẹn dữ liệu và khả năng truy vết trong môi trường đặc thù của lực lượng CAND.

Dựa trên việc tổng hợp và phân tích các tài liệu, công trình nghiên cứu trong và ngoài nước liên quan đến tự động hóa CI/CD và bảo mật mã nguồn, tôi đã tích lũy được cơ sở lý luận và thực tiễn vững chắc để triển khai đề tài “Nghiên cứu xây dựng hệ thống phần mềm tự động hóa CI/CD trong quản lý và rà quét mã nguồn bảo mật tại Trường Đại học Kỹ thuật - Hậu cần CAND”. Hệ thống được kỳ vọng sẽ góp phần nâng cao năng lực quản lý, giám sát chất lượng mã nguồn, tự động hóa quy trình phát hiện và xử lý lỗ hổng bảo mật, qua đó thúc đẩy quá trình chuyển đổi số và bảo đảm an toàn thông tin trong Nhà trường nói riêng và lực lượng CAND nói chung.

3. Mục đích nghiên cứu

Mục đích của đề tài “Nghiên cứu xây dựng hệ thống phần mềm tự động hóa CI/CD trong quản lý và rà quét mã nguồn bảo mật tại Trường Đại học Kỹ thuật - Hậu cần CAND” là thiết kế và triển khai một hệ thống tích hợp - triển khai liên tục (CI/CD) có khả năng tự động hóa toàn bộ quy trình quản lý, kiểm tra và rà quét bảo mật mã nguồn trong môi trường phát triển phần mềm.

Hệ thống được xây dựng nhằm tăng cường hiệu quả quản lý mã nguồn, rút ngắn thời gian kiểm thử và triển khai, đồng thời phát hiện sớm các lỗ hổng bảo mật trong quá trình phát triển, qua đó đảm bảo an toàn thông tin và nâng cao chất lượng phần mềm.

4. Nhiệm vụ nghiên cứu.

Để đạt được mục tiêu của đề tài “Nghiên cứu xây dựng hệ thống phần mềm tự động hóa CI/CD trong quản lý và rà quét mã nguồn bảo mật tại Trường Đại học Kỹ thuật - Hậu cần CAND”, các nhiệm vụ nghiên cứu cụ thể được xác định như sau:

Xây dựng hệ thống CI/CD đáp ứng đầy đủ các chức năng cần thiết phục vụ công tác tự động hóa quy trình phát triển, kiểm thử và triển khai phần mềm tại Nhà trường. Hệ thống có khả năng nâng cấp, mở rộng và tùy chỉnh linh hoạt, đảm bảo phù hợp với hạ tầng công nghệ thông tin và định hướng chuyển đổi số

của Nhà trường.

Thiết kế và triển khai hạ tầng lưu trữ dữ liệu mẫu, phục vụ việc quản lý mã nguồn, cấu hình pipeline, kết quả quét bảo mật và lịch sử triển khai. Dữ liệu được chuẩn hóa, lưu trữ tập trung và có khả năng chia sẻ giữa các công cụ và hệ thống khác trong môi trường DevSecOps nội bộ.

Tích hợp các công cụ bảo mật tự động (như Trivy, SonarQube, Dependency Check, v.v.) vào pipeline CI/CD để phát hiện và đánh giá lỗ hổng bảo mật trong mã nguồn và container images, đồng thời xây dựng cơ chế cảnh báo, báo cáo và theo dõi kết quả quét.

Thiết kế giao diện web quản lý trực quan, thân thiện với người dùng, giúp cán bộ kỹ thuật và quản trị viên dễ dàng theo dõi tiến trình build, deploy, và kết quả quét bảo mật. Giao diện hỗ trợ chức năng tìm kiếm, thống kê, tạo báo cáo và biểu đồ phân tích giúp đánh giá tình hình bảo mật, chất lượng mã nguồn và hiệu quả triển khai.

Đảm bảo hệ thống an toàn, bảo mật và phân quyền rõ ràng, bao gồm xác thực người dùng, phân tách vai trò (developer, reviewer, admin), kiểm soát truy cập và ghi nhật ký (audit log) đầy đủ.

Ứng dụng các công nghệ hiện đại như Docker, GitLab Runner, Harbor, SonarQube và các công cụ DevSecOps để xây dựng hệ thống tự động hóa CI/CD đồng bộ, bảo mật và có tính thực tiễn cao, góp phần đẩy mạnh quá trình chuyển đổi số trong lĩnh vực an toàn thông tin và quản lý phát triển phần mềm tại Trường Đại học Kỹ thuật - Hậu cần CAND.

5. Đối tượng nghiên cứu

Nhóm người sử dụng hệ thống: Bao gồm cán bộ kỹ thuật, quản trị viên hệ thống và giảng viên phụ trách công tác quản lý, triển khai và giám sát quy trình phát triển phần mềm. Nhóm này sử dụng hệ thống để cấu hình pipeline, theo dõi quá trình build - deploy - scan, và khai thác các báo cáo bảo mật mã nguồn trong Nhà trường.

Nhóm đối tượng quản lý và giám sát bảo mật mã nguồn: Tập trung vào việc nghiên cứu cách thức hệ thống tự động phát hiện, phân loại và theo dõi các lỗ hổng trong mã nguồn, container images và dependencies trong quá trình

CI/CD. Mục tiêu là giúp Nhà trường kiểm soát được rủi ro an ninh phần mềm một cách kịp thời và có hệ thống.

Nhóm công nghệ và hạ tầng kỹ thuật: Gồm các nền tảng và công cụ phục vụ quy trình CI/CD và DevSecOps như GitLab, Docker, Harbor, GitLab Runner, SonarQube, Trivy, Jenkins, MongoDB... Đề tài tập trung nghiên cứu cách kết hợp, tích hợp và tối ưu các công nghệ này trong môi trường mạng nội bộ của Trường Đại học Kỹ thuật - Hậu cần CAND, đảm bảo khả năng mở rộng, ổn định và bảo mật cao.

Nhóm quy trình quản lý và kiểm soát an toàn thông tin: Nghiên cứu mô hình tổ chức và vận hành hệ thống DevSecOps trong môi trường thực tế của Nhà trường, bao gồm cơ chế phân quyền, ghi nhật ký (audit logs), quy trình rà quét tự động, quản lý cảnh báo và báo cáo kết quả quét mã nguồn.

6. Phương pháp nghiên cứu

Đề tài sử dụng kết hợp nhiều phương pháp nghiên cứu khác nhau nhằm đảm bảo tính khoa học và tính ứng dụng cao, cụ thể như sau:

- Phương pháp nghiên cứu lý thuyết tìm hiểu, tổng hợp cơ sở lý luận thông qua giáo trình, tài liệu học thuật, các công trình nghiên cứu trong và ngoài nước liên quan đến DevOps, CI/CD, và bảo mật phần mềm (DevSecOps).

- Phương pháp khảo sát và thực nghiệm xây dựng và triển khai thử nghiệm hệ thống CI/CD tự động trong môi trường thật của Trường Đại học Kỹ thuật - Hậu cần CAND, qua đó đánh giá hiệu năng, độ an toàn và mức độ đáp ứng nhu cầu quản lý mã nguồn.

- Phương pháp phân tích và thiết kế hướng đối tượng (OOAD) áp dụng các mô hình UML (Use Case, Sequence, Activity, Class Diagram,...) và công cụ hỗ trợ thiết kế hệ thống để phân tích yêu cầu và mô hình hóa quy trình hoạt động của hệ thống.

- Phương pháp trao đổi và thu thập ý kiến chuyên gia trao đổi, tham khảo ý kiến từ các cán bộ, giảng viên Khoa Công nghệ và An toàn thông tin (K7), Trung tâm T1, T2, cùng các đơn vị kỹ thuật có liên quan, nhằm hoàn thiện mô hình và đảm bảo hệ thống phù hợp với thực tiễn triển khai trong Nhà trường.

7. Phạm vi nghiên cứu.

Đề tài tập trung vào việc xây dựng, triển khai và vận hành hệ thống tự động hóa CI/CD phục vụ quản lý và rà quét mã nguồn bảo mật trong môi trường nội bộ của Trường Đại học Kỹ thuật - Hậu cần CAND.

Phạm vi không bao gồm việc triển khai trên môi trường điện toán đám mây công cộng hay các hệ thống ngoài phạm vi quản lý của Nhà trường.

8. Các đóng góp của ĐA/KLTN.

Về mặt kiến thức:

Đề tài cung cấp nền tảng kiến thức toàn diện về xây dựng, triển khai và tích hợp hệ thống CI/CD kết hợp kiểm thử bảo mật (DevSecOps), giúp nâng cao hiểu biết và năng lực thực hành về tự động hóa quy trình phát triển phần mềm an toàn trong nhà trường.

Về mặt sản phẩm:

Xây dựng được một hệ thống CI/CD hoàn chỉnh tích hợp các công cụ tự động quét mã nguồn và container (Trivy, SonarQube, Harbor, Jenkins/GitLab Runner...), có khả năng quản lý, giám sát và báo cáo tình trạng bảo mật theo thời gian thực.

Hệ thống có thể áp dụng trực tiếp trong công tác quản lý phát triển, kiểm thử và đảm bảo an toàn thông tin phần mềm tại Nhà trường, góp phần vào tiến trình chuyển đổi số và tự động hóa hạ tầng CNTT trong lực lượng CAND.

9. Bố cục của Đồ án

Nội dung của đồ án được trình bày trong 03 chương chính như sau:

Chương 1. Tổng quan về hệ thống tự động hóa CI/CD và các giải pháp rà quét mã nguồn bảo mật.

Chương 2. Phân tích và thiết kế hệ thống tự động hóa CI/CD trong quản lý và rà quét mã nguồn bảo mật.

Chương 3. Xây dựng, triển khai và kiểm thử hệ thống tại Trường Đại học Kỹ thuật - Hậu cần CAND.

CHƯƠNG 1. TỔNG QUAN VỀ CHUYỂN ĐỔI SỐ VÀ CÁC MÔ HÌNH TỰ ĐỘNG HÓA TRIỂN KHAI LIÊN TỤC

1.1. Tổng quan về tình hình chuyển đổi số và yêu cầu đặt ra

Chuyển đổi số đã trở thành một chủ đề trung tâm trong hơn một thập kỷ qua, nhưng cách tiếp cận và tư duy về chuyển đổi số đang thay đổi mạnh mẽ vào giai đoạn 2024-2025, mở cánh cửa bước vào năm 2026. *Thay đổi tư duy về chuyển đổi số từ số hóa quy trình đến tái tạo mô hình chuyển dần sang tự động hóa*, nếu như trong giai đoạn trước, nhiều cơ quan tổ chức thường hiểu chuyển đổi số là số hóa các quy trình hiện tại - tức áp dụng công nghệ để tăng hiệu suất cho những gì đã có. Nhưng bước sang 2026, cách hiểu này được mở rộng đáng kể. Chuyển đổi số không còn đơn thuần là một sáng kiến CNTT, mà là sự tái hình dung mô hình trong nền kinh tế số. Điều này có nghĩa là thay vì chỉ “bắt vít” thêm công nghệ vào các bộ phận rời rạc, cơ quan tổ chức, doanh nghiệp cần thiết kế lại cấu trúc tổ chức, quy trình vận hành và cách thức tạo ra giá trị từ đầu, xoay quanh và nhu cầu của người sử dụng.

Từ bối cảnh trên, yêu cầu đặt ra đối với các cơ sở đào tạo và nghiên cứu công nghệ thông tin là phải chuyển từ cách tiếp cận công nghệ rời rạc sang xây dựng các quy trình phát triển phần mềm có tính tự động hóa, kiểm soát chất lượng và bảo mật ngay từ đầu. Hoạt động phát triển phần mềm không chỉ dừng ở việc hoàn thiện chức năng mà cần bảo đảm tính ổn định, khả năng mở rộng và an toàn thông tin trong suốt vòng đời hệ thống.

Điều này đòi hỏi phải hình thành môi trường phát triển hiện đại, nơi các bước tích hợp, kiểm thử, đánh giá chất lượng mã nguồn và rà quét lỗ hổng bảo mật được thực hiện tự động, liên tục và có thể giám sát. Việc áp dụng các mô hình CI/CD kết hợp DevSecOps trở thành yêu cầu tất yếu nhằm giảm thiểu sai sót thủ công, phát hiện sớm rủi ro kỹ thuật và nâng cao độ tin cậy của sản phẩm phần mềm.

Đối với Trường Đại học Kỹ thuật - Hậu cần CAND, yêu cầu này càng có ý nghĩa quan trọng khi vừa phục vụ nghiên cứu, vừa phục vụ đào tạo nguồn nhân lực công nghệ chất lượng cao. Việc xây dựng hệ thống tự động hóa CI/CD tích hợp kiểm soát bảo mật không chỉ nâng cao hiệu quả quản lý mã nguồn tiếp nhận chuyển giao công nghệ mà còn tạo môi trường thực hành sát với thực tế, đáp ứng xu hướng phát triển phần mềm an toàn trong kỷ nguyên số.

Chuyển đổi số không phải đích đến cố định mà là một hành trình liên tục. Trong thời kỳ thế giới định hình lại cách tiếp cận chuyển đổi số, toàn diện hơn, nhân văn hơn và gắn chặt với giá trị thực. Những tổ chức, quốc gia nắm bắt được “tinh thần mới” này của chuyển đổi số sẽ không chỉ tồn tại, mà còn bứt phá dẫn đầu trong kỷ nguyên số đầy hứa hẹn phía trước.

1.2. Khái niệm và quy trình nghiệp vụ trong hoạt động triển khai tự động hóa.

1.2.1. Khái niệm về tự động hóa, triển khai liên tục.

1.2.1.1. Một số khái niệm về tự động hóa, triển khai liên tục.

- **Tích hợp liên tục** (tên Tiếng anh là *Understanding Continuous Integration viết tắt là CI*) Liên tục tích hợp (Continuous Integration - CI) là quy trình cho phép tích hợp thường xuyên và đều đặn các thay đổi của mã nguồn vào nhánh chính của kho lưu trữ. Mỗi thay đổi được thực hiện bởi lập trình viên (thông qua thao tác commit hoặc merge) sẽ tự động kích hoạt các bước kiểm thử và xây dựng (build) hệ thống. Cơ chế này giúp nhóm DevOps nhanh chóng phát hiện xung đột mã nguồn, các lỗi tiềm ẩn, đồng thời duy trì tính ổn định của toàn bộ cơ sở mã. Mục tiêu cốt lõi của CI là đảm bảo rằng mã nguồn triển khai luôn ở trạng thái hoạt động ổn định, có chất lượng cao, và có thể được cung cấp cho người dùng một cách nhanh chóng và an toàn.

Trước khi triển khai CI, có hai nguyên tắc quan trọng cần xem xét:

(1) Chia nhỏ mã nguồn và tích hợp theo các phiên bản nhỏ.

Trong hầu hết các dự án phần mềm, nhiều lập trình viên sẽ đồng thời phát triển các tính năng khác nhau. Điều này khiến cho việc theo dõi tiến độ của từng thành viên trở nên khó khăn, đặc biệt khi các nhánh tính năng tách biệt trong thời gian dài và chỉ được hợp nhất vào nhánh chính khi hoàn tất. Việc hợp nhất như vậy thường dẫn đến xung đột mã lớn hoặc xuất hiện các vấn đề bất ngờ, bao gồm cả lỗi bảo mật, gây trì hoãn quá trình phát hành (release).

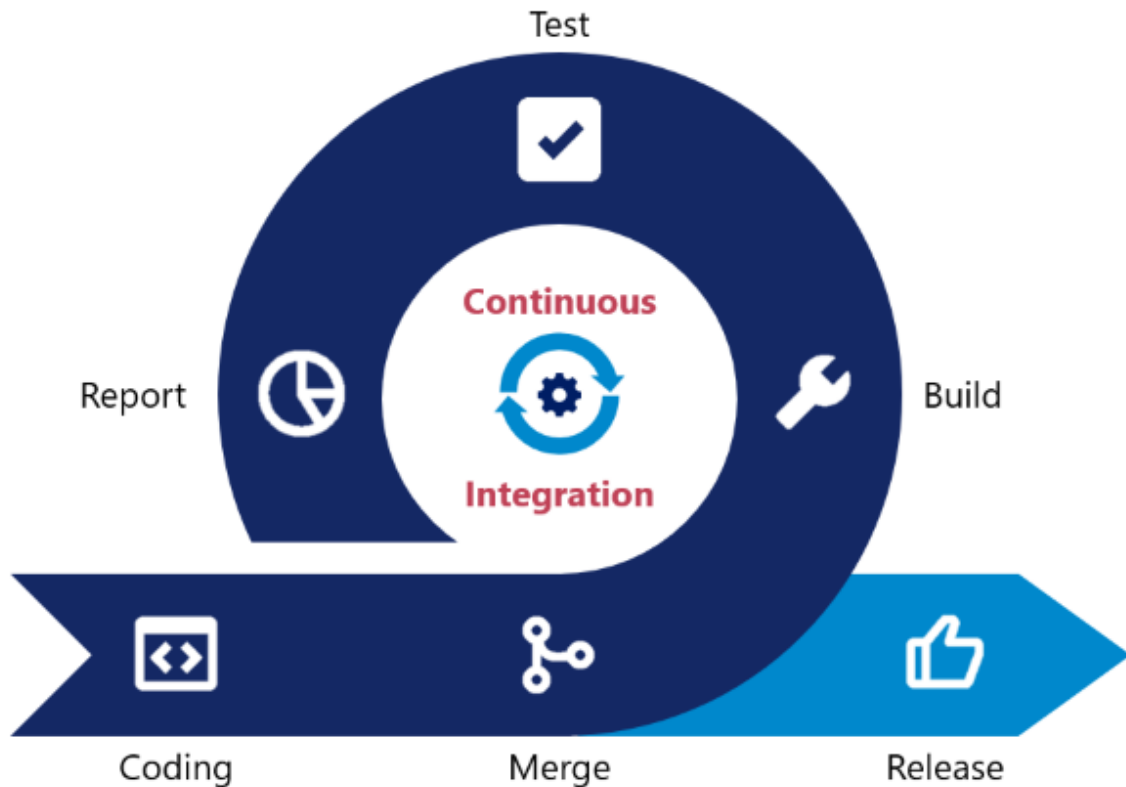
Việc tích hợp thường xuyên từng phần nhỏ của mã nguồn lên nhánh chính giúp nhóm tiến gần hơn đến mô hình CI thực sự, giảm thiểu xung đột và làm cho quy trình phát hành trở nên dễ dự đoán hơn. Một số phương pháp như feature flags có thể hỗ trợ nhóm phát triển tính năng mới nhanh chóng mà vẫn đảm bảo an toàn.

(2) Thiết lập kiểm thử tự động nhằm đảm bảo tính an toàn và bảo mật cho mã nguồn.

Trước đây, các giai đoạn build và test trong quy trình phát triển phần mềm thường tách biệt, và chỉ tiến hành kiểm thử bảo mật sau khi mã đã hoàn thiện. Trong mô hình CI, kiểm thử liên tục (continuous testing) đóng vai trò thiết yếu, nhằm phát hiện lỗi và lỗ hổng bảo mật xuyên suốt quá trình phát triển. Tuy nhiên, việc kiểm thử thủ công toàn bộ thay đổi là rất khó khả thi.

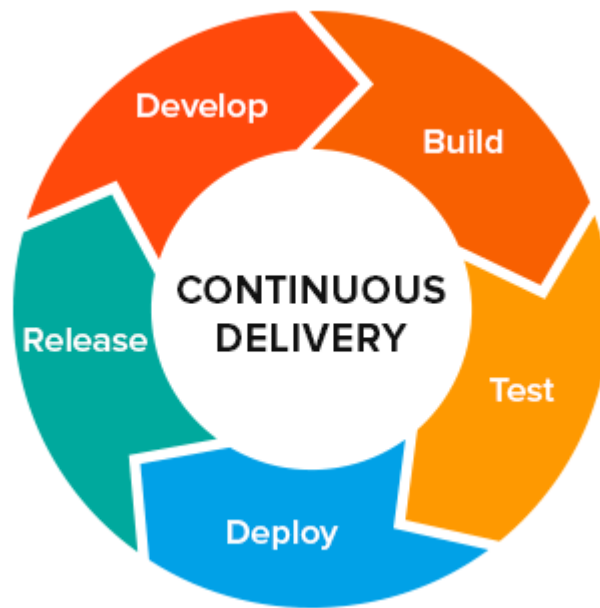
Do đó, các hệ thống CI hiện đại triển khai cơ chế kiểm thử tự động, tự động chạy các bộ kiểm thử như kiểm thử đơn vị (unit test) và kiểm thử tích hợp

(integration test) cho từng thay đổi được đưa vào kho mã. Điều này giúp đảm bảo rằng mã nguồn luôn đáp ứng các yêu cầu về chất lượng, độ an toàn và tính ổn định.



Hình 1.1. Tích hợp liên tục CI

- **Triển khai liên tục** (tên Tiếng anh là *Continuous Delivery* viết tắt là *CD*) là một quy trình phát triển phần mềm hoạt động song song và kế thừa từ *Continuous Integration (CI)* nhằm tự động hóa toàn bộ quá trình phát hành ứng dụng. Thông qua CD, các nhóm phát triển có thể đưa các cập nhật mã nguồn ra môi trường sản xuất theo các chu kỳ ngắn và liên tục, dựa trên mức độ tự động hóa cao để rút ngắn thời gian phát hành và giảm thiểu sự can thiệp thủ công. CD bao gồm đầy đủ các bước trong chu trình sản xuất phần mềm, từ xây dựng (build), kiểm thử (test), cấu hình (configure) cho đến triển khai (deploy). Mục tiêu cuối cùng của CD là đảm bảo phần mềm đã sẵn sàng để cung cấp tới người dùng một cách an toàn, nhất quán và hiệu quả.



Hình 1.2. Triển khai liên tục CD

CD thường được xem như phần mở rộng tự nhiên của CI, và hai khái niệm này thường được đề cập cùng nhau dưới thuật ngữ CI/CD. Quy trình này bao gồm việc tích hợp mã nguồn vào kho lưu trữ chung và thực hiện các hoạt động build và test ngay lập tức cho mỗi thay đổi. Các thao tác đó diễn ra hoàn toàn tự động và có thể lặp lại nhiều lần trong ngày nhằm đảm bảo tính ổn định của cơ sở mã nguồn.

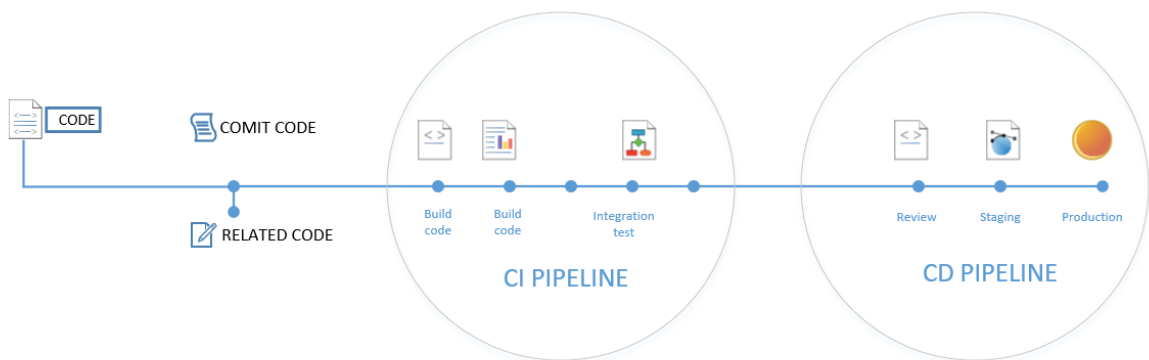
Sau khi CI hoàn tất quá trình xây dựng và kiểm thử mã nguồn trong môi trường chung, CD tiếp tục thực hiện các bước cuối cùng để bảo đảm rằng việc phát hành phần mềm là một quy trình nhất quán, ít rủi ro và dễ tái lập. Nhờ đó, doanh nghiệp có thể phân phối phần mềm tới người dùng với chất lượng đảm bảo và thời gian rút ngắn đáng kể.

Trong thực tiễn, thuật ngữ Continuous Delivery thường được sử dụng thay thế cho Continuous Deployment, tuy nhiên giữa hai mô hình vẫn tồn tại sự khác biệt đáng kể. Continuous Deployment là cấp độ tự động hóa cao nhất: tất cả các thay đổi mã nguồn đã vượt qua quy trình CI sẽ được triển khai trực tiếp lên môi trường sản xuất một cách tự động, không cần sự phê duyệt thủ công. Ngược lại, Continuous Delivery chỉ đảm bảo rằng mã nguồn có thể được triển khai bất cứ lúc nào - quyết định triển khai vẫn tùy thuộc vào con người.

Nói cách khác, Continuous Deployment chỉ có thể được thực hiện khi đã xây dựng hoàn chỉnh nền tảng Continuous Delivery. Điểm khác biệt chính giữa hai mô hình nằm ở mức độ tự động hóa bước triển khai cuối cùng.

1.2.1.2. Khái niệm về tự động hóa, triển khai liên tục (viết tắt là CICD).

Tự động hóa (Automation) là việc áp dụng công nghệ, hệ thống điều khiển và phần mềm để thực hiện các quy trình hoặc tác vụ mà không cần hoặc cần rất ít sự can thiệp từ con người. Mục tiêu chính của tự động hóa là tăng cường hiệu suất, đảm bảo độ chính xác, và giảm thiểu chi phí hoạt động trong các lĩnh vực công nghiệp và phi công nghiệp. Trong đó, CICD là một cách thực hiện nhằm tự động hóa quy trình triển khai và kiểm thử bảo mật cho ứng dụng hoặc mã nguồn.



Hình 1.3. Mô hình chung

Continuous Integration - Continuous Delivery viết tắt là CICD là một quy trình khép kín, tự động và liên tục trong quá trình phát triển phần mềm từ lúc đẩy mã nguồn phần mềm lên hệ thống quản lý mã nguồn, rà quét mã nguồn phần mềm, kiểm tra tính năng phần mềm đến khi mã nguồn được triển khai trên môi trường product.

1.2.2. Phân tích các nội hàm khái niệm.

Các khái niệm về tự động hóa, tích hợp liên tục (CI) và triển khai liên tục (CD) không tồn tại độc lập mà có mối quan hệ mật thiết, hình thành nên một hệ sinh thái công nghệ thống nhất phục vụ mục tiêu tối ưu hóa quy trình phát triển phần mềm. Việc phân tích nội hàm của các khái niệm này góp phần làm rõ bản chất, vai trò và ý nghĩa của chúng trong bối cảnh chuyển đổi số và tự động hóa vận hành hệ thống.

Trước hết, tự động hóa đóng vai trò là nền tảng của toàn bộ mô hình CI/CD. Tự động hóa cho phép các quy trình lặp đi lặp lại như xây dựng mã nguồn, kiểm thử, phân tích bảo mật, đóng gói và triển khai được thực hiện một cách nhất quán, giảm thiểu sai sót và rút ngắn thời gian xử lý. Sự hiện diện của

tự động hóa giúp đảm bảo tính chính xác của quy trình, đồng thời tạo điều kiện để các nhóm phát triển tập trung vào các nhiệm vụ mang tính sáng tạo và có giá trị cao hơn.

Trên nền tảng đó, tích hợp liên tục (CI) mở rộng khả năng tự động hóa vào khâu đầu tiên của chu trình phát triển phần mềm: tích hợp mã nguồn. Nội hàm quan trọng của CI là tính “liên tục” – nghĩa là mọi thay đổi dù nhỏ nhất đều phải được tích hợp, kiểm thử và đánh giá ngay khi xuất hiện. Điều này giúp loại bỏ tình trạng tồn đọng thay đổi mã trong thời gian dài, giảm thiểu xung đột và đảm bảo tính ổn định của cơ sở mã. CI giúp hình thành một trạng thái “mã nguồn luôn sẵn sàng” (always-ready codebase), là tiền đề quan trọng cho việc triển khai nhanh và linh hoạt.

Trong khi đó, triển khai liên tục (CD) tiếp tục mở rộng tự động hóa sang các bước sau của quy trình phát triển, bao gồm kiểm thử sâu hơn, cấu hình hệ thống, đóng gói và triển khai ứng dụng. Nội hàm cốt lõi của CD nằm ở khả năng bảo đảm rằng mọi phiên bản phần mềm sau khi vượt qua CI đều có thể được đưa lên môi trường sản xuất một cách an toàn và có thể lặp lại. CD không chỉ làm giảm rủi ro trong quá trình phát hành mà còn tăng tốc độ phân phối phần mềm, đáp ứng yêu cầu thay đổi nhanh trong hoạt động của doanh nghiệp.

Một khía cạnh khác cần được phân tích trong nội hàm CD là sự tương tác của nó với Continuous Deployment. Mặc dù thường bị nhầm lẫn, hai khái niệm này có mức độ tự động hóa khác nhau. Continuous Delivery bảo đảm khả năng triển khai bất kỳ lúc nào, trong khi Continuous Deployment thực hiện triển khai tự động hoàn toàn. Sự khác biệt này cho thấy một tầng bậc trong nội hàm của CD: từ trạng thái “sẵn sàng triển khai” hướng đến trạng thái “triển khai hoàn toàn tự động”, tùy theo mức độ trưởng thành của hệ thống và sự chấp nhận rủi ro của tổ chức.

Tổng thể lại, nội hàm khái niệm của CI/CD thể hiện sự liên kết chặt chẽ giữa tự động hóa, tích hợp và triển khai. Ba yếu tố này tạo thành một quy trình thống nhất, liên tục và khép kín, góp phần nâng cao chất lượng phần mềm, tối ưu hóa vận hành hệ thống và thúc đẩy chuyển đổi số trong doanh nghiệp. CI/CD không chỉ là một tập hợp công cụ hay phương pháp kỹ thuật, mà còn là triết lý phát triển hiện đại hướng đến sự linh hoạt, hiệu quả và khả năng đáp ứng nhanh trước sự thay đổi của môi trường công nghệ.

1.3. Thực trạng cơ sở vật chất.

1.3.1. Thực trạng.

Trong bối cảnh nhu cầu phát triển phần mềm ngày càng gia tăng, các hoạt động quản lý mã nguồn, kiểm thử và đảm bảo an toàn thông tin tại Trường Đại

học Kỹ thuật - Hậu cần CAND vẫn còn phụ thuộc nhiều vào quy trình thủ công hoặc bán tự động. Điều này dẫn đến thời gian triển khai kéo dài, khó kiểm soát phiên bản, và thiếu cơ chế giám sát liên tục về chất lượng cũng như mức độ bảo mật của mã nguồn.

Các bước như lưu trữ mã nguồn, kiểm tra tiêu chuẩn kỹ thuật, chạy kiểm thử đơn vị, rà quét lỗ hổng, đóng gói và triển khai đều được phân tán ở nhiều công cụ khác nhau và thiếu tính tích hợp. Điều này không chỉ làm chậm tiến độ phát triển mà còn tiềm ẩn nguy cơ bỏ sót các lỗ hổng phần mềm trong quá trình vận hành.

Do đó, việc nghiên cứu và xây dựng một hệ thống CI/CD tự động hóa, có khả năng tích hợp các quy trình quản lý mã nguồn, kiểm thử và rà quét bảo mật theo chuẩn DevSecOps là yêu cầu cấp thiết. Hệ thống này sẽ hỗ trợ tiêu chuẩn hóa quy trình làm việc, đảm bảo tính nhất quán, liên tục và an toàn cho các dự án phần mềm được triển khai tại Nhà trường.

1.3.2. Cơ sở vật chất kỹ thuật.

Nhà trường hiện sở hữu hạ tầng kỹ thuật đáp ứng được yêu cầu triển khai hệ thống tự động hóa CI/CD. Cụ thể:

Các phòng làm việc được trang bị máy tính cấu hình đủ mạnh, kết nối mạng nội bộ ổn định phục vụ quá trình phát triển phần mềm.

Hệ thống máy chủ nội bộ bao gồm máy chủ nguồn, máy chủ dịch vụ Web, máy chủ cơ sở dữ liệu và máy chủ ảo hóa có thể sử dụng để cài đặt các công cụ như GitLab, Jenkins, SonarQube, Nexus Repository, hoặc các nền tảng tương đương trong chuỗi DevSecOps. Mạng nội bộ của Nhà trường có khả năng triển khai các môi trường thử nghiệm (test environment) và môi trường tích hợp (staging environment), tạo điều kiện thuận lợi cho việc vận hành pipeline tự động.

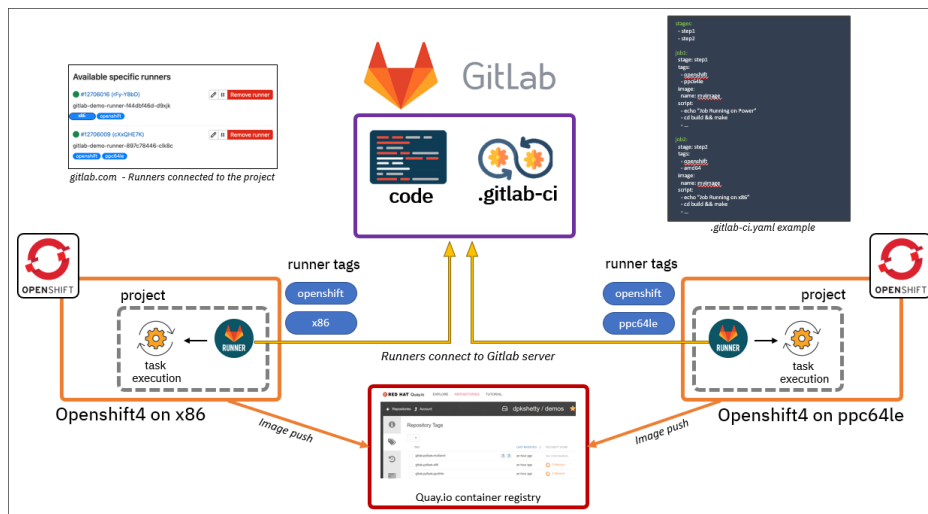
Nhìn chung, cơ sở hạ tầng của Nhà trường đáp ứng tương đối đầy đủ cho việc xây dựng và vận hành một hệ thống CI/CD hoàn chỉnh, hỗ trợ quản lý, kiểm thử và triển khai mã nguồn theo chuẩn kỹ thuật hiện đại.

1.4. Các công nghệ triển khai CI/CD phổ biến.

Trong quá trình xây dựng hệ thống tự động hóa CI/CD, nhiều công nghệ và công cụ khác nhau được sử dụng để đảm bảo quy trình phát triển phần mềm diễn ra liên tục, an toàn và có thể mở rộng. Mỗi công cụ đảm nhiệm một vai trò riêng trong chuỗi DevOps - DevSecOps, tạo thành một hệ sinh thái hoàn chỉnh hỗ trợ quản lý mã nguồn, kiểm thử, phân tích bảo mật, đóng gói và triển khai tự động. Dưới đây là các công nghệ phổ biến cùng phân tích từng module chức năng.

1.4.1. Công nghệ trên hệ thống quản lý mã nguồn.

Git, GitLab SCM và GitHub là những nền tảng quản lý mã nguồn hiện đại, đóng vai trò cốt lõi trong toàn bộ vòng đời phát triển phần mềm, đặc biệt trong các hệ thống tự động hóa CI/CD. Các nền tảng này cung cấp một môi trường lưu trữ mã nguồn tập trung, bảo đảm kiểm soát phiên bản theo từng thay đổi, giúp duy trì tính nhất quán và an toàn của mã nguồn trong những dự án có nhiều lập trình viên cùng tham gia. Dựa trên mô hình kiểm soát phiên bản phân tán, Git cho phép mỗi lập trình viên làm việc độc lập trên máy trạm của mình và chỉ đồng bộ hóa phần việc thông qua cơ chế commit và push, giúp quá trình cộng tác trở nên hiệu quả, linh hoạt và tránh xung đột.



Hình 1.4. Mô hình gitlab quản lý code

Bên trong GitLab và GitHub, mã nguồn được tổ chức theo các nhánh (branch), cho phép triển khai chiến lược phát triển song song theo từng tính năng, bản vá hoặc thử nghiệm. Khi các nhánh hoàn thiện, cơ chế Merge Request/Pull Request giúp rà soát, đánh giá và kiểm tra chất lượng mã trước khi hợp nhất vào nhánh chính, bảo đảm mọi thay đổi đều được xem xét cẩn thận bởi nhóm phát triển. Toàn bộ lịch sử phát triển, bao gồm commit, thay đổi tệp tin, bình luận đánh giá, được lưu trữ đầy đủ, giúp việc truy vết nguyên nhân lỗi, phục hồi trạng thái và đánh giá quá trình phát triển trở nên đơn giản và minh bạch hơn.

Bên cạnh chức năng quản lý mã nguồn, GitLab và GitHub còn tích hợp các mô-đun hỗ trợ quản lý dự án lớn, cho phép theo dõi tiến độ, phân công nhiệm vụ và điều phối công việc theo quy trình chuyên nghiệp. Điểm quan trọng nhất là khả năng tích hợp trực tiếp với hệ thống CI/CD: mỗi hành động như commit, push hoặc tạo merge request đều có thể tự động kích hoạt pipeline để thực thi các bước build, test, phân tích mã nguồn, quét bảo mật hoặc triển khai ứng dụng. Điều này biến SCM trở thành điểm khởi đầu cho toàn bộ quy trình tự

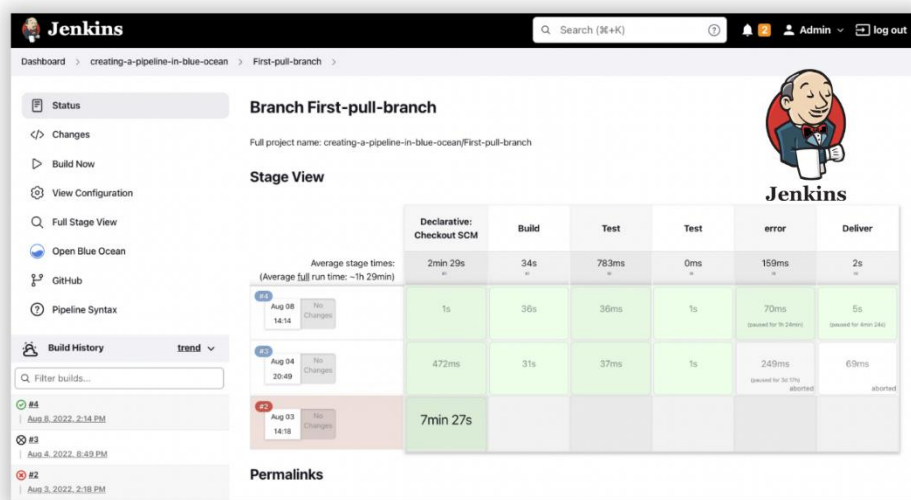
động hóa, đảm bảo rằng mọi thay đổi đều được kiểm tra một cách nhanh chóng, nhất quán và không phụ thuộc vào thao tác thủ công.

Trong hệ thống *nghiên cứu xây dựng giải pháp tự động hóa CI/CD phục vụ quản lý và rà quét mã nguồn bảo mật tại Trường Đại học Kỹ thuật - Hậu cần CAND*, Git, GitLab SCM và GitHub không chỉ là nơi lưu trữ mã nguồn mà còn là trung tâm điều phối toàn bộ quy trình kiểm thử và rà quét bảo mật. Mỗi lần mã nguồn được cập nhật, nền tảng SCM ngay lập tức kích hoạt pipeline để biên dịch, chạy kiểm thử tự động, phân tích chất lượng mã bằng SonarQube và quét lỗ hổng bằng Trivy, giúp đảm bảo rằng mọi thay đổi đều tuân thủ nghiêm ngặt các yêu cầu an toàn thông tin.

Nhờ khả năng kết nối sâu với các công cụ CI/CD và vai trò nền tảng trong quản lý mã nguồn, Git - GitLab - GitHub trở thành lựa chọn tối ưu cho hệ thống tự động hóa phát triển và kiểm soát chất lượng phần mềm, đóng góp quan trọng trong việc nâng cao tính minh bạch, độ tin cậy và an toàn của mã nguồn trong toàn bộ quy trình phát triển tại Nhà trường.

1.4.2. Công nghệ tích hợp liên tục sử dụng Jenkins.

Trong các mô hình phát triển phần mềm hiện đại, hệ thống tích hợp liên tục (Continuous Integration - CI) giữ vai trò trung tâm trong việc tự động hóa quá trình xây dựng và kiểm thử mã nguồn. Các nền tảng tiêu biểu như Jenkins, GitLab CI hay GitHub Actions cung cấp một môi trường CI mạnh mẽ, nơi mọi thay đổi trong kho mã nguồn đều được xử lý thông qua các quy trình tự động hóa nhằm bảo đảm chất lượng, tính ổn định và sự nhất quán của phần mềm ngay từ giai đoạn phát triển. Tiêu biểu: *Jenkins, GitLab CI, GitHub Actions, Pipelines*.



Hình 1.5. Quá trình quản lý build bằng Jenkins

Hệ thống CI hoạt động dựa trên cơ chế tự động build mã nguồn thành các gói thực thi mỗi khi lập trình viên thực hiện commit hoặc push lên hệ thống

quản lý mã nguồn. Khi quá trình build được kích hoạt, hệ thống tự động chạy các lớp kiểm thử như unit test, integration test hay static testing để phát hiện lỗi sớm, giảm thiểu rủi ro trong các giai đoạn sau. Thay vì phụ thuộc vào thao tác thủ công, toàn bộ quy trình được tổ chức thành pipeline gồm nhiều giai đoạn kế tiếp nhau như biên dịch, kiểm thử, phân tích chất lượng mã, đóng gói ứng dụng và có thể mở rộng đến triển khai tự động.

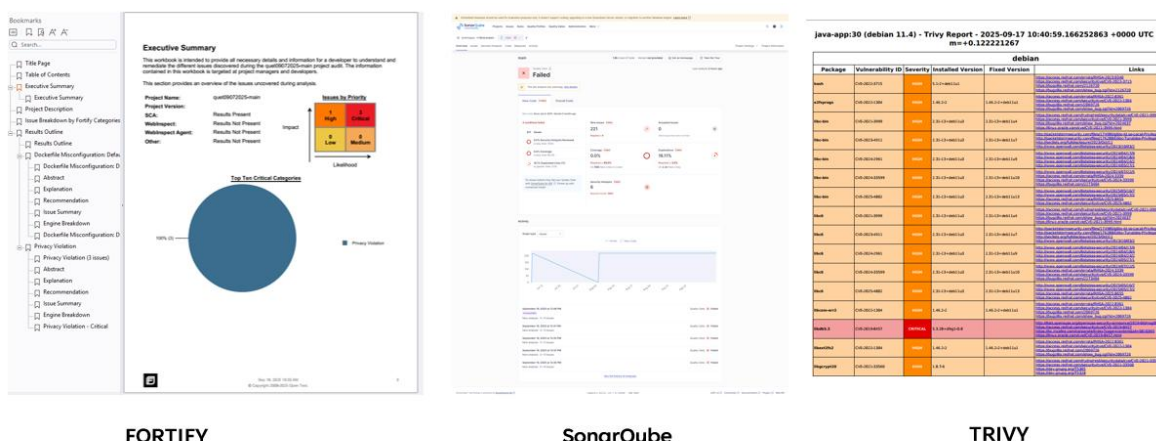
Về mặt cấu trúc, hệ thống CI bao gồm nhiều module phối hợp chặt chẽ với nhau. Pipeline Orchestrator là thành phần điều phối toàn bộ chuỗi quy trình, xác định thứ tự thực thi các bước cũng như điều kiện rẽ nhánh khi một giai đoạn gặp lỗi. Runner/Executor là tác nhân trực tiếp thực thi các tập lệnh, có thể chạy trên máy chủ vật lý, máy ảo hoặc container, bảo đảm tính cô lập và khả năng mở rộng khi số lượng job tăng cao. Job Management chịu trách nhiệm theo dõi trạng thái của từng tác vụ trong pipeline, ghi nhận kết quả build như *Success*, *Failed* hay *Skipped*, giúp nhóm phát triển dễ dàng đánh giá tình trạng hệ thống, giúp nhóm phát triển nắm bắt lỗi ngay khi pipeline thất bại, từ đó rút ngắn thời gian phản ứng và khắc phục sự cố.

Trong tổng thể quy trình CI/CD, hệ thống tích hợp liên tục đóng vai trò như “trung tâm điều phối” toàn bộ hoạt động tự động hóa, bảo đảm mọi thay đổi mã nguồn đều được kiểm tra có hệ thống, nhất quán và không gây gián đoạn cho môi trường triển khai. Đối với đề tài *Nghiên cứu xây dựng hệ thống phần mềm tự động hóa CI/CD trong quản lý và rà quét mã nguồn bảo mật tại Trường Đại học Kỹ thuật - Hậu cần CAND*, nền tảng CI không chỉ giúp chuẩn hóa quy trình build và kiểm thử, mà còn là nền tảng để tích hợp các công cụ phân tích bảo mật, từ đó tạo ra một quy trình kiểm soát chất lượng phần mềm toàn diện, tự động và đáng tin cậy.

1.4.3. Module phân tích và rà quét bảo mật mã nguồn.

Trong quy trình CI/CD hiện đại, việc bảo đảm an toàn mã nguồn ngay từ giai đoạn phát triển là yêu cầu bắt buộc nhằm giảm thiểu rủi ro bảo mật và nâng cao chất lượng phần mềm. Module phân tích và rà quét bảo mật mã nguồn được xem là một thành phần trọng yếu của mô hình DevSecOps, trong đó SonarQube và Trivy là hai công cụ tiêu biểu, thường xuyên được sử dụng trong các hệ thống CI/CD quy mô lớn.

SonarQube là nền tảng phân tích mã nguồn tĩnh (Static Application Security Testing - SAST) mạnh mẽ, hỗ trợ đánh giá chất lượng mã dựa trên hàng nghìn quy tắc kiểm tra. Công cụ này giúp phát hiện lỗi lập trình, vi phạm chuẩn coding conventions, các mùi mã (code smells) và đặc biệt là các lỗ hổng bảo mật theo các bộ tiêu chuẩn quốc tế như OWASP Top 10, CWE/SANS Top 25 và CERT. SonarQube vận hành dựa trên Static Analyzer Engine, bộ máy phân tích chuyên dụng được tối ưu theo từng ngôn ngữ lập trình, từ đó cung cấp báo cáo chi tiết về cấu trúc mã nguồn, mức độ phức tạp, khả năng mở rộng và tính an toàn. Bên cạnh đó, Vulnerability Database của SonarQube liên tục được cập nhật, bảo đảm khả năng phát hiện sớm các lỗ hổng mới nổi. Đặc biệt, hệ thống Quality Gate cho phép đặt ra các tiêu chí buộc mã nguồn phải đáp ứng (ví dụ: không có lỗi mức Critical/Blocker, tỷ lệ test coverage tối thiểu...), từ đó đảm bảo tính minh bạch và an toàn trước khi mã được chấp nhận để merge hoặc triển khai. Giao diện Report Dashboard trực quan giúp lập trình viên dễ dàng xác định vị trí lỗi, mức độ rủi ro và phương án khắc phục.



TRIVY

Hình 1.7. Kết quả đánh giá từ 3 phần mềm quét

Trong khi đó, Trivy là công cụ quét bảo mật đa chức năng, chuyên sâu vào phân tích hình ảnh Docker và môi trường container. Trivy thực hiện kiểm tra lỗ hổng (vulnerabilities) dựa trên cơ sở dữ liệu CVE được cập nhật liên tục, hỗ trợ quét các thành phần như: hệ điều hành trong container, thư viện phụ thuộc ứng dụng, cấu hình Dockerfile hay các gói phần mềm bên thứ ba. Khả năng quét nhanh, chính xác và tích hợp dễ dàng khiến Trivy trở thành lựa chọn phổ biến trong quy trình DevSecOps. Trivy đặc biệt quan trọng trong bối cảnh các ứng dụng hiện đại được đóng gói dưới dạng container, khi lỗ hổng trong một package hệ điều hành có thể ảnh hưởng trực tiếp đến an toàn của toàn bộ ứng dụng. Nhờ việc tích hợp Trivy vào pipeline CI/CD, mọi hình ảnh Docker trước khi đẩy lên registry đều được xác minh an toàn, hạn chế tối đa nguy cơ bị khai thác sau triển khai.

Xét về cấu trúc module, cả SonarQube và Trivy đều đóng góp vào một hệ thống gồm bốn thành phần chính:

- (1) Static Analyzer Engine: bộ xử lý phân tích mã nguồn (SonarQube) hoặc thành phần container/hệ điều hành (Trivy).
- (2) Vulnerability Database: kho dữ liệu lỗ hổng CVE, coding rules, OWASP Top 10, CWE.
- (3) Quality Gate / Severity Threshold: tiêu chí đánh giá mức an toàn của mã hoặc container.
- (4) Report Dashboard: giao diện hiển thị kết quả phân tích, mức độ rủi ro và hướng dẫn khắc phục.

Trong hệ thống tự động hóa CI/CD tại Trường Đại học Kỹ thuật - Hậu cần CAND, module phân tích bảo mật này giữ vai trò trung tâm trong pha “Security Verification”. SonarQube đảm nhiệm đánh giá chất lượng và tính an toàn của mã nguồn, trong khi Trivy chịu trách nhiệm kiểm tra các lỗ hổng của image sau khi build. Sự kết hợp của hai công cụ hình thành quy trình kiểm thử bảo mật nhiều lớp, giúp phát hiện sớm rủi ro ngay trong pipeline, đảm bảo rằng mọi sản phẩm phần mềm trước khi triển khai đều được kiểm soát an toàn chặt chẽ và tuân thủ đúng yêu cầu bảo mật trong môi trường giáo dục và nghiên cứu của Nhà trường.

1.4.4. Module quản lý thư viện, đóng gói và ảo hóa.

Trong hệ sinh thái CI/CD hiện đại, một thành phần không thể thiếu để đảm bảo tính ổn định và khả năng tái lập của toàn bộ pipeline chính là module quản lý thư viện và đóng gói. Đây là nơi tập trung lưu trữ tất cả các gói phụ thuộc (packages), thư viện, artifact build và các Docker image được tạo ra trong suốt quá trình phát triển. Hai nền tảng tiêu biểu thường được triển khai là Nexus Repository

và GitLab Package Registry, đóng vai trò như “kho trung tâm” (central artifact hub) giúp hệ thống DevOps vận hành ổn định, có kiểm soát và an toàn.

Trong quá trình phát triển, tất cả mã nguồn khi được build đều tạo ra artifact tương ứng. Những artifact này cần được lưu trữ tập trung để pipeline có thể tái sử dụng ở các bước tiếp theo, tránh trường hợp build lại từ đầu hoặc lệ thuộc vào môi trường bên ngoài. Chính vì vậy, Package Registry là lớp đầu tiên giữ chức năng lưu trữ các gói thư viện theo từng ngôn ngữ như Maven (Java), npm (NodeJS), PyPI (Python), NuGet (.NET)... Nexus Repository hỗ trợ mạnh mẽ nhất thông qua hệ thống repository đa dạng và khả năng tạo repository dạng *proxy*, cho phép cache các thư viện từ internet, giúp pipeline không bị gián đoạn khi nguồn ngoài gặp sự cố. Trong khi đó, GitLab Package Registry tích hợp trực tiếp vào từng dự án GitLab, giúp luồng quản lý CI/CD gắn kết hơn khi artifact luôn đi kèm với repository mã nguồn.

Việc quản lý Docker Image cũng đóng vai trò quan trọng trong các kiến trúc hiện đại. Docker Registry trong Nexus hay GitLab Container Registry đều cung cấp khả năng lưu trữ, phiên bản hóa image và cho phép pipeline tự động đẩy/pull image theo từng phiên bản. GitLab Container Registry đặc biệt tối ưu khi kết hợp với GitLab CI/CD, tự động tạo image trong mỗi pipeline và gắn tag tương ứng với commit, release hay branch. Ngược lại, Nexus lại phù hợp với mô hình triển khai doanh nghiệp đòi hỏi tách biệt rõ ràng giữa các nhóm, dự án và mức độ phân quyền cao.

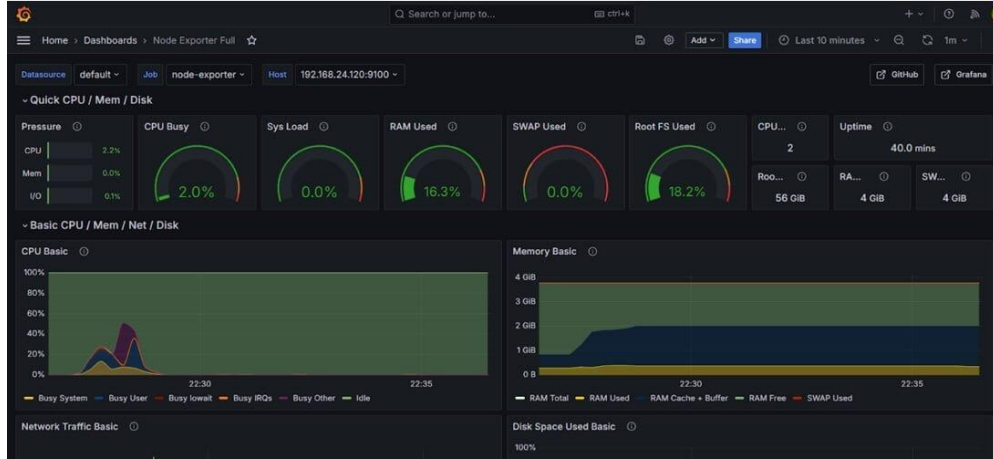
Nhờ thiết kế như vậy, module quản lý thư viện và đóng gói trở thành xương sống của CI/CD, bảo đảm các artifact được kiểm soát, phiên bản hóa, an toàn và luôn sẵn sàng cho các pipeline triển khai. Đây cũng là thành phần quan trọng trong DevSecOps, đặc biệt khi kết hợp với các công cụ quét lỗ hổng thư viện như Trivy hoặc Nexus IQ/OSS Index, giúp phát hiện sớm lỗi bảo mật trong dependency trước khi artifact được phát hành ra môi trường vận hành.

1.4.5. Module giám sát và theo dõi hoạt động.

Trong kiến trúc CI/CD hiện đại, khả năng giám sát liên tục và phân tích hoạt động hệ thống đóng vai trò đặc biệt quan trọng nhằm đảm bảo pipeline vận hành ổn định, kịp thời phát hiện sự cố và tối ưu hóa tài nguyên. Module giám sát và theo dõi hoạt động, với hai công nghệ tiêu biểu là Prometheus và Grafana, được triển khai như một lớp quan sát (*observability layer*) cho toàn bộ nền tảng DevOps.

Prometheus đảm nhiệm vai trò thu thập và lưu trữ dữ liệu metrics theo thời gian thực, từ hiệu năng máy chủ, tình trạng container, tốc độ phản hồi của dịch vụ, cho đến các chỉ số liên quan trực tiếp tới CI/CD pipeline như thời gian chạy job, tỷ lệ lỗi, số lượng yêu cầu, hay mức tải của runner. Thông qua Metrics Collector, Prometheus chủ động “kéo” dữ liệu từ các exporter được cài đặt trên

từng thành phần của hệ thống (máy chủ, Docker, Kubernetes, Runner CI/CD...), đảm bảo dữ liệu giám sát luôn chính xác, nhất quán và có khả năng mở rộng. Các sự cố hoặc bất thường được phát hiện ngay lập tức nhờ Alert Manager, nơi thiết lập các quy tắc cảnh báo tự động dựa trên ngưỡng hoạt động.



Hình 1.8. Quản lý thông tin máy chủ server

Grafana là lớp trình bày dữ liệu trực quan, xây dựng Visualization Dashboard trực tiếp từ nguồn dữ liệu của Prometheus và các hệ thống log. Nhờ giao diện trực quan, người quản trị có thể theo dõi tức thời tình trạng CI/CD pipeline, mức sử dụng tài nguyên, lưu lượng truy cập, số lỗi build/test, hay mức độ ổn định của service sau mỗi lần deploy. Các dashboard được thiết kế linh hoạt, hỗ trợ người vận hành đưa ra quyết định tối ưu hệ thống, điều chỉnh tài nguyên hoặc cải thiện hiệu suất pipeline.

Với sự kết hợp giữa Prometheus và Grafana, module giám sát và theo dõi hoạt động trở thành yếu tố không thể thiếu trong mô hình DevSecOps, không chỉ giúp phát hiện sự cố sớm mà còn hỗ trợ phân tích hiệu năng, đánh giá xu hướng và bảo đảm hệ thống CI/CD hoạt động bền vững, ổn định và có khả năng mở rộng theo thời gian.

1.5. Ý nghĩa và kết quả mang lại của đề án

Đề án “Nghiên cứu xây dựng hệ thống phần mềm tự động hóa CI/CD trong quản lý và rà quét mã nguồn bảo mật tại Trường Đại học Kỹ thuật - Hậu cần CAND” mang ý nghĩa quan trọng cả về mặt khoa học lẫn thực tiễn triển khai trong môi trường đào tạo và nghiên cứu công nghệ.

Về ý nghĩa khoa học, đề án góp phần hệ thống hóa cơ sở lý thuyết về mô hình CI/CD kết hợp DevSecOps, làm rõ vai trò của tự động hóa trong toàn bộ vòng đời phát triển phần mềm hiện đại. Việc tích hợp các công cụ như Jenkins, SonarQube và Trivy vào một quy trình thống nhất giúp minh chứng cách thức áp dụng các kỹ thuật phân tích mã nguồn tĩnh, kiểm thử tự động và rà quét lỗ hổng bảo mật ngay trong pipeline phát triển. Điều này tạo cơ sở nghiên cứu cho các

hướng mở rộng như tự động hóa kiểm thử bảo mật nâng cao, quản lý chất lượng phần mềm theo chuẩn quốc tế, và xây dựng môi trường phát triển phần mềm an toàn theo hướng DevSecOps.

Về ý nghĩa thực tiễn, hệ thống CI/CD được xây dựng giúp chuẩn hóa quy trình phát triển và kiểm soát mã nguồn tại Nhà trường. Thay vì kiểm thử và rà soát thủ công, toàn bộ quá trình build, test, phân tích chất lượng mã và quét lỗ hổng bảo mật được thực hiện tự động, liên tục và có thể lặp lại. Điều này giúp phát hiện sớm lỗi lập trình, cấu hình sai hoặc lỗ hổng nghiêm trọng trong thư viện và Docker image trước khi đưa vào môi trường triển khai. Nhờ đó, chất lượng phần mềm được nâng cao, rủi ro bảo mật giảm đáng kể, đồng thời tiết kiệm thời gian và công sức cho đội ngũ phát triển.

Đồ án mang lại kết quả cụ thể về mặt hệ thống, bao gồm việc xây dựng thành công một pipeline CI/CD hoàn chỉnh có khả năng: tự động lấy mã nguồn từ hệ thống quản lý phiên bản, thực hiện build và kiểm thử bằng Maven, phân tích tĩnh mã nguồn bằng SonarQube, đóng gói ứng dụng thành Docker image và tiến hành quét bảo mật bằng Trivy với cơ chế ngưỡng cảnh báo. Hệ thống cũng hỗ trợ lưu trữ báo cáo và theo dõi lịch sử pipeline, giúp cán bộ quản lý dễ dàng giám sát chất lượng và mức độ an toàn của mã nguồn theo từng phiên bản.

Tổng thể, đồ án xây dựng được mô hình triển khai thực tế, có khả năng áp dụng vào môi trường đào tạo và phát triển phần mềm tại Nhà trường. Đây là nền tảng quan trọng để tiến tới xây dựng môi trường phát triển phần mềm an toàn, tự động hóa cao và phù hợp với xu hướng chuyển dịch sang DevSecOps trong các hệ thống công nghệ thông tin hiện đại.

1.6. Kết luận chương 1

Trong Chương 1, đồ án đã trình bày cơ sở lý luận và bối cảnh chung liên quan đến việc nghiên cứu xây dựng hệ thống tự động hóa CI/CD phục vụ quản lý và rà quét mã nguồn bảo mật tại Trường Đại học Kỹ thuật - Hậu cần CAND. Chương này đã hệ thống hóa các khái niệm nền tảng về tự động hóa, tích hợp liên tục (CI), triển khai liên tục (CD) và mô hình CI/CD mở rộng theo hướng DevSecOps. Đồng thời, các nội hàm quan trọng như quản lý mã nguồn, tự động kiểm thử, phân tích bảo mật, đóng gói và triển khai ứng dụng đã được làm rõ dưới góc độ kỹ thuật.

Bên cạnh phần lý thuyết, chương cũng phân tích thực trạng về hạ tầng kỹ thuật hiện có tại Nhà trường, mức độ sẵn sàng trong việc triển khai hệ thống tự động hóa, đồng thời đánh giá các yêu cầu thực tiễn trong công tác quản lý mã nguồn, kiểm thử và đảm bảo an toàn thông tin. Các công nghệ phổ biến trong hệ sinh thái CI/CD - bao gồm hệ thống SCM, công cụ CI/CD pipeline, nền tảng container hóa, kho lưu trữ artifact, và các công cụ quét bảo mật SAST - đã được

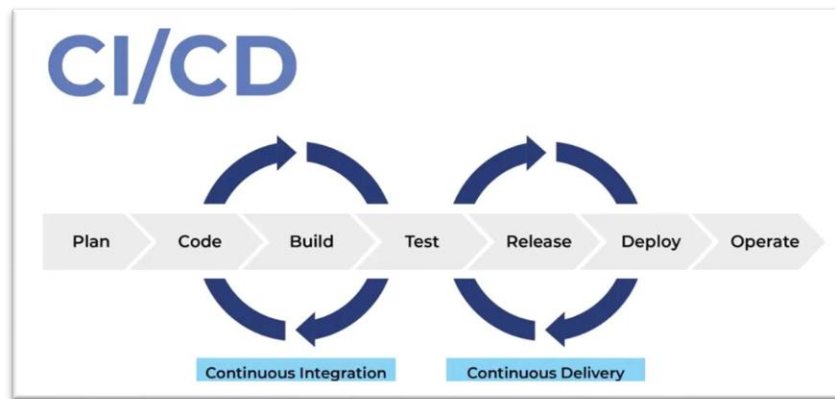
trình bày và phân tích chi tiết nhằm làm rõ khả năng ứng dụng trong môi trường của Nhà trường.

Nhờ vậy, Chương 1 đóng vai trò là nền tảng lý thuyết và thực tiễn vững chắc, giúp định hình yêu cầu, phạm vi và hướng tiếp cận của đề tài. Những cơ sở này sẽ là tiền đề quan trọng cho quá trình phân tích, đặc tả yêu cầu và thiết kế hệ thống tự động hóa CI/CD trình bày trong Chương 2.

CHƯƠNG 2. PHÂN TÍCH QUÁ TRÌNH XÂY DỰNG TÍCH HỢP VÀ TRIỂN KHAI HỆ THỐNG.

2.1. Tổng quan về yêu cầu hệ thống.

Đồ án tập trung nghiên cứu và xây dựng hệ thống tích hợp và triển khai liên tục (CI/CD) nhằm tự động hóa toàn bộ quy trình phát triển phần mềm, từ quản lý mã nguồn, tích hợp liên tục, kiểm thử chất lượng, rà quét bảo mật đến triển khai và giám sát vận hành. Hệ thống CI/CD được thiết kế theo hướng tập trung, chuẩn hóa và có khả năng mở rộng, giúp kiểm soát chặt chẽ vòng đời phần mềm và giảm thiểu rủi ro trong quá trình phát triển và triển khai.



Hình 2.1. Hệ thống tích hợp và triển khai liên tục (CI/CD)

Quy trình CI/CD được tổ chức xoay quanh kho quản lý mã nguồn tập trung, nơi mọi thay đổi đều được kiểm soát phiên bản, phân quyền truy cập và tự động kích hoạt pipeline tích hợp liên tục. Các bước xây dựng và đóng gói ứng dụng được thực hiện hoàn toàn tự động, đảm bảo tính nhất quán giữa mã nguồn, sản phẩm triển khai và môi trường vận hành. Việc tự động hóa giúp rút ngắn thời gian phản hồi khi phát sinh lỗi và hạn chế tối đa các sai sót khi thao tác bằng phương pháp thủ công.

Hệ thống CI/CD tích hợp các cơ chế kiểm thử và đánh giá chất lượng trong từng giai đoạn pipeline. Phân tích mã nguồn tĩnh được sử dụng để kiểm soát chất lượng lập trình và tuân thủ tiêu chuẩn kỹ thuật, trong khi phân tích động được áp dụng nhằm phát hiện các lỗi hỏng bảo mật tồn tại trong thư viện phụ thuộc và môi trường thực thi. Kết quả kiểm thử được tổng hợp và sử dụng làm căn cứ quyết định cho phép hoặc dừng quá trình triển khai.

Sau khi vượt qua các bước kiểm soát chất lượng và bảo mật, sản phẩm phần mềm được đóng gói, lưu trữ và triển khai tự động theo các kịch bản đã định nghĩa. Quy trình triển khai đảm bảo tính liên tục, khả năng kiểm soát và phục hồi khi xảy ra sự cố, đồng thời hỗ trợ giám sát hoạt động nhằm duy trì tính ổn định và sẵn sàng của hệ thống.

2.2. Sơ đồ hóa quy trình nghiệp vụ.

2.2.1. Danh sách dịch vụ

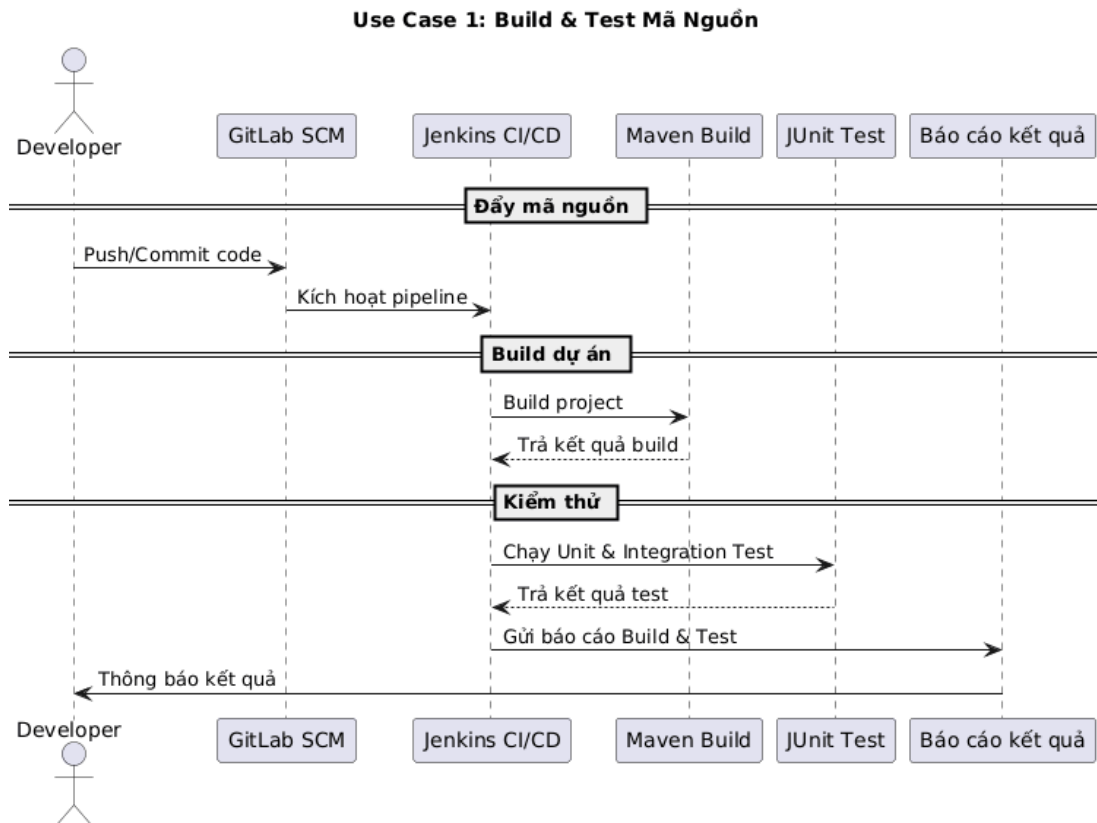
Bảng 2.1. Danh sách module và mục đích sử dụng

TT	Module	Mục đích sử dụng
1	Module đồng bộ mã nguồn;	Dịch vụ cho phép hệ thống CI/CD tự động đồng bộ mã nguồn từ GitLab/GitHub về máy chủ CI mỗi khi có thay đổi mới. Đây là bước khởi đầu trong toàn bộ pipeline, đảm bảo mã nguồn luôn cập nhật chính xác để phục vụ phân tích bảo mật và build ứng dụng.
2	Module phân tích bảo mật mã nguồn tĩnh;	Chịu trách nhiệm thu thập và phân tích mã nguồn theo các chuẩn an toàn thông tin (OWASP, CWE). Dịch vụ tự động phát hiện lỗi lập trình, điểm yếu bảo mật, code smells và áp dụng Quality Gate nhằm ngăn chặn mã nguồn không đạt chuẩn đi vào giai đoạn triển khai.
3	Module quét bảo mật container và thư viện.	Dịch vụ này đảm nhiệm quét image Docker, thư viện phụ thuộc và package đi kèm ứng dụng nhằm phát hiện lỗ hổng mức HIGH/CRITICAL. Kết quả quét được chuẩn hóa thành báo cáo JSON, HTML hoặc PDF, hỗ trợ đánh giá rủi ro trước khi triển khai ứng dụng.

2.2.2. Đặc tả usecase CICD.

2.2.2.1. Biểu đồ tuần tự.

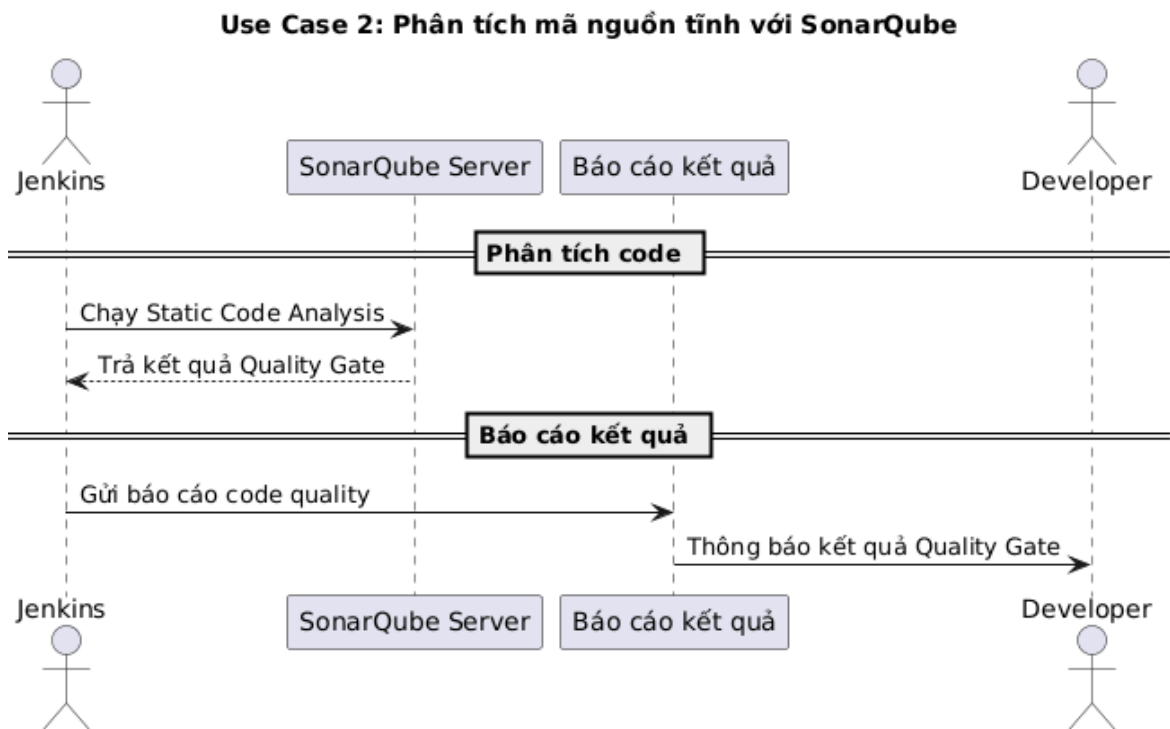
Use Case 1: Build & Test Mã Nguồn



Hình 2.2. Usecase build và triển khai mã nguồn

Quy trình bắt đầu từ hành động của lập trình viên khi đẩy mã nguồn (commit/push) lên GitLab SCM. GitLab tự động kích hoạt pipeline trên Jenkins CI/CD, nơi dự án được xây dựng bằng Maven. Sau khi build thành công, JUnit thực hiện các bài kiểm thử đơn vị và tích hợp, nhằm phát hiện lỗi lập trình và đảm bảo tính ổn định của mã nguồn. Kết quả của build và test được gửi về module báo cáo, giúp lập trình viên nắm bắt nhanh các vấn đề và có thể phản hồi ngay lập tức, giảm thiểu rủi ro phát sinh từ việc tích hợp chậm.

Use Case 2: Tiến trình phân tích mã nguồn tĩnh

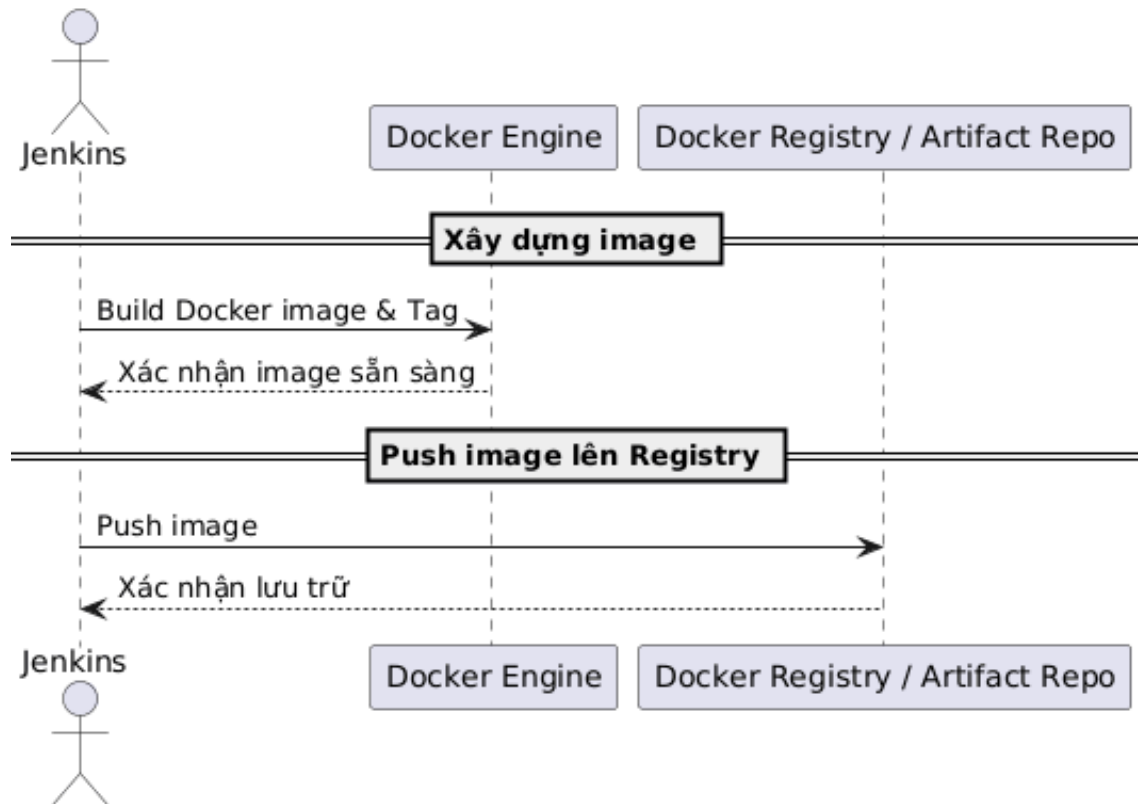


Hình 2.3. Tiến trình phân tích mã nguồn tĩnh SonarQube

Sau khi build và test, pipeline tiếp tục thực hiện phân tích tĩnh mã nguồn thông qua SonarQube. Công cụ này rà soát code theo các tiêu chuẩn chất lượng và bảo mật, xác định code smells, lỗ hổng bảo mật và kiểm tra tuân thủ các quy tắc lập trình. SonarQube cung cấp Quality Gate, là bộ tiêu chí bắt buộc để mã nguồn được merge hoặc tiếp tục quá trình triển khai. Kết quả phân tích được tổng hợp và gửi đến lập trình viên, đảm bảo rằng bất kỳ vấn đề nào về bảo mật hoặc chất lượng đều được phát hiện sớm và xử lý kịp thời, từ đó nâng cao tính ổn định và an toàn của hệ thống.

Use Case 3: Build & Tag Docker Image

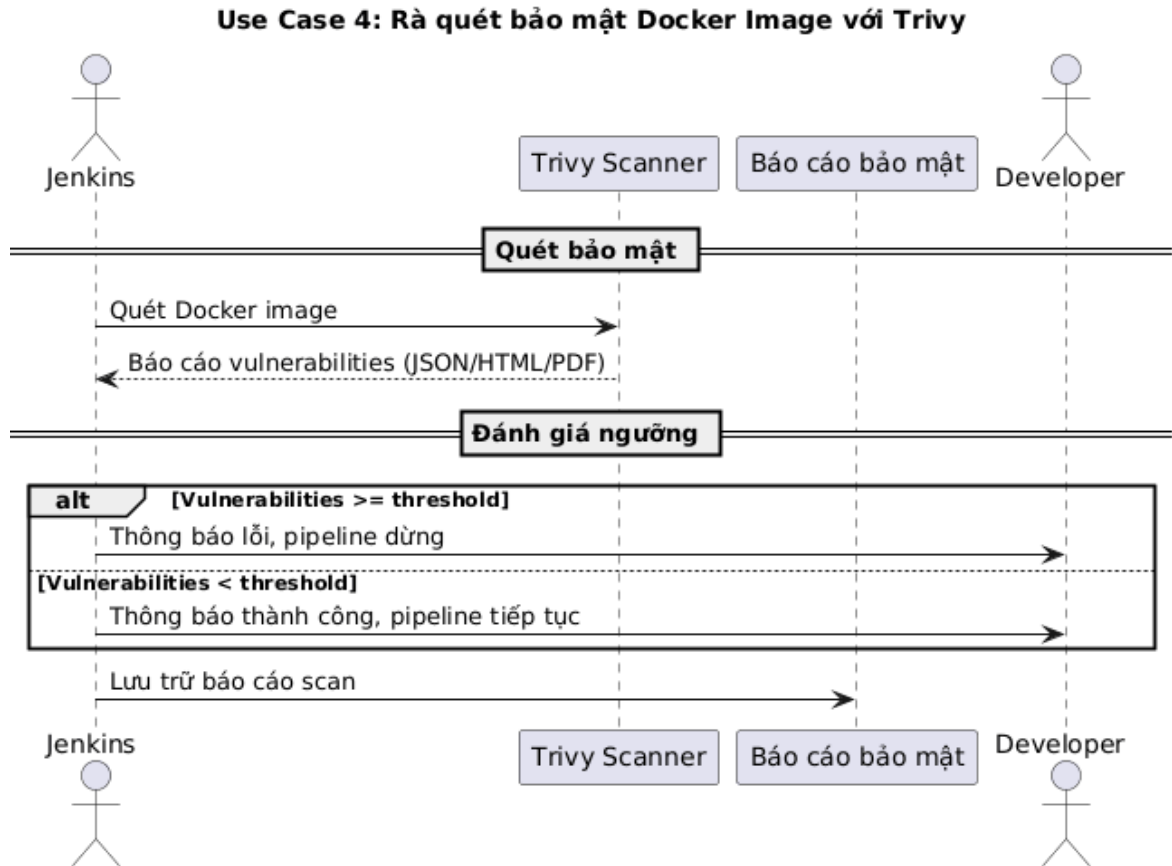
Use Case 3: Build & Tag Docker Image



Hình 2.4. Usecase build và tag Docker Image

Khi mã nguồn đã vượt qua các kiểm thử và Quality Gate, Jenkins tiến hành xây dựng Docker image, đóng gói ứng dụng cùng các phụ thuộc cần thiết, và gán tag dựa trên số build để dễ dàng quản lý phiên bản. Image sau đó được push lên Docker Registry, đảm bảo môi trường triển khai đồng nhất và có thể tái lập. Module này đóng vai trò quan trọng trong việc chuẩn hóa môi trường triển khai, giảm thiểu các xung đột do sự khác biệt giữa môi trường phát triển và môi trường production.

Use Case 4: Rà quét bảo mật.



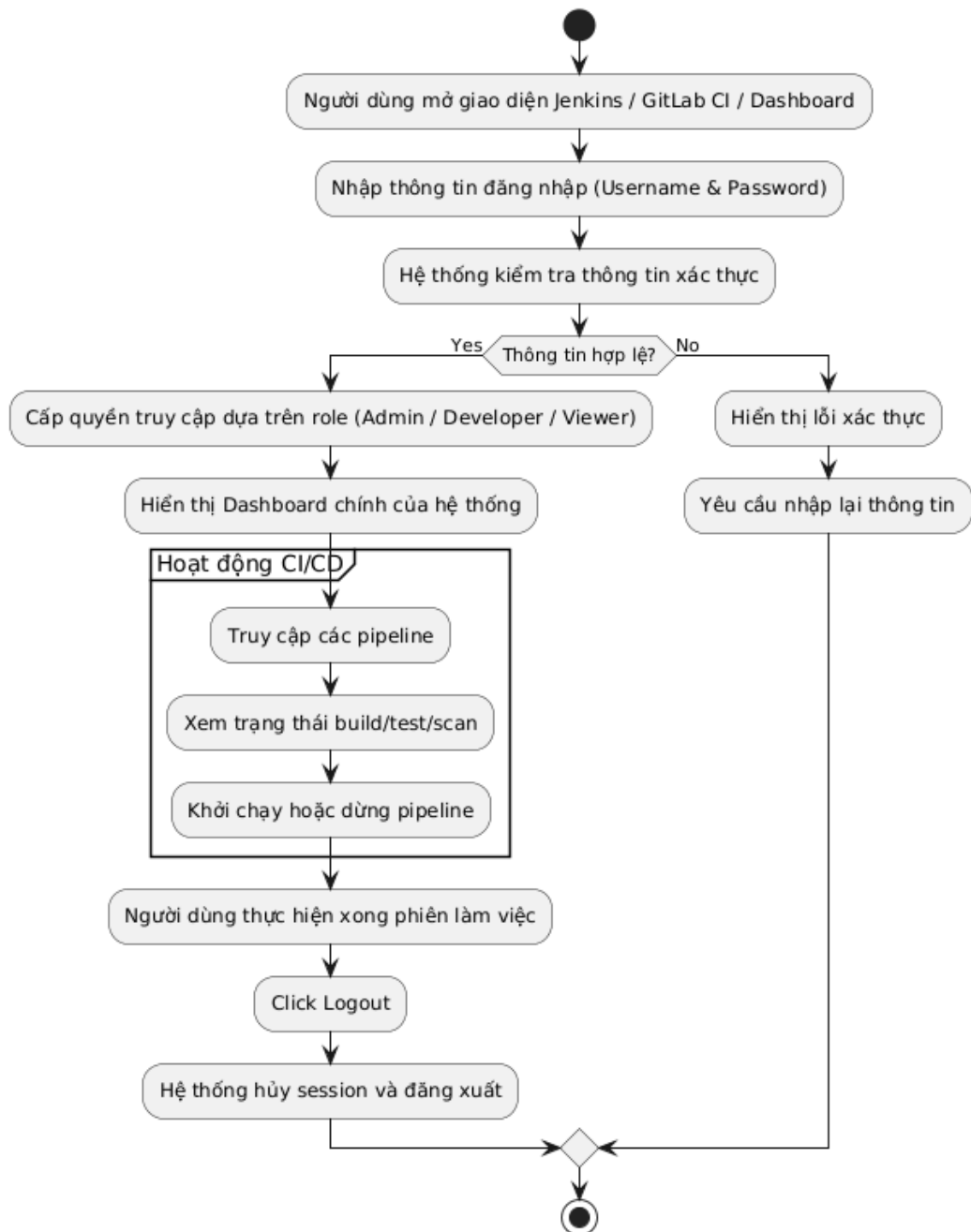
Hình 2.5. Use Case rà quét bảo mật Docker Image với Trivy.

Giai đoạn cuối cùng của pipeline là quét bảo mật Docker image bằng Trivy, nhằm phát hiện các lỗ hổng nghiêm trọng (HIGH/CRITICAL) trong image và thư viện phụ thuộc. Trivy tạo báo cáo dưới nhiều định dạng (JSON, HTML, PDF) để phục vụ kiểm tra và lưu trữ. Nếu số lượng lỗ hổng vượt ngưỡng cho phép, pipeline sẽ dừng và thông báo cho lập trình viên, đảm bảo rằng chỉ các bản build an toàn mới được triển khai. Nếu không vượt ngưỡng, pipeline tiếp tục và báo cáo được lưu trữ, giúp duy trì tính liên tục và an toàn trong quy trình DevSecOps.

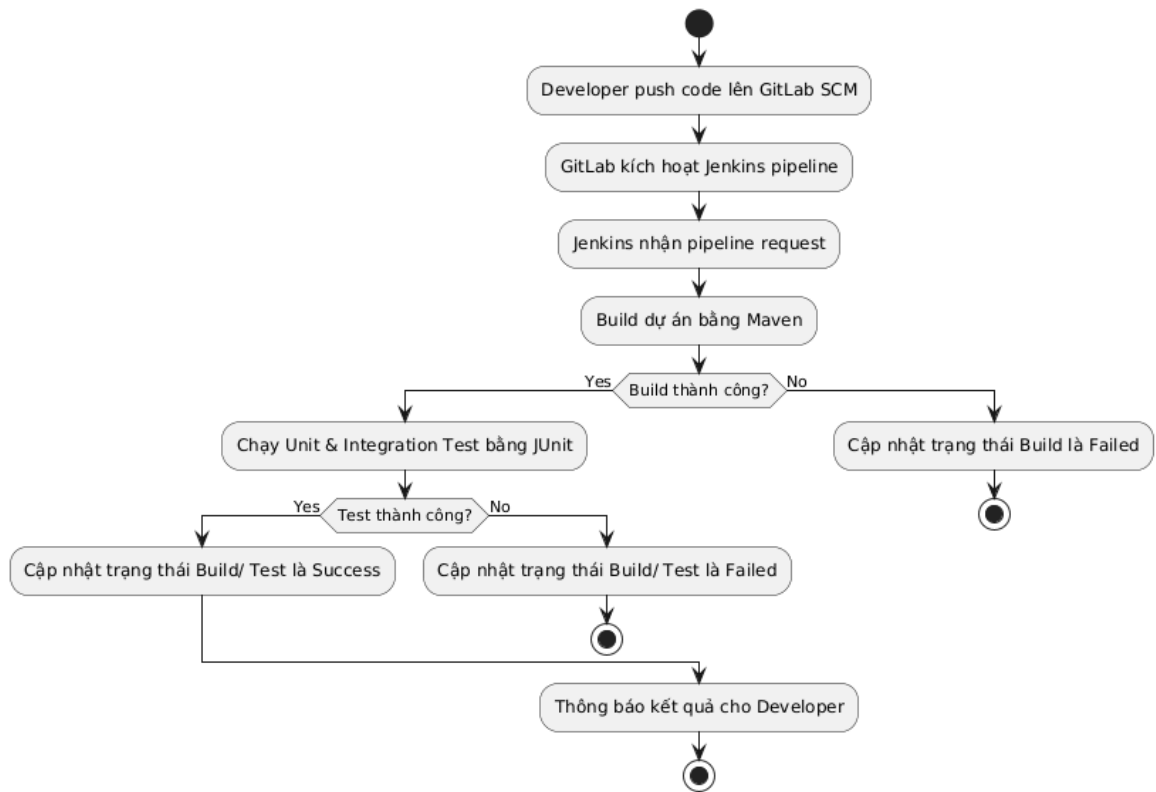
Như vậy, bốn use case này liên kết chặt chẽ, tạo thành một chuỗi CI/CD khép kín và tự động: từ việc đẩy code, build/test, phân tích code tĩnh, đóng gói Docker, đến kiểm tra bảo mật. Mỗi bước đều đảm bảo rằng mã nguồn luôn ổn định, chất lượng cao và an toàn, đồng thời hỗ trợ phản hồi tức thời cho lập trình viên và quản lý các artifact một cách chuẩn hóa. Cấu trúc này tạo nền tảng vững chắc cho việc triển khai hệ thống tự động hóa CI/CD trong quản lý và rà quét mã nguồn bảo mật tại Nhà trường.

2.2.2.2. Biểu đồ hoạt động.

- Biểu đồ đăng nhập và đăng xuất Hệ thống CI/CD:

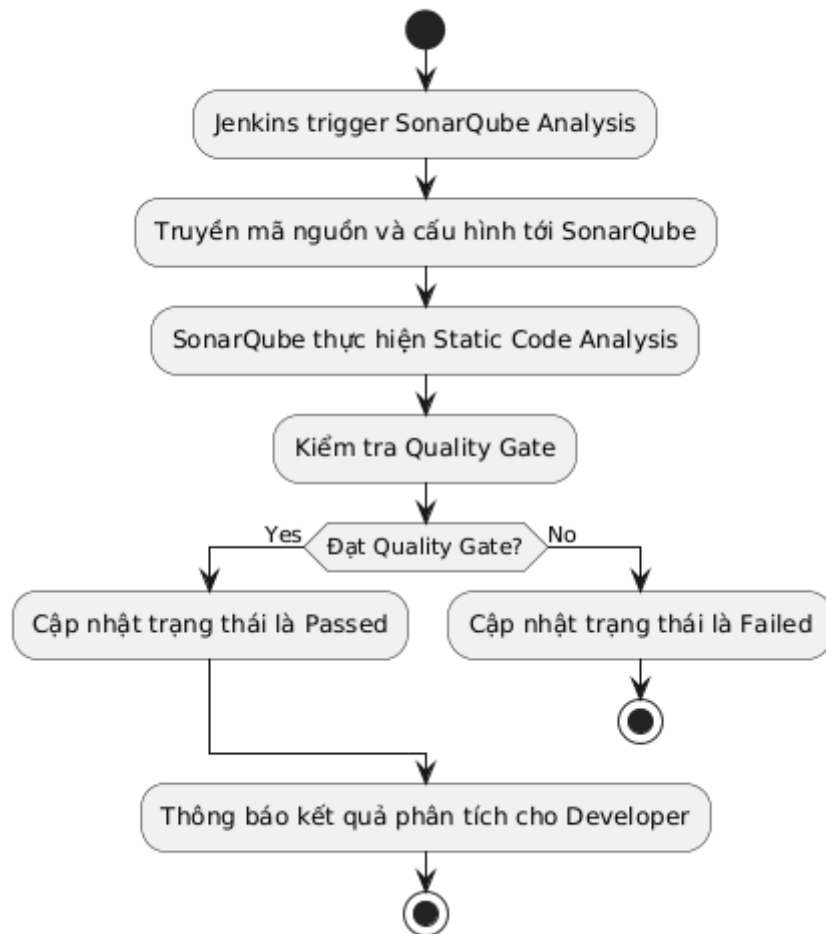
Activity Diagram: Đăng nhập & Đăng xuất Hệ thống CI/CD**Hình 2.6.** Biểu đồ đăng nhập và đăng xuất Hệ thống CI/CD

- Biểu đồ hoạt động xây dựng mã nguồn:

Biểu đồ hoạt động: Build & Test Mã Nguồn**Hình 2.7.** Biểu đồ hoạt động xây dựng mã nguồn

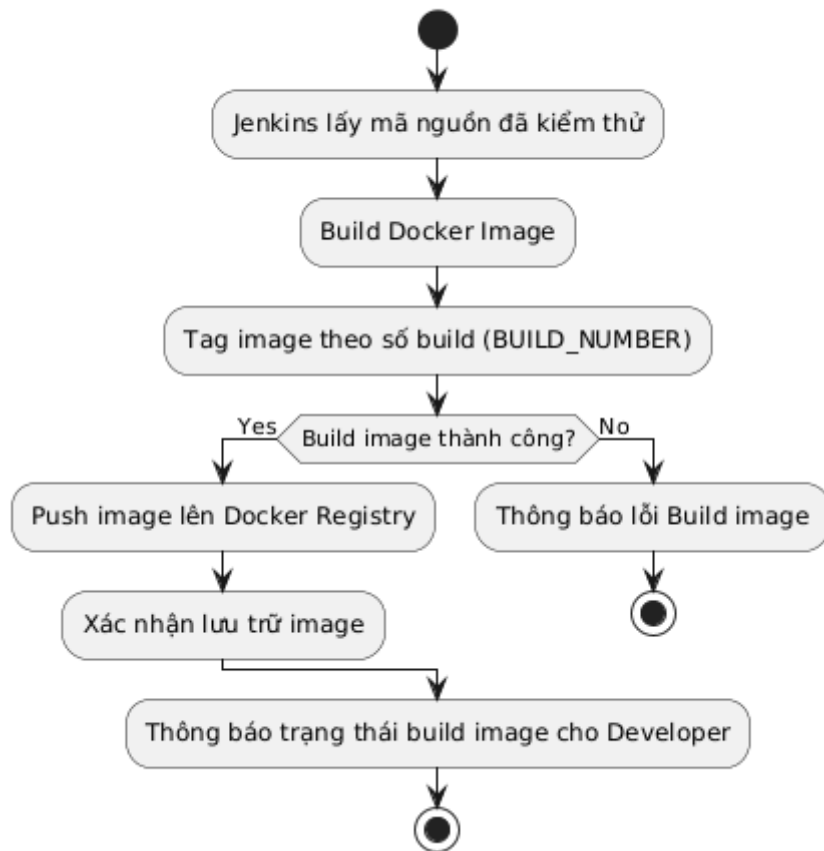
Giải thích: Biểu đồ hoạt động này mô tả luồng tự động từ khi lập trình viên đẩy code lên SCM, Jenkins nhận trigger, tiến hành build bằng Maven, và chạy JUnit test. Mọi kết quả đều được báo cáo lại cho developer, giúp phát hiện sớm lỗi build hoặc test, đảm bảo tính ổn định cho các bước tiếp theo.

- Biểu đồ Phân tích mã nguồn tĩnh (SonarQube):

Biểu đồ hoạt động: Phân tích mã nguồn tĩnh**Hình 2.8.** Biểu đồ Phân tích mã nguồn tĩnh

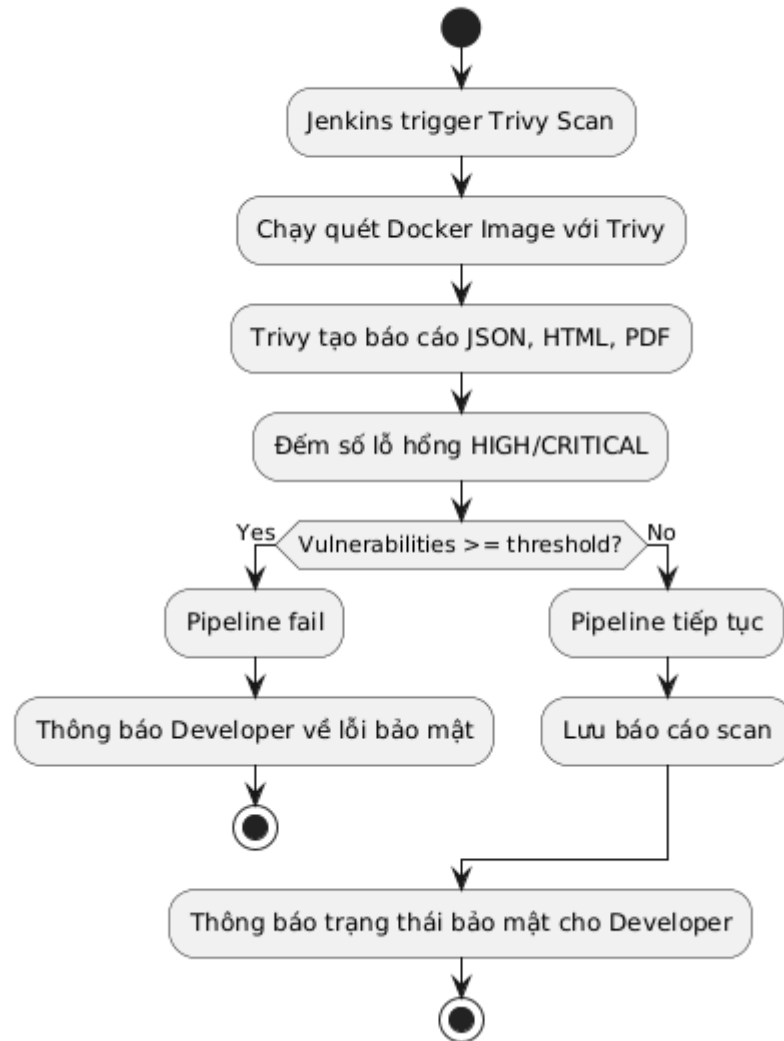
Giải thích: Biểu đồ này mô tả luồng phân tích code tĩnh, từ việc Jenkins gửi mã nguồn sang SonarQube, phân tích code theo Quality Gate và báo cáo kết quả. Đây là bước then chốt đảm bảo code tuân thủ các tiêu chuẩn chất lượng và bảo mật, giảm rủi ro trước khi đóng gói Docker.

- Biểu đồ Build & Tag Docker Image:

Biểu đồ hoạt động: Build & Tag Docker Image**Hình 2.9.** Biểu đồ build và tag Docker Image

Giải thích: Biểu đồ này mô tả luồng đóng gói Docker image, từ build, tag cho tới push lên Docker Registry. Module này đảm bảo môi trường triển khai đồng nhất, dễ quản lý phiên bản và chuẩn bị cho giai đoạn rà quét bảo mật.

- Biểu đồ Rà quét bảo mật Docker Image (Trivy):

Biểu đồ hoạt động: Rà quét bảo mật Docker Image với Trivy**Hình 2.10.** Biểu đồ rà quét bảo mật Docker Image

Giải thích: Biểu đồ này mô tả luồng rà quét bảo mật Docker image bằng Trivy. Pipeline sẽ đếm số lỗ hổng nghiêm trọng, nếu vượt ngưỡng, pipeline dừng và thông báo lập trình viên. Nếu không, pipeline tiếp tục, đồng thời lưu trữ báo cáo để đảm bảo tính an toàn trước khi triển khai production. Đây là bước quan trọng trong DevSecOps.

Kết nối 4 biểu đồ: Bốn biểu đồ hoạt động trên phản ánh quy trình tuần tự trong pipeline CI/CD:

1. Build & Test → 2. Static Analysis (SonarQube) → 3. Docker Build & Tag → 4. Security Scan (Trivy).

Tất cả các bước đều tự động hóa, kết hợp feedback tức thời cho developer và đảm bảo chất lượng, bảo mật, ổn định của phần mềm. Chuỗi workflow này

tạo nền tảng cho việc quản lý và rà soát mã nguồn bảo mật tại Trường Đại học Kỹ thuật - Hậu cần CAND.

2.3. Quy trình cài đặt môi trường CICD

2.3.1. Cài đặt docker

File docker.sh (build quyền root or tk docker)

```
#!/bin/bash

for pkg in docker.io docker-doc docker-compose docker-compose-v2 podman-
docker containerd runc; do sudo apt-get remove $pkg; done

# Add Docker's official GPG key:

sudo apt-get update

sudo apt-get install ca-certificates curl gnupg

sudo install -m 0755 -d /etc/apt/keyrings

curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --
dearmor -o /etc/apt/keyrings/docker.gpg

sudo chmod a+r /etc/apt/keyrings/docker.gpg

...

sudo groupadd docker

sudo usermod -aG docker $USER

newgrp docker

sudo chown "$USER":"$USER" /home/"$USER"/.docker -R

sudo chmod g+rx "$HOME/.docker" -R

sudo systemctl enable docker.service

sudo systemctl enable containerd.service
```

Tạo docker network

```
root@ubuntu-virtual-machine:/home/ubuntu/Desktop# docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
919275e7632b        bridge              bridge              local
1e01eb97cdd4        host                host                local
b167cbd13e67        jenkins-java        bridge              local
d42ed222014e        none                null                local
```

Hình 2.11. Tạo docker network

2.3.2. Cài đặt gitlab server

Bước 1: Cập nhật hệ thống và cài gói cần thiết

```
sudo apt update && sudo apt upgrade -y
```

```
sudo apt install -y curl openssh-server ca-certificates tzdata perl
```

Bước 2: Thêm GitLab APT Repository

```
curl https://packages.gitlab.com/install/repositories/gitlab/gitlab-ee/script.deb.sh
sudo bash
```

Bước 3: Cài đặt GitLab EE và set IP 192.168.68.248

```
sudo EXTERNAL_URL="http://192.168.68.248" apt install -y gitlab-ee
```



Hình 2.12. Cài đặt Gitlab thành công

2.3.3. Cài đặt Nexus Repository

1. Cài Java (OpenJDK 17)

```
apt update
apt install openjdk-17-jdk -y
```

Kiểm tra version:

```
java -version
```

2. Tải và giải nén Nexus

```
cd /opt
sudo wget https://download.sonatype.com/nexus/3/latest-unix.tar.gz
sudo tar -xvzf latest-unix.tar.gz
```



```
sudo mv nexus-3* nexus
```

3. Tạo user nexus

```
sudo useradd -M -d /opt/nexus -s /bin/false nexus
sudo chown -R nexus:nexus /opt/nexus /opt/sonatype-work
```

4. Chỉnh file `nexus.rc`

```
sudo nano /opt/nexus/bin/nexus.rc
```

Thêm dòng (hoặc bỏ dấu # nếu có sẵn):

```
run_as_user="nexus"
```

5. Tạo service systemd để chạy Nexus

```
sudo nano /etc/systemd/system/nexus.service
```

Nội dung:

```
[Unit]
Description=Nexus Repository Manager
After=network.target

[Service]
Type=forking
LimitNOFILE=65536
User=nexus
Group=nexus
ExecStart=/opt/nexus/bin/nexus start
ExecStop=/opt/nexus/bin/nexus stop
Restart=on-abort

[Install]
WantedBy=multi-user.target
```

6. Kích hoạt Nexus service

```
sudo systemctl daemon-reexec
sudo systemctl daemon-reload
sudo systemctl enable nexus
sudo systemctl start nexus
```

Kiểm tra trạng thái:

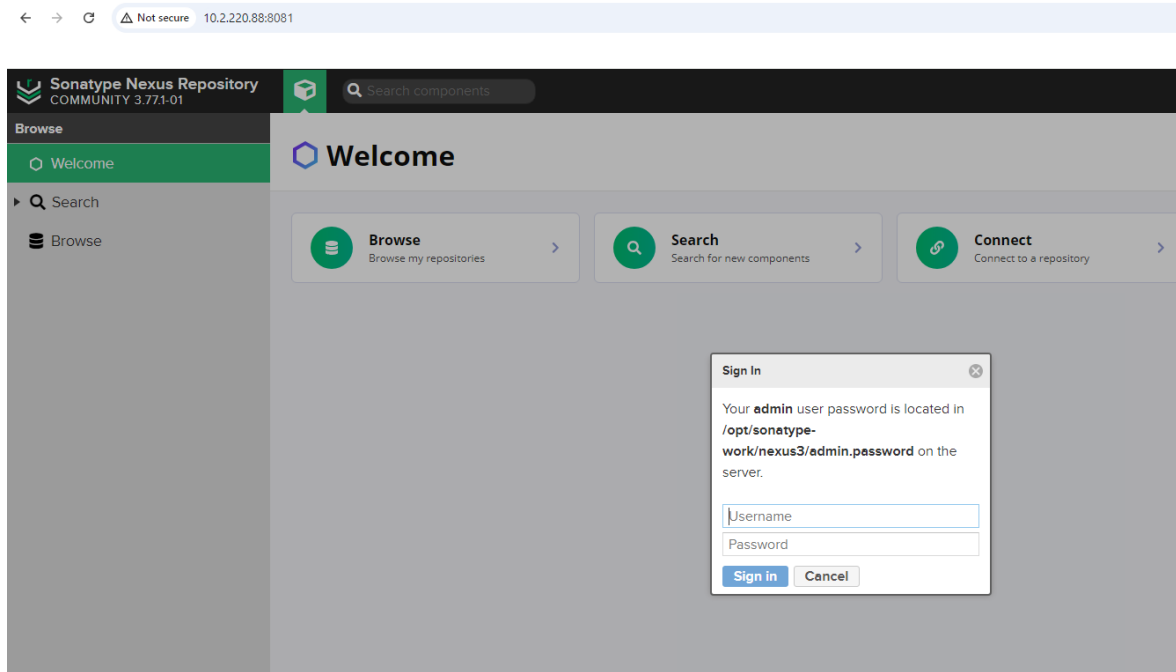
```
sudo systemctl status nexus
```

7. Mở port nếu dùng UFW

```
sudo ufw allow 8081/tcp
```

8. Đăng nhập Nexus

Truy cập trình duyệt: <http://<IP>:8081>



Hình 2.13. Thành công khi cài Nexus

2.3.4. Cài Jenkins

Tải thư viện jenkins và upload:

```
root@ubuntu-virtual-machine:/home/ubuntu/Desktop# docker run -d --name jenkins-dind \
-p 8080:8080 \
-p 50000:50000 \
-v /var/run/docker.sock:/var/run/docker.sock \
-v $(which docker):/usr/bin/docker \
-u root \
-e DOCKER_GID=$(getent group docker | cut -d: -f3) \
--network jenkins-java \
jenkins/jenkins:lts
```

Hình 2.14. Cài Jenkins thành công

Kiểm tra xem Jenkins Docker đã chạy ổn định chưa

```
docker ps
```

```
root@ubuntu-virtual-machine:/home/ubuntu/Desktop# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
7f989ac7df92   gitlab/gitlab-ce:latest             "/assets/init-conta..." 6 weeks ago   Up 2 weeks   443/tcp, 0.0.0.0:2224->22/tcp, [::]:2224->22/tcp, 0.0.0.0:8929->80/tcp, [::]:8
929->80/tcp    gitlab
62a6b3e8d3e3   sonarqube:latest                    "/opt/sonarqube/dock..." 6 weeks ago   Up 2 weeks   0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp
33fc00f17baf   jenkins/jenkins:lts                 "/usr/bin/tini -- /u..." 6 weeks ago   Up 2 weeks   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 0.0.0.0:50000->50000/tcp, [::]:50000-
50000/tcp     jenkins-dind
```

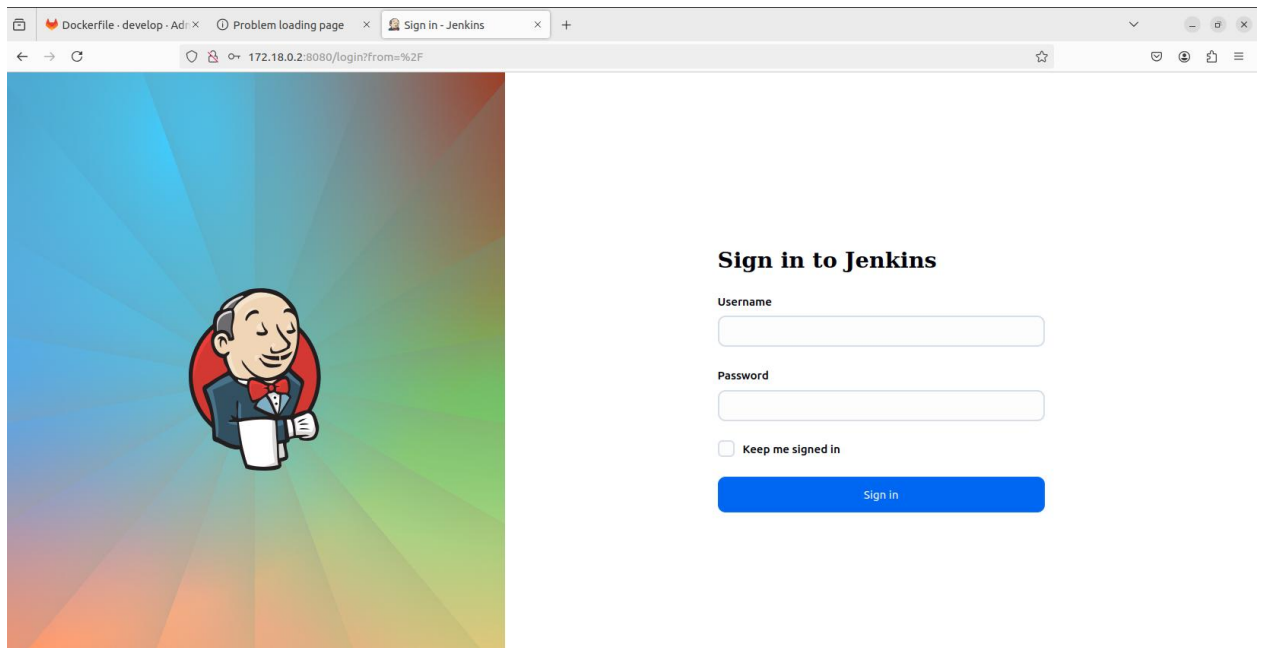
Lấy Jenkins Container IP Address

```
docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' jenkins-dind
```

```
root@ubuntu-virtual-machine:/home/ubuntu/Desktop# docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' jenkins-dind
172.18.0.2
```

On your browser, open

```
ContainerIP:8080
```



Thiết lập Docker trong vùng chứa Jenkins

Bên trong vùng chứa, chạy các lệnh sau:

```
docker --version
```

```
groupadd -for -g $DOCKER_GID docker
```

```
usermod -aG docker jenkins
```

```
exit
```

```
docker ps
```

```
docker restart jenkins-dind
```

Xong truy cập vào đường dẫn để add các credentials

```
http://192.168.68.247:8080/user/admin/credentials/
```

Credentials

T	P	Store	Domain	ID	Name
		System	(global)	jenkins-gcp	root/*****
		System	(global)	sonartoken	sonartoken
		System	(global)	ca062ffa-c4f8-4113-9d62-50b074d6c2b4	root/*****

Stores scoped to User: Admin

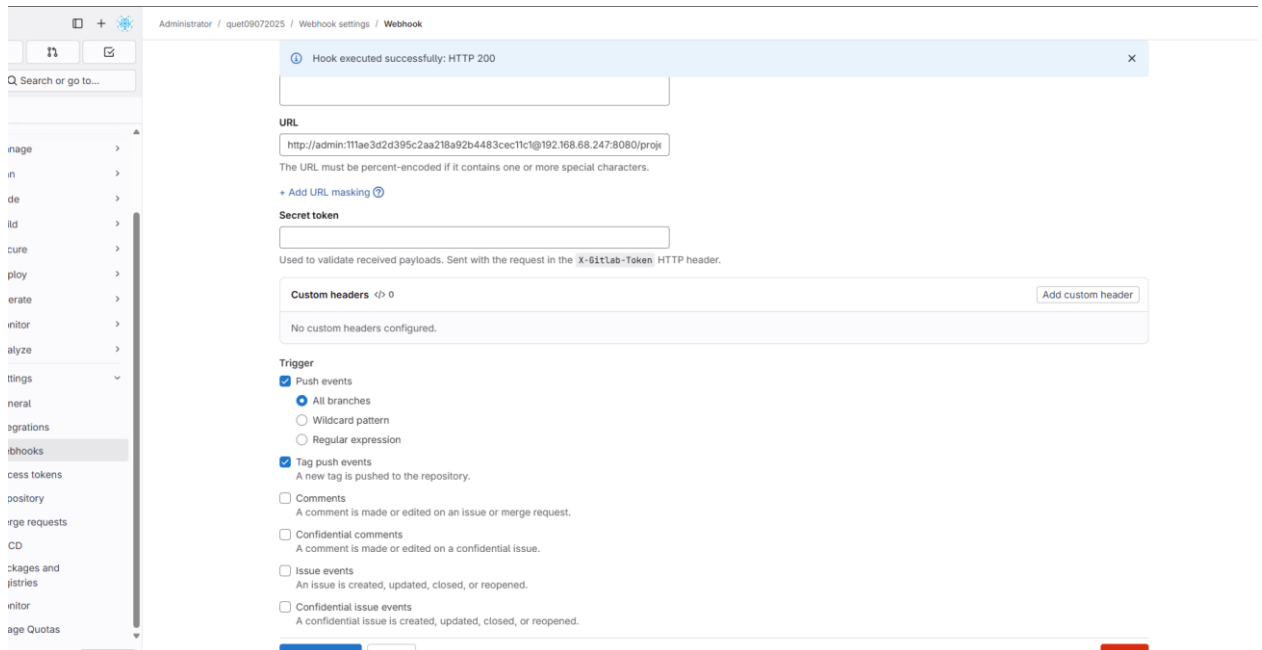
P	Store	Domains
	User: Admin	(global)

Stores from parent

P	Store	Domains
	System	(global)

Icons: S M L

Hình 2.15. Thêm các credentials thành công



Hình 2.16. Thành công chạy rà quét dự án

2.3.5. Cài Java & Maven cho Jenkins Container.

Vào trong docker jenkins

```
docker exec -it jenkins-dind bash
```

Check Java version (Usually installed with Jenkins since Java is one of Jenkins' Dependencies) & JAVA_HOME

```
java -version
```

```
mvn -version
```

```
root@lab-server:~# java -version
openjdk version "17.0.9" 2023-10-17
OpenJDK Runtime Environment (build 17.0.9+9-Ubuntu-120.04)
OpenJDK 64-Bit Server VM (build 17.0.9+9-Ubuntu-120.04, mixed mode, sharing)
root@lab-server:~#
```

Install Maven & get MAVEN_HOME as well (usually: /usr/share/maven)

```
apt update -y
```

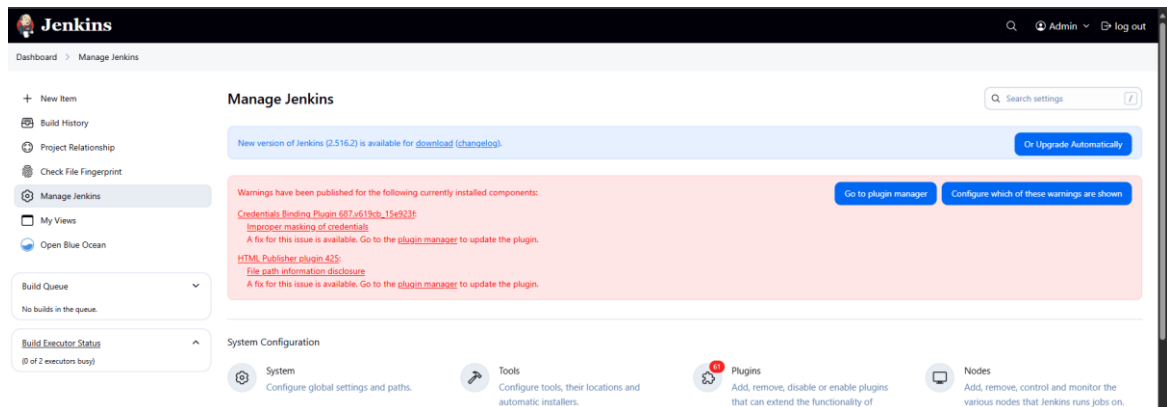
```
apt install maven -y
```

```
mvn -version
```

```
exit
```

- Vào trong Jenkins GUI

Add these tools in the Jenkinsfile for 'jdk' and 'maven'.



Hình 2.17. Tải các plugin về gitlab, blue ocean cho giám sát

2.3.6. Cài SonarQube

Bước 1: Set up SonarQube with a Container

Run the command below to create the SonarQube Container on the same Docker network as Jenkins:

```
docker run -d --name sonarqube-dind \
-e SONAR_ES_BOOTSTRAP_CHECKS_DISABLE=true \
-p 9000:9000 \
--network jenkins-java \
sonarqube:latest
```

Check the logs of the SonarQube Container (Wait till SonarQube is Operational):

```
docker logs -f sonarqube-dind
```

```

2025.08.24 03:04:41 INFO web[[[o.s.s.p.master.service.filter] Initializing Servlet Filter org.sonar.server.plugins.tugnitsktsktsentfilter@esaduse [pattern:0.0.0.0/0.0.0.0]
[/*], exclusions=[*.css, ...]]]
2025.08.24 03:04:41 INFO web[[[o.s.s.p.PlatformLevelStartup] Running Community Edition
2025.08.24 03:04:42 INFO web[[[o.s.s.p.Platform] Web server is operational
2025.08.24 03:04:42 INFO app[[[o.s.a.SchedulerImpl] Process[web] is up
2025.08.24 03:04:42 INFO app[[[o.s.a.ProcessLauncherImpl] Launch process[COMPUTE_ENGINE] from [/opt/sonarqube]: /opt/java/openjdk/bin/java -Djava.awt.headless=true -Dfile.encoding=UTF-8 -Djava.io.tmpdir=/opt/sonarqube/temp -XX:-OmitStackTraceInFastThrow --add-opens=java.base/java.util=ALL-UNNAMED --add-exports=java.base/jdk.internal.ref=ALL-UNNAMED --add-opens=java.base/java.lang=ALL-UNNAMED --add-opens=java.base/java.nio=ALL-UNNAMED --add-opens=java.base/sun.nio.ch=ALL-UNNAMED --add-opens=java.management/sun.management=ALL-UNNAMED --add-opens=jdk.management/com.sun.management=ALL-UNNAMED -Xms512m -Xmx128m -XX:+HeapDumpOnOutOfMemoryError -Dhttp.nonProxyHosts=localhost|127.*|:::1 -cp ./lib/sonar-applcaton-25.7.0.110598.jar:/opt/sonarqube/lib/jdbc/h2/h2-2.3.232.jar org.sonar.ce.app.CeServer /opt/sonarqube/temp/sq-process12871898999361882363properties
2025.08.24 03:04:43 INFO ce[[[o.s.ce.app.CeServer] Compute Engine starting up...
Standard Commons Logging discovery in action with spring-jcl: please remove commons-logging.jar from classpath in order to avoid potential conflicts
2025.08.24 03:04:43 INFO ce[[[o.s.d.DefaultDataSource] Create JDBC data source for jdbc:h2:tcp://127.0.0.1:9092/sonar;NON_KEYWORDS=VALUE
2025.08.24 03:04:44 INFO ce[[[c.z.h.HikariDataSource] HikariPool-1 - Starting...
2025.08.24 03:04:44 INFO ce[[[c.z.h.p.HikariPool] HikariPool-1 - Added connection conn0: url=jdbc:h2:tcp://127.0.0.1:9092/sonar user=
2025.08.24 03:04:44 INFO ce[[[c.z.h.HikariDataSource] HikariPool-1 - Start completed.
2025.08.24 03:04:44 WARN ce[[[o.s.db dialect.H2] H2 database should be used for evaluation purpose only.
2025.08.24 03:04:46 INFO ce[[[o.s.s.p.ServerFileSystemImpl] SonarQube home: /opt/sonarqube
2025.08.24 03:04:47 INFO ce[[[o.s.c.c.CePluginRepository] Load plugins
2025.08.24 03:04:49 INFO ce[[[o.s.c.c.ComputeEngineContainerImpl] Running Community edition
2025.08.24 03:04:49 INFO ce[[[o.s.ce.app.CeServer] Compute Engine is started
2025.08.24 03:04:49 INFO app[[[o.s.a.SchedulerImpl] Process[ce] is up
2025.08.24 03:04:49 INFO app[[[o.s.a.SchedulerImpl] SonarQube is operational
2025.08.24 03:42:43 WARN es[[[o.e.n.j.JvmGcMonitorService] [gc][young][2306][13] duration [1.6s], collections [1]/[2.3s], total [1.6s]/[1.2s], memory [318.2mb]->[53.9mb]/[512mb], 11 pools: [[CodeHeap 'non-nmethods'] [2.2mb]->[2.2mb]/[15.5mb]] [[Metaspace] [86.2mb]->[86.2mb]/[0b]] [[CodeHeap 'profiled nmethods'] [20.0mb]->[120.9mb]/[117.2mb]] [[Compressed class space] [11.5mb]->[11.5mb]/[1gb]] [[young] [256mb]->[0b]/[0b]] [[old] [16.5mb]->[35.2mb]/[512mb]] [[survivor] [37.7mb]->[18.7mb]/[0b]] [[CodeHeap 'non-profiled nmethods'] [8.4mb]->[8.4mb]/[117.2mb]]]
2025.08.24 03:42:44 WARN es[[[o.e.n.j.JvmGcMonitorService] [gc][2306] overhead, spent [1.6s] collecting in the last [2.3s]

```

Hình 2.18. Log không chứa lỗi

Bước 2. Check the Docker Processes Running

docker ps

```

root@ubuntu-virtual-machine:/home/ubuntu/Desktop# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED    STATUS    PORTS
7f989ac7df92   gitlab/gitlab-ce:latest             "/assets/init-conta..." 6 weeks ago Up About an hour (healthy) 443/tcp, 0.0.0.0:2224->22/tcp, [::]:2224->22/tcp, 0.0.0.0:8929->80/tcp, [::]:8929->80/tcp
62a6b3e8d3e3   sonarqube:latest                    "/opt/sonarqube/dock..." 6 weeks ago Up About an hour          0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp
33fc00f17baf   jenkins/jenkins-lts                 "/usr/bin/tini -- /u..." 6 weeks ago Up About an hour          0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 0.0.0.0:50000->50000/tcp, [::]:50000->50000/tcp

```

SonarQube chạy cổng 9000 thành công

2.3.7. Cài Trivy scan images

Tải DB offline đầy đủ từ máy có Internet:

```

# trên máy có Internet
trivy image --download-db-only

# copy toàn bộ thư mục ~/.cache/trivy/db
tar -czvf trivy-full-offline-db.tgz ~/.cache/trivy/db

```

Trên máy có Internet (máy 1)

a) Cài Trivy: Đảm bảo Trivy hoạt động bình thường và có thể download DB:

```

export TRIVY_CACHE_DIR=$HOME/trivy/db
mkdir -p $TRIVY_CACHE_DIR

```

b) Tải OS DB offline

```
trivy image --download-db-only
```

Lúc này `$HOME/trivy/db` sẽ có DB OS.

c) Tải Java DB offline (không qua docker pull)

- Vào thư mục cache DB Java của Trivy sau khi scan image có Java trên máy có Internet:

```

# Trên máy có Internet
trivy image openjdk:8

```

- Trivy sẽ tự tải **Java DB** vào cache (\$HOME/.cache/trivy/db/offline_db_ready hoặc tương tự).

2. Nén toàn bộ DB Trivy (OS + Java):

```
tar -czvf trivy-full-offline-db.tgz -C $HOME/trivy db
```

- trivy-full-offline-db.tgz này sẽ chứa cả OS DB + Java DB.

Trên máy offline (máy 2)

1. Copy file trivy-full-offline-db.tgz sang máy offline.
2. Giải nén vào thư mục bạn muốn làm cache DB:

```
mkdir -p /home/dev/trivy/db
tar -xzf trivy-full-offline-db.tgz -C /home/dev/trivy/
```

3. Sử dụng Trivy offline:

```
export TRIVY_CACHE_DIR=/home/dev/trivy/db
trivy image --skip-db-update --cache-backend fs --severity HIGH,CRITICAL
java-app:9
```

Lưu ý: **Máy offline không cần docker pull Java DB** nữa, vì Trivy đã có DB Java trong cache offline. Chỉ cần chắc chắn **trivy-full-offline-db.tgz** có đủ OS DB và Java DB.

2.3.8. Viết Pipeline

```
pipeline {
    agent any

    tools {
        jdk 'java17015'
        maven 'maven387'
    }

    environment {
        SONAR_SCANNER_HOME = tool 'sonar7'
        IMAGE_NAME = "java-app"
        IMAGE_TAG = "${BUILD_NUMBER}"
        GCP_PROJECT_ID = "focal-dock-440200-u5"
        FULL_IMAGE_NAME = "us-docker.pkg.dev/${GCP_PROJECT_ID}/java-app-repo-02/${IMAGE_NAME}:${IMAGE_TAG}"
        SERVICE_NAME = "java-app-service"
        REGION = "us-central1"
    }
}
```

```

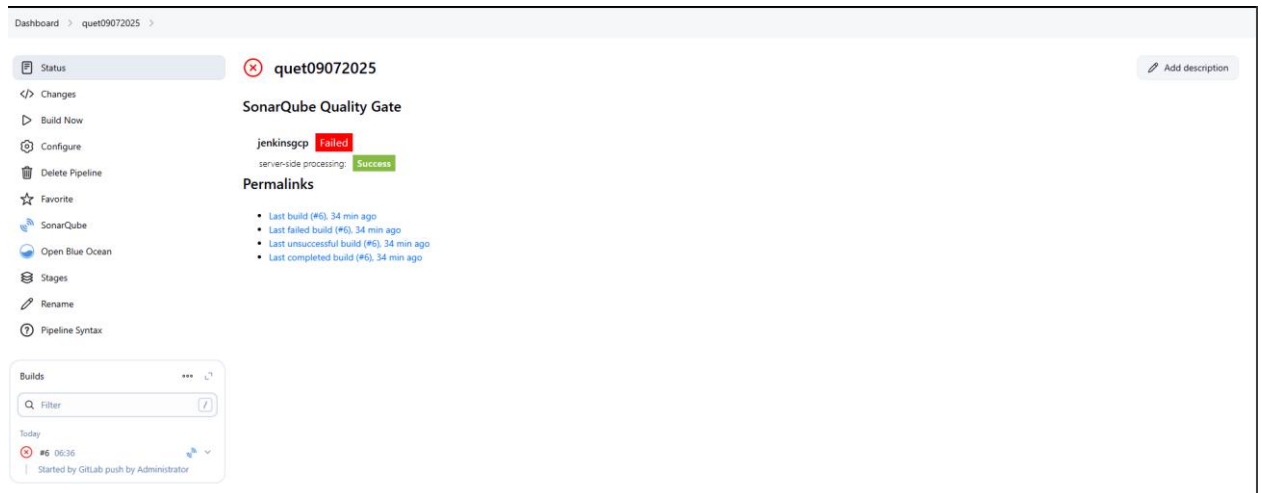
stages {
    ...

    echo '📦 Archiving Trivy scan reports (JSON, HTML, PDF) ...'

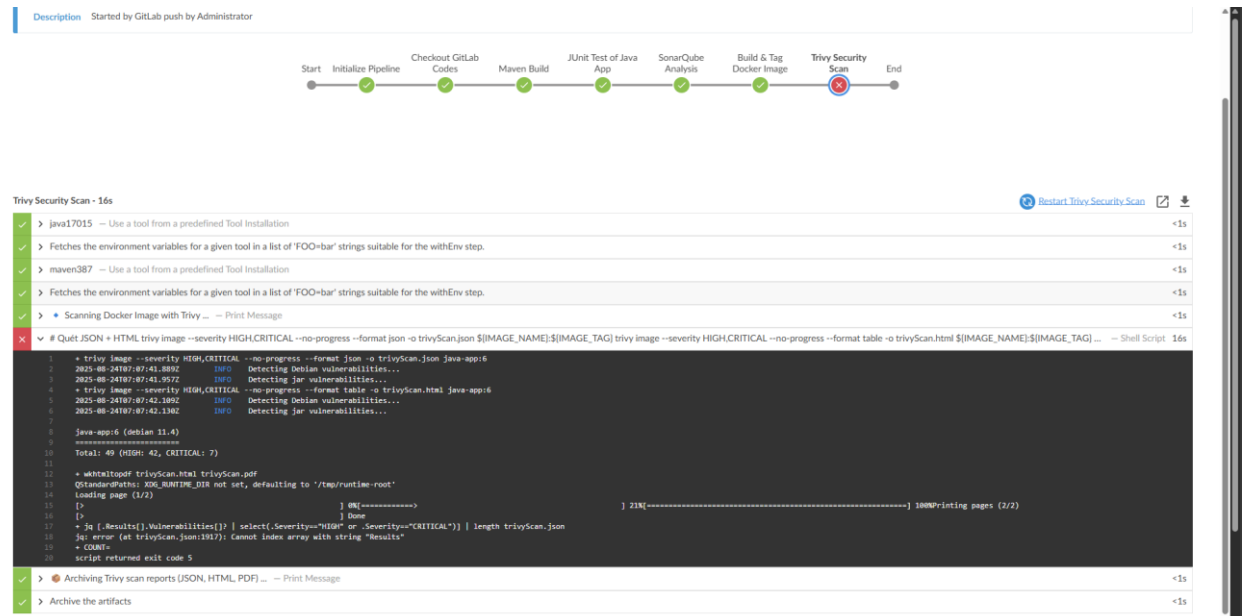
    archiveArtifacts artifacts: 'trivyScan.*', fingerprint: true}}}

```

Kết quả:



Hình 2.19. Nếu source không lỗi thì sẽ thành công và không tự ngắt tiến trình



Hình 2.20. Logic fail nếu ≥ 2 lỗi HIGH/CRITICAL.

Khi có từ 2 lỗi Quality Gate → Conditions có điều kiện nào bị fail (ví dụ: Coverage < 80% hoặc 2 lỗi).

Jenkins

Dashboard > quet09072025 > #6 > Pipeline Overview

#6

Started 39 min ago | Queued 7.4 sec | Took 31 min | Changes | Artifacts

Graph

Start → Checkout SCM → Tool Install → Initialize Pipeline → Checkout GitLab Codes → Maven Build → JUnit Test of Java App → SonarQube Analysis → Build & Tag Docker... → Trivy Security Scan → Post Actions → End

Trivy Security Scan (16 giây) | Started 9 phút 12 giây ago | Jenkins

- ✓ Checkout SCM 2.4 giây
- ✓ Tool Install 0.33 giây
- ✓ Initialize Pipeline 2.1 giây
- ✓ Checkout GitLab Codes 1.0 giây
- ✓ Maven Build 11 giây
- ✓ JUnit Test of Java App 6.4 giây
- ✓ SonarQube Analysis 30 phút
- ✗ Build & Tag Docker Image 6.7 giây
- ✗ Trivy Security Scan 16 giây
- ✓ Post Actions 0.13 giây

Trivy Security Scan Log:

```

0 + trivy image --severity HIGH,CRITICAL --no-progress --format json -o trivyScan.json java-app:6
1 2025-08-24T07:07:41.889Z INFO Detecting Debian vulnerabilities...
2 2025-08-24T07:07:41.957Z INFO Detecting jar vulnerabilities...
3 + trivy image --severity HIGH,CRITICAL --no-progress --format table -o trivyScan.html java-app:6
4 2025-08-24T07:07:42.109Z INFO Detecting Debian vulnerabilities...
5 2025-08-24T07:07:42.130Z INFO Detecting jar vulnerabilities...
6
7 java-app:6 (debian 11.4)
8 =====
9 Total: 49 (HIGH: 42, CRITICAL: 7)
10
11 + wkhtmltopdf trivyScan.html trivyScan.pdf
12 QStandardPaths: XDG_RUNTIME_DIR not set, defaulting to '/tmp/runtime-root'
13 Loading page (1/2)
14 [> ] 0%
15 [=====] 21%
16 [=====] 100%
17 Printing pages (2/2)
18 [> ]
19 Done
20 + jq [-Results[]..Vulnerabilities[]? | select(.Severity=="HIGH" or .Severity=="CRITICAL")] | length trivyScan.json
21 jq: error (at trivyScan.json:1917): Cannot index array with string "Results"
22 + COUNT=
23 script returned exit code 5
  
```

Hình 2.21. Kết quả quét đã tự động ngắt khi có kết quả code không sạch

2.4. Quy trình phân luồng làm việc Gitlab quản lý mã nguồn.

2.4.1. Các bước tạo các nhánh Git trên GitLab UI

- Tạo nhánh develop từ main: Vào repo GitLab của bạn.

+ Chọn **Repository > Branches**.

+ Nhấn nút **New branch**.

- Tạo nhánh feature từ develop:

+ Branch name ví dụ: feature/add-login.

+ Create from: **develop**.

Lưu ý: Mỗi chức năng nên là một nhánh feature/ten-tinh-nang.

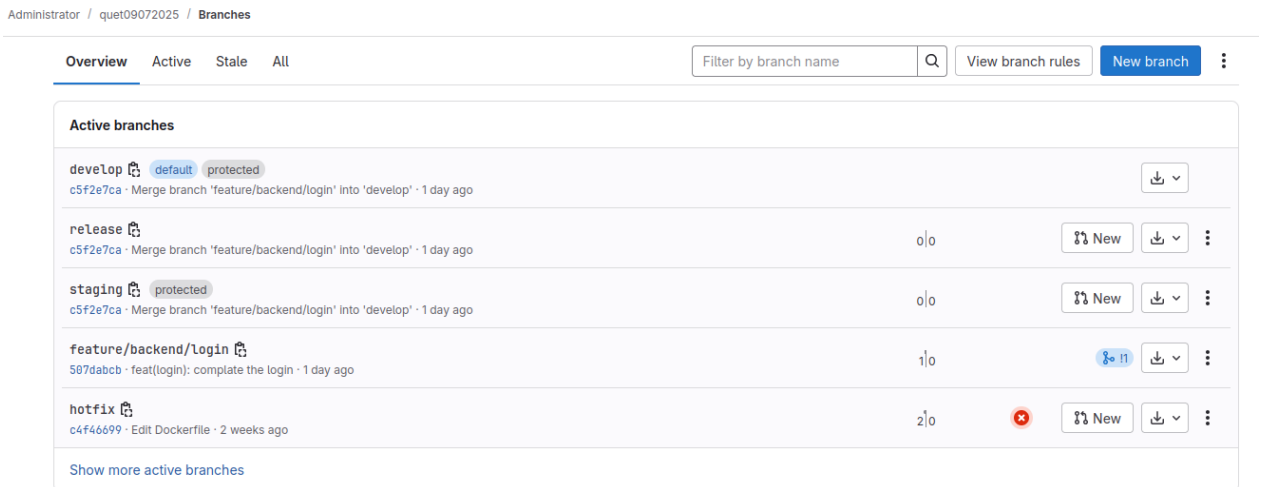
- Tạo nhánh release từ develop:

Branch name ví dụ: release/1.0.0.

Create from: **develop**.

Lưu ý: Dùng để kiểm thử, chỉ chứa code đã ổn định sắp lên production.

Kết quả:



Hình 2.22. Tạo nhánh Git trên GitLab UI

2.4.2. Tạo tag (gắn nhãn version)

- Cách tạo tag:
- Vào Repository > Tags.
- Nhấn New tag.

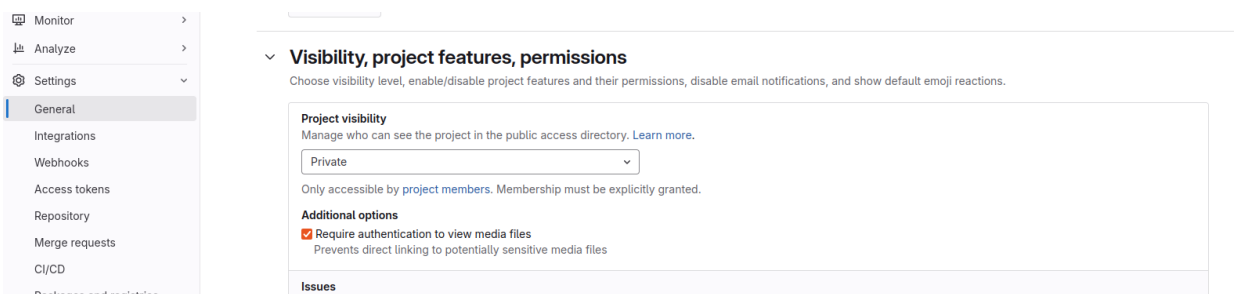
Điền:

- + Tag name: v1.0.0 hoặc 1.0
- + Create from: main hoặc nhánh đã release
- + Message (tùy chọn): ghi chú mô tả release

Lưu ý: Tag thường dùng để đánh dấu version release chính thức.

2.4.3. Bảo vệ nhánh

- Chuyển chế độ private



Khi chuyển chế độ người dùng không thể tìm thấy chỉ khi member có quyền cao hơn mới được add thêm member vào dự án.

- Branch Protection (Bảo vệ nhánh)

▼ Branch defaults

Select the default branch for this project, and configure the template for branch names.

Default branch

All merge requests and commits are made against this branch unless you specify a different one.

develop

☒ Auto-close referenced issues on default branch

When merge requests and commits in the default branch close, any issues they reference also close. [?](#)

Branch name template

Branches created from issues follow this pattern.

%{id}-%{title}

Leave empty to use default template. Maximum 255 characters. [What variables can I use?](#)

Save changes

Đầu tiên, ta chuyển giao đưa quyền nhánh phân quyền cho defaults: **develop. Các bước tiến hành:**

Vào: Settings > Repository > Protected Branches

- + Cấm push trực tiếp vào main, release, develop
- + Bắt buộc merge qua MR
- + Yêu cầu approval khi merge
- + Bắt CI/CD pipeline pass trước khi merge

Protected branches 3
By default, protected branches restrict who can modify the branch. [Learn more.](#)

Protect a branch

Branch:

Select branch or create wildcard

Wildcards such as `*-stable` or `production/*` are supported. [Branch names are case-sensitive.](#)

Allowed to merge:

Select

Allowed to push and merge:

Select

Allowed to force push:

☒

Allow all users with push access to [force push](#).

Protect Cancel

Branch	Allowed to merge	Allowed to push and merge	Allowed to force push ?	
develop default	Maintainers	No one	☒	Unprotect
main	Maintainers	Maintainers	☐	Unprotect
staging	Maintainers	Maintainers	☒	Unprotect

Hình 2.23. Tiến hành thêm các bảo vệ (No one)

Branch rules

Define rules for who can push, merge, and the required approvals for each branch. [Leave feedback.](#)

Edit branch protections, approval rules, and status checks from a single page. [How to use branch rules?](#)

Branch rules 4 Add branch rule

All branches View details

- 6 matching branches
- Squash commits: Allow

develop default protected View details

- Allowed to merge: Maintainers
- Allowed to push and merge: No one

main protected View details

- Allowed to force push
- Allowed to merge: Maintainers
- Allowed to push and merge: Maintainers

staging protected View details

- Allowed to merge: Maintainers
- Allowed to push and merge: Maintainers

Kết quả: Kiểm tra kết quả thấy các nhánh develop, staging, main không được tự ý push trực tiếp mà được bảo vệ.

2.4.4. Phân quyền người dùng (Access Control)

Project members

You can invite a new member to quet09072025 or invite another group.

Import from a project Invite a group Invite members

Members 2

Filter members

Account	Source	Activity
Administrator It's you @root	Direct member by Administrator	8+ Jul 09, 2025 ✓ Jul 09, 2025 ✗ Jul 30, 2025
dev1 @dev1	Direct member by Administrator	8+ Jul 29, 2025 ✓ Jul 29, 2025 ✗ Jul 29, 2025

Change role

- Guest
- Planner
- Reporter
- ✓ Developer
- Maintainer
- Owner

Developer

Vào: Project > Manage > Members

Role	Quyền
Guest	Xem repo
Planner	Xem repo, truy cập, quản lý các công cụ lập kế hoạch (epics, roadmap, issue boards) nhưng không push code.
Reporter	Xem + xem pipeline/log nhưng không push code.
Developer	Có quyền read/write repo, push code, tạo merge request (MR).
Maintainer	Có quyền quản lý nhánh (merge, xoá nhánh), CI/CD, quản lý dự án.
Owner (Group)	Toàn quyền

Source	Role	Expiration	Activity
Direct member by Administrator	Owner	Expiration date	8+ Jul 09, 2025 ✓ Jul 09, 2025 ⌘ Jul 30, 2025 Remove member
Direct member by Administrator	Developer	Expiration date	8+ Jul 29, 2025 ✓ Jul 29, 2025 ⌘ Jul 29, 2025

Ngoài ra, chúng ta còn có thể set thời gian tài khoản, ghi chú lịch hoạt động và xóa member khỏi toàn dự án

Lưu ý: Không để nhiều người là Maintainer; chỉ cho 1–2 người duyệt MR và merge

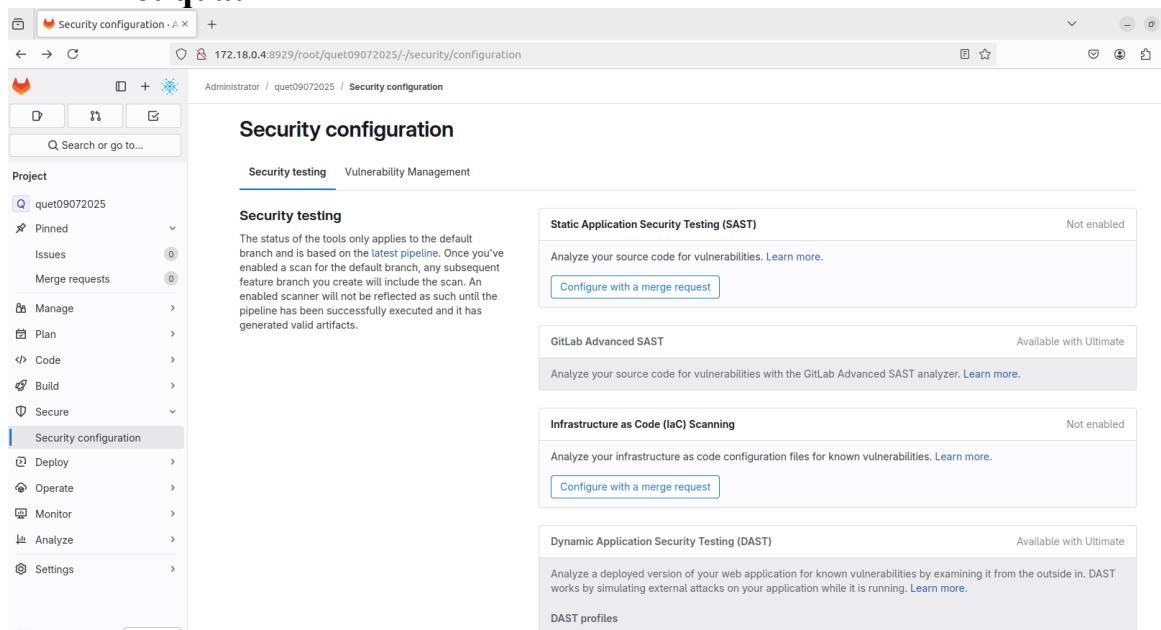
2.4.5. Dashboard quản lý an ninh mã nguồn.

Vào: Security configuration > Dashboard

Tại đây sẽ thấy:

- + Báo cáo lỗ hổng từ SAST, Dependency Scan, Container Scan
- + Theo dõi tổng quan bảo mật theo từng nhánh và commit
- + Lịch sử ai push lỗ hổng vào code

Kết quả:



- Require Merge Request Approvals(Phân quyền người được merge)

Vào: Settings > General > Merge request approvals

- + Yêu cầu **1–2 người** approve trước khi merge
- + Chỉ định **ai là người có thể approve**
- + Bắt buộc pass **CI/CD** mới được merge

- Audit Events (Lịch sử hành động)

Vào: Group > Settings > Audit Events

Theo dõi:

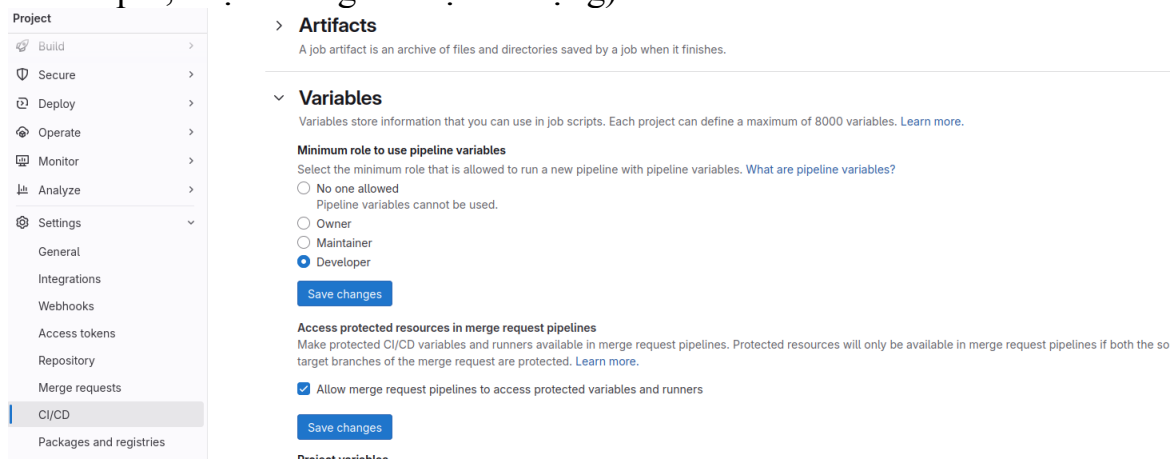
- + Ai push code
- + Ai xóa nhánh
- + Ai thêm user
- + Ai xoá biến môi trường

- CI/CD Settings (Bảo vệ biến môi trường)

Vào: Settings > CI/CD > Variables

- + Đặt biến môi trường nhạy cảm (API key, token)
- + Bật **Protected** để chỉ job trên nhánh protected mới được truy cập
- + Bật **Masked** để ẩn giá trị khi log

Minimum role to use pipeline variables: cho phép bạn giới hạn vai trò user nào được phép chạy pipeline với các biến này (Owner, Maintainer, Developer, hoặc không ai được sử dụng).



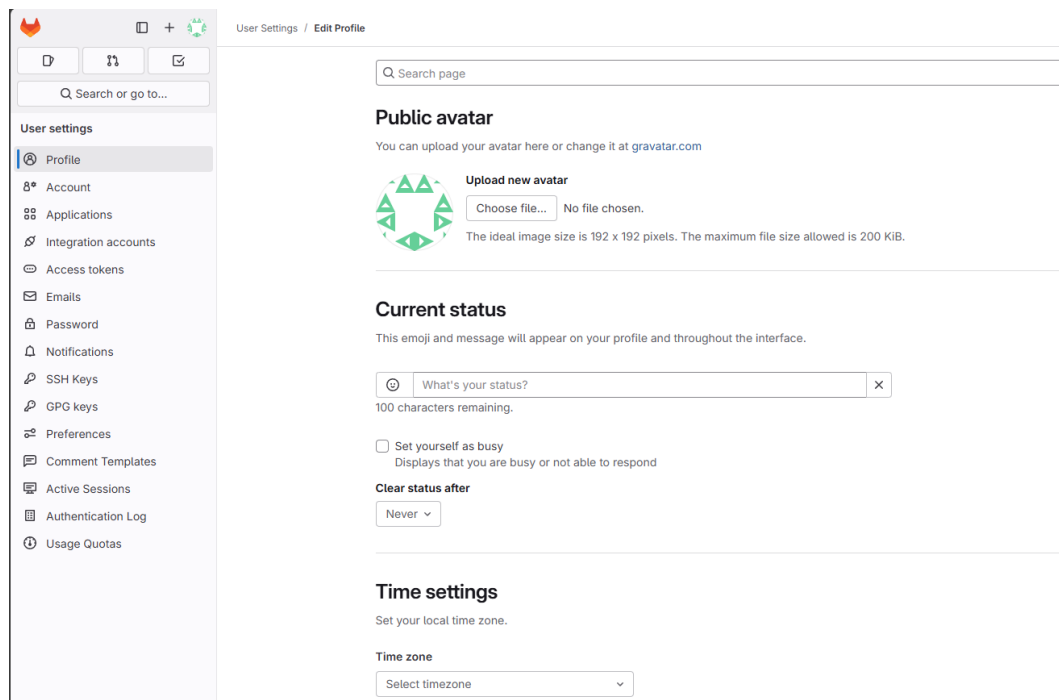
Kết quả: Nhánh develop được bảo vệ CICD

2.4.6. Cơ chế bảo mật xác thực hai lớp 2FA và giao thức OTP

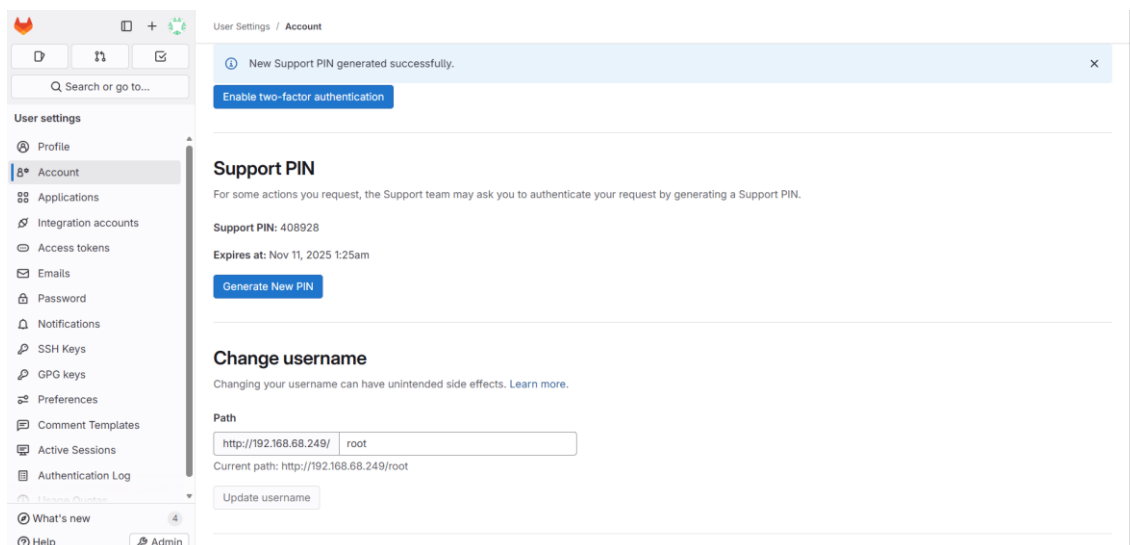
Vào: User Settings > Account

- + Yêu cầu người dùng kích hoạt bảo mật 2 lớp (2FA) bằng Authenticator
- + Bảo vệ tài khoản GitLab cá nhân

Cách setup OTP : Quyền Admin



Chọn Account



Hình 2.24. Pin có giá trị 7 ngày từ khi tạo. **Hỗ trợ PIN:** 4xxxxxx



GitLab Enterprise Edition

Enter verification code

Enter the code from your two-factor authenticator app. If you've lost your device, you can enter one of your recovery codes.

Verify code

User Settings / Account / Two-Factor Authentication

Register a one-time password authenticator

Use a one-time password authenticator on your mobile device or computer to enable two-factor authentication (2FA). We recommend using cloud-based authenticator applications that can restore access if you lose your hardware device. [What are some examples?](#)

Can't scan the code?
To add the entry manually, provide the following details to the application on your phone.
Account: 192.168.68.249:gitlab_admin_c691c1@example.com
Key: YL4S ZNX7 5BK3 3CIK UH0Z 4TFF PI7E RMHA
Time based: Yes

Current password

Your current password is required to register a two-factor authenticator app.

[Enter verification code](#)

[Register with two-factor app](#)

Hình 2.25. Kết quả tạo xác thực 2FA thành công

2.5. Kết luận Chương 2

Trong Chương 2, đề án đã tập trung phân tích và thiết kế hệ thống tự động hóa CI/CD trong quản lý và rà quét mã nguồn bảo mật tại Trường Đại học Kỹ thuật - Hậu cần CAND, cung cấp cái nhìn toàn diện về quy trình triển khai pipeline tự động và tích hợp DevSecOps. Chương đã xác định các chức năng cốt lõi bao gồm đăng nhập/đăng xuất người dùng đảm bảo bảo mật và phân quyền truy cập, build và kiểm thử mã nguồn tự động với unit test và integration test để phát hiện lỗi sớm, phân tích tĩnh mã nguồn bằng SonarQube nhằm đảm bảo chất lượng và tuân thủ coding standards, đóng gói Docker image và quản lý phiên bản để chuẩn bị triển khai, rà quét bảo mật Docker image bằng Trivy nhằm phát hiện lỗ hổng nghiêm trọng, đồng thời cho phép lưu trữ báo cáo và theo dõi trạng thái pipeline liên tục.

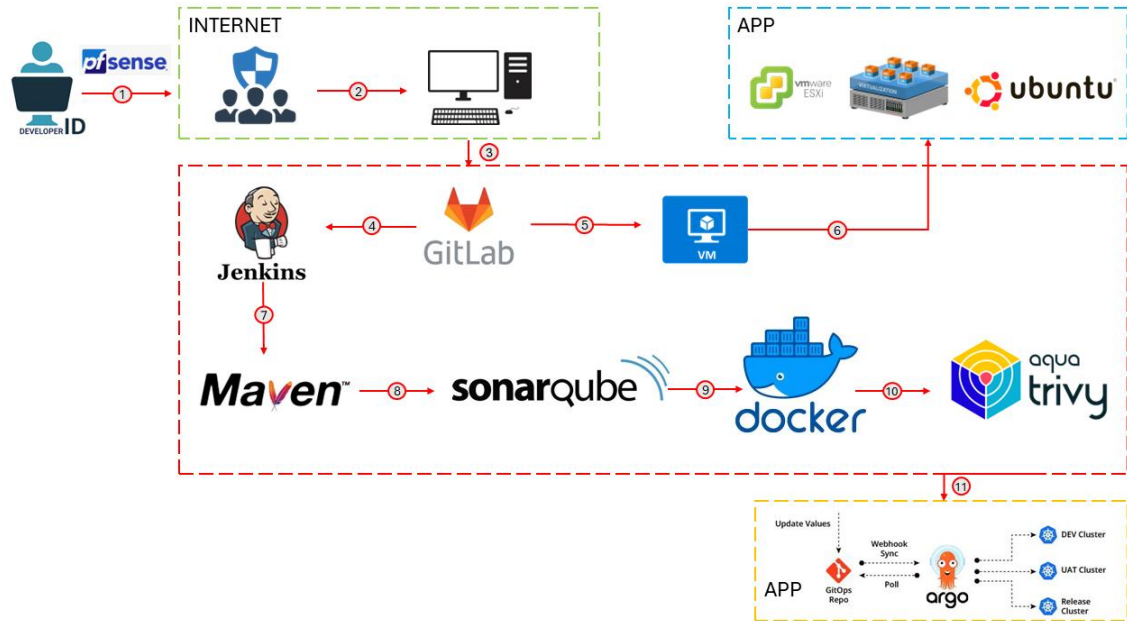
Chương 2 cũng đã triển khai chi tiết bốn biểu đồ tuần tự tương ứng với các module chính trong pipeline CI/CD và biểu đồ hoạt động đăng nhập/đăng xuất, minh họa rõ ràng luồng dữ liệu, tương tác giữa người dùng, các công cụ CI/CD và hệ thống kiểm thử/bảo mật. Cách thức cài đặt triển khai hệ thống trước khi tiến hành chạy thử nghiệm.

Thông qua việc phân tích và thiết kế luồng hoạt động cũng như cài đặt, Chương 2 cung cấp cơ sở để triển khai pipeline CI/CD hoàn chỉnh với tính năng tự động hóa và rà quét bảo mật, tối ưu hóa quá trình phát triển phần mềm, giảm thiểu rủi ro lỗi và lỗ hổng bảo mật, đồng thời nâng cao hiệu quả quản lý và kiểm soát mã nguồn tại Nhà trường. Như vậy, Chương 2 đã tạo nền tảng vững chắc cho việc triển khai thực nghiệm và đánh giá hệ thống, là tiền đề quan trọng để Chương 3 đi sâu vào xây dựng và vận hành thực tế hệ thống CI/CD tự động hóa.

CHƯƠNG 3. XÂY DỰNG TRIỂN KHAI VÀ KIỂM THỬ HỆ THỐNG

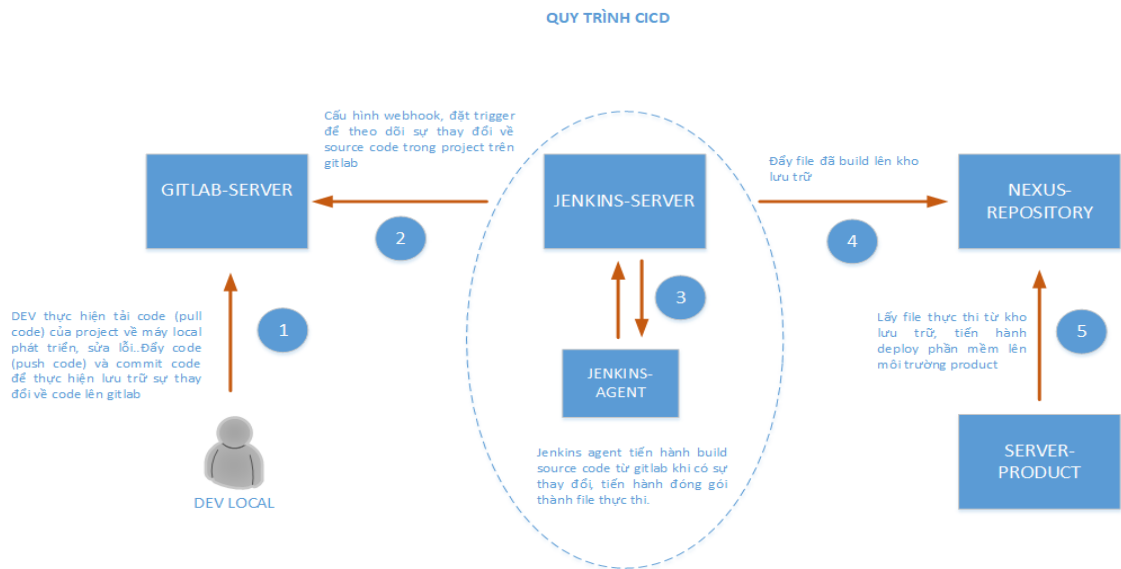
3.1. Mô hình triển khai hệ thống.

3.1.1. Mô hình vật lý tổng thể CI/CD.



Hình 3.1. Mô hình vật lý tổng thể

3.1.2. Mô hình logic CI/CD.



Hình 3.2. Mô hình logic

1) Dev tải mã nguồn xuống môi trường local để tiến hành phát triển, sửa đổi. Sau khi hoàn thành mã nguồn được đẩy lên kho lưu trữ mã nguồn (gitlab).

2) CI Server (Jenkins server) giám sát kho lưu trữ mã nguồn và kiểm tra xem liệu có thay đổi nào trên kho lưu trữ hay không, khi thay đổi thì xảy ra commit ở gitlab server.

3) CI server phát hiện kho lưu trữ có thay đổi sẽ tiến hành gửi yêu cầu cho CI agent (Jenkins agent) tải code mới từ kho lưu trữ về và sau đó tiến hành build, chạy unit và test.

4) Mã nguồn được build và test sẽ đẩy lên một kho lưu trữ mới (nexus repository hoặc harbor).

5) Thực hiện deploy phần mềm mới lên môi trường product

3.2. Xây dựng các kịch bản kiểm thử.

3.2.1. Kiểm thử về môi trường khởi tạo

Mục tiêu: Xác nhận môi trường Jenkins sẵn sàng và các công cụ đã được cài đặt (Java, Maven).

Điều kiện đầu vào: Jenkins agent đang hoạt động; các tool java17015 và maven387 đã được cấu hình.

JDK installations

JDK installations ^ Edited

Add JDK

JDK

Name

java17015

JAVA_HOME

/opt/java/openjdk

☐ Install automatically ?

Add JDK

Maven installations

Maven installations ^ Edited

Add Maven

Maven

Name

maven387

MAVEN_HOME

/usr/share/maven

☐ Install automatically ?

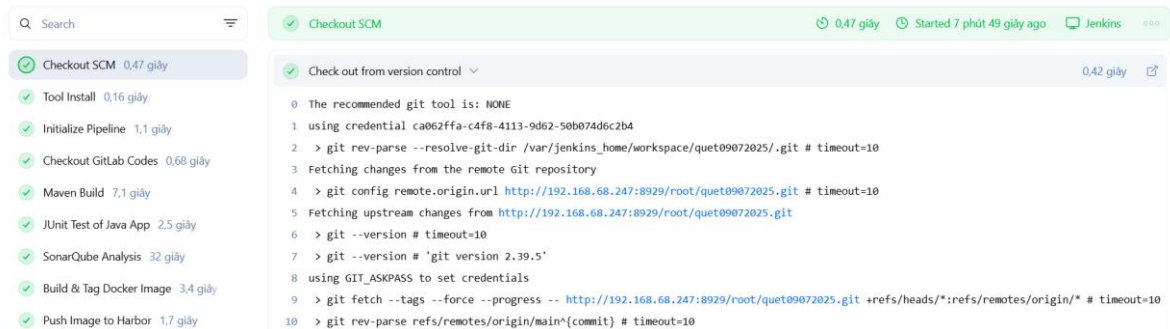
Hình 3.3. Xác nhận môi trường các công cụ đã được cài đặt Java, Maven.

- *Bước thực hiện:*

1. Chạy stage Initialize Pipeline.
2. Kiểm tra phiên bản Java bằng lệnh `java -version`.
3. Kiểm tra phiên bản Maven bằng lệnh `mvn -version`.

- *Kết quả mong đợi:*

- 1) Java và Maven được nhận diện đúng phiên bản.
- 2) Stage báo cáo thành công, pipeline có thể tiếp tục.



Hình 3.4. Stage Java và Maven báo cáo thành công

3.2.2. Kiểm thử quá trình lưu trữ mã nguồn từ GitLab.

Mục tiêu: Xác nhận mã nguồn được tải về chính xác từ repository và branch chính.

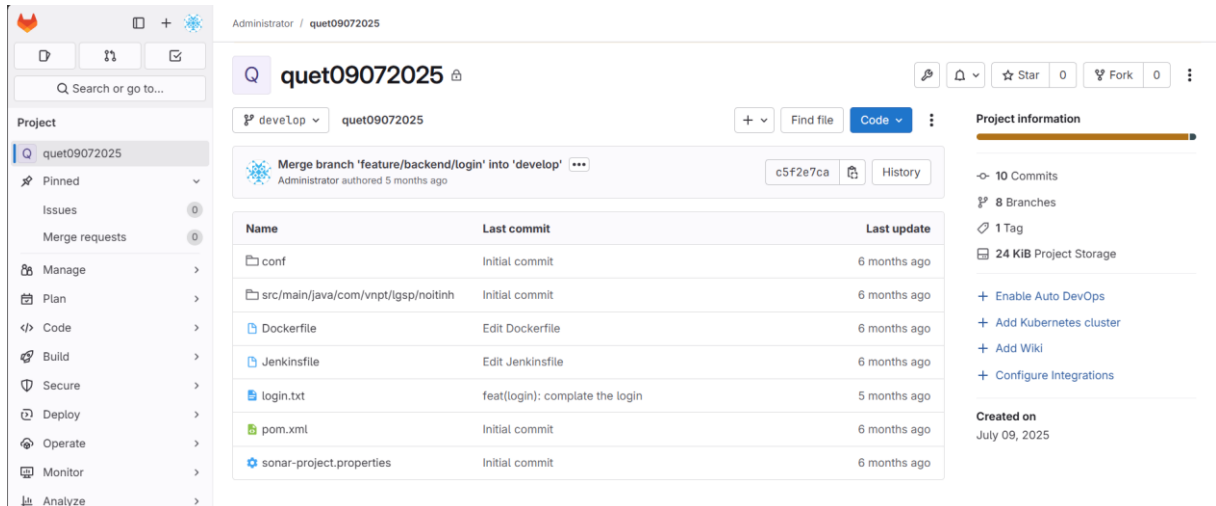
Điều kiện đầu vào: Repository GitLab có branch main; Jenkins credential jenkins-gcp đã được cấu hình.

Bước thực hiện:

1. Chạy stage Checkout GitLab Codes.
2. Kết nối tới repository với thông tin credential.
3. Lấy branch main và lưu vào workspace Jenkins.

Kết quả mong đợi:

1. Mã nguồn được tải đầy đủ, đúng branch.
2. Pipeline báo thành công nếu không có lỗi kết nối hoặc quyền truy cập.



Hình 3.5. Kiểm thử quá trình lưu trữ mã nguồn từ GitLab ổn định

3.2.3. Kiểm thử quá trình tự động xây dựng ứng dụng.

3.2.3.1. Xác nhận quá trình build ứng dụng Java diễn ra thành công.

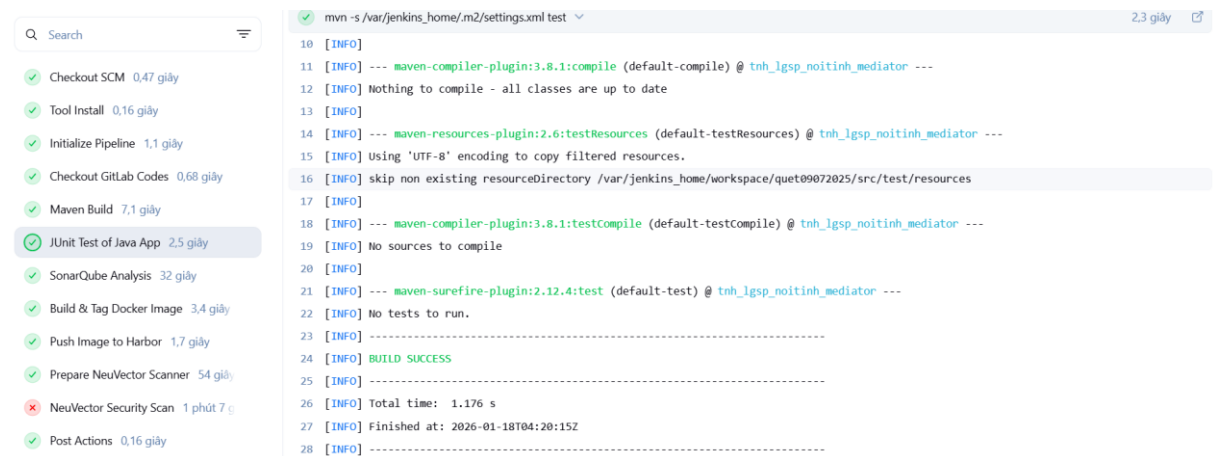
Điều kiện đầu vào: Stage checkout đã thành công, file pom.xml tồn tại trong workspace.

Bước thực hiện:

1. Chạy stage Maven Build.
2. Thực hiện mvn clean package với cấu hình Maven.

Kết quả mong đợi:

Ứng dụng được build thành công, tạo file JAR/WAR trong thư mục target/. Không có lỗi compile.



Hình 3.6. Kiểm thử quá trình tự động xây dựng ứng dụng

3.2.3.2. Build và gắn thẻ Docker Image (Build & Tag Docker Image)

Mục tiêu: Xác nhận Docker image được build thành công với tag đúng.

Điều kiện đầu vào: Stage SonarQube Analysis thành công; Docker daemon hoạt động.

Bước thực hiện:

Chạy stage Build & Tag Docker Image.

Thực hiện docker build -t java-app:\${BUILD_NUMBER} ..

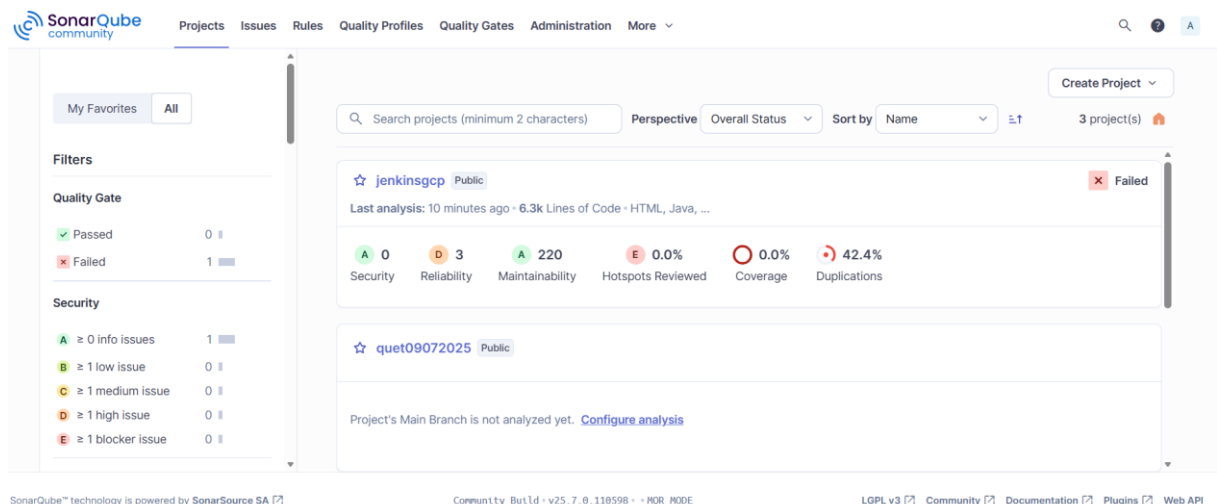
Kết quả mong đợi:

Image Docker tạo thành công. Tag \${BUILD_NUMBER} chính xác.



Hình 3.7. Build và gắn thẻ Docker Image (Build & Tag Docker Image)

3.2.4. Kiểm thử phân tích mã nguồn tĩnh với SonarQube.



Hình 3.8. Kiểm thử phân tích mã nguồn tĩnh với SonarQube chạy ổn định

SonarQube được sử dụng trong hệ thống CI/CD nhằm phân tích mã nguồn tĩnh (SAST) ngay tại giai đoạn tích hợp liên tục (CI), với các mục tiêu kỹ thuật chính:

- (1) Phát hiện sớm các lỗi bảo mật (Security Hotspot, Vulnerability) trong mã nguồn.
- (2) Đánh giá chất lượng mã (Code Smell, Bug) trước khi triển khai

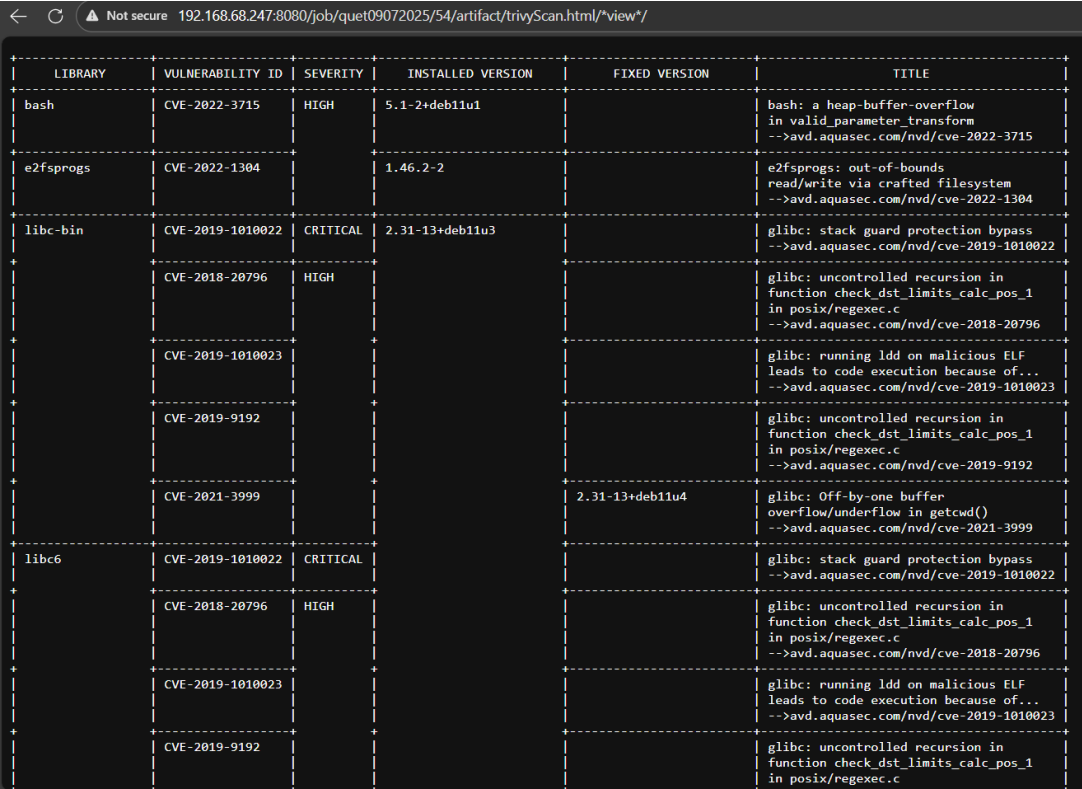
(3) Ngăn chặn mã nguồn không đạt chuẩn về bảo mật đi tiếp sang giai đoạn CD

(4) Tự động hóa kiểm thử bảo mật, không phụ thuộc kiểm tra thủ công

3.2.5. Kiểm thử phân tích động với Trivy.

Trong khuôn khổ nghiên cứu xây dựng hệ thống tự động hóa CI/CD phục vụ quản lý và rà quét mã nguồn bảo mật tại Trường Đại học Kỹ thuật - Hậu cần CAND, công cụ Trivy được lựa chọn để triển khai kiểm thử phân tích động nhằm phát hiện sớm các lỗ hổng bảo mật tồn tại trong quá trình xây dựng và triển khai ứng dụng. Trivy là công cụ mã nguồn mở, có khả năng quét nhanh và hiệu quả các lỗ hổng bảo mật trong container image, thư viện phụ thuộc, hệ điều hành nền và cấu hình hạ tầng dưới dạng mã (Infrastructure as Code – IaC).

Trong hệ thống CI/CD được đề xuất, Trivy được tích hợp trực tiếp vào pipeline tự động, thực hiện quét bảo mật ngay sau các giai đoạn build và đóng gói ứng dụng. Cách tiếp cận này cho phép phát hiện các điểm yếu bảo mật phát sinh trong môi trường thực thi, bao gồm lỗ hổng từ thư viện bên thứ ba, cấu hình sai lệch hoặc các thành phần hệ thống tiềm ẩn rủi ro, từ đó giảm thiểu nguy cơ đưa sản phẩm không an toàn vào môi trường vận hành.



LIBRARY	VULNERABILITY ID	SEVERITY	INSTALLED VERSION	FIXED VERSION	TITLE
bash	CVE-2022-3715	HIGH	5.1-2+deb11u1		bash: a heap-buffer-overflow in valid_parameter_transform -->avd.aquasec.com/nvd/cve-2022-3715
e2fsprogs	CVE-2022-1384		1.46.2-2		e2fsprogs: out-of-bounds read/write via crafted filesystem -->avd.aquasec.com/nvd/cve-2022-1384
libc-bin	CVE-2019-1010022	CRITICAL	2.31-13+deb11u3		glibc: stack guard protection bypass -->avd.aquasec.com/nvd/cve-2019-1010022
	CVE-2018-20796	HIGH			glibc: uncontrolled recursion in function check_dst_limits_calc_pos_1 in posix/regexec.c -->avd.aquasec.com/nvd/cve-2018-20796
	CVE-2019-1010023				glibc: running ldd on malicious ELF leads to code execution because of... -->avd.aquasec.com/nvd/cve-2019-1010023
	CVE-2019-9192				glibc: uncontrolled recursion in function check_dst_limits_calc_pos_1 in posix/regexec.c -->avd.aquasec.com/nvd/cve-2019-9192
	CVE-2021-3999			2.31-13+deb11u4	glibc: Off-by-one buffer overflow/underflow in getcwd() -->avd.aquasec.com/nvd/cve-2021-3999
libc6	CVE-2019-1010022	CRITICAL			glibc: stack guard protection bypass -->avd.aquasec.com/nvd/cve-2019-1010022
	CVE-2018-20796	HIGH			glibc: uncontrolled recursion in function check_dst_limits_calc_pos_1 in posix/regexec.c -->avd.aquasec.com/nvd/cve-2018-20796
	CVE-2019-1010023				glibc: running ldd on malicious ELF leads to code execution because of... -->avd.aquasec.com/nvd/cve-2019-1010023
	CVE-2019-9192				glibc: uncontrolled recursion in function check_dst_limits_calc_pos_1 in posix/regexec.c

Hình 3.9. Báo cáo dạng JSON

Scan Report

Summary

Registry URL	Repository	Tag	High severity VULs	High severity threshold	Medium severity VULs	Medium severity threshold	VULs to fail the build	Total VULs
http://192.168.68.247:8081	library/java-app	54	106	2	111	5		237

Vulnerabilities

Name	Severity	Score	Package	Filename	Fixed_version	Vectors	Description	Feed_rating
CVE-			openssl/libssl 1.1.1-1		1.1.1-1		Issue summary: Calling the OpenSSL API function SSL_select_next_proto with an empty supported client protocols buffer may cause a crash or memory contents to be sent to the peer. Impact summary: A buffer overflow can have a range of potential consequences such as unexpected application behaviour or a crash. In particular this issue could result in up to 255 bytes of arbitrary private data from memory being sent to the peer leading to a loss of confidentiality. However, only applications that directly call the SSL_select_next_proto function with a 0 length list of supported client protocols are affected by this issue. This would normally never be a valid scenario and is typically not under attacker control but may occur by accident in the case of a configuration or programming error in the calling application. The OpenSSL API function SSL_select_next_proto is typically used by TLS applications that support ALPN (Application Layer Protocol Negotiation) or NPN (Next Protocol Negotiation). NPN is older, was never standardised and is deprecated in favour of ALPN. We believe that ALPN is significantly more widely deployed than NPN. The SSL_select_next_proto function accepts a list of protocols from the server and a list of protocols from the client and returns the first protocol that appears in the server list that also appears in the client list. In the case of no overlap between the two lists it returns the first item in the client list. In either case it will signal whether an overlap between the two lists was found. In the case where SSL_select_next_proto is called with a zero length client list it fails to notice this condition and returns the memory immediately following the client list pointer (and reports that there was no overlap in the lists). This function is typically called from a server side application callback for ALPN or a client side application callback for NPN. In the case of ALPN the list of protocols provided by the server is not used.	not set

Hình 3.10. Báo cáo dạng HTML

Kết quả kiểm thử của Trivy được xuất ra dưới hai định dạng chính là JSON và HTML nhằm đáp ứng đồng thời nhu cầu xử lý tự động và theo dõi trực quan. Định dạng JSON được sử dụng để tích hợp với các hệ thống quản lý bảo mật, công cụ phân tích tập trung và các bước kiểm soát tự động trong pipeline CI/CD, cho phép trích xuất, tổng hợp và đánh giá mức độ rủi ro một cách có hệ thống. Trong khi đó, định dạng HTML cung cấp báo cáo trực quan, dễ đọc, hỗ trợ cán bộ phát triển và quản trị hệ thống nhanh chóng nắm bắt tình trạng bảo mật, phân loại mức độ nghiêm trọng của lỗ hổng và đưa ra phương án khắc phục phù hợp.

Việc áp dụng Trivy trong kiểm thử phân tích động giúp nâng cao hiệu quả phát hiện lỗ hổng bảo mật theo thời gian thực, đảm bảo tính liên tục và ổn định của hệ thống CI/CD. Qua đó, quy trình phát triển phần mềm tại Nhà trường không chỉ được tự động hóa mà còn được tăng cường khả năng kiểm soát rủi ro bảo mật, góp phần xây dựng môi trường phát triển ứng dụng an toàn, bền vững và phù hợp với yêu cầu chuyển đổi số hiện nay.

3.2.6. Lưu trữ báo cáo và artifacts

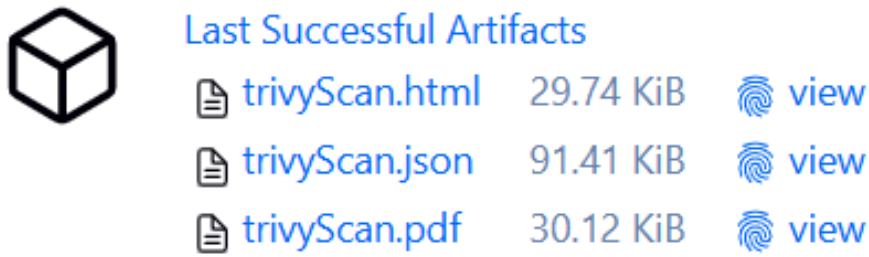
Mục tiêu: Đảm bảo tất cả báo cáo và artifacts quan trọng được lưu trữ và truy xuất dễ dàng.

Điều kiện đầu vào: Các stage trước đã hoàn thành (hoặc fail).

Bước thực hiện:

Chạy post action always.

Lưu trữ các file báo cáo Trivy (trivyScan.*) với archiveArtifacts.



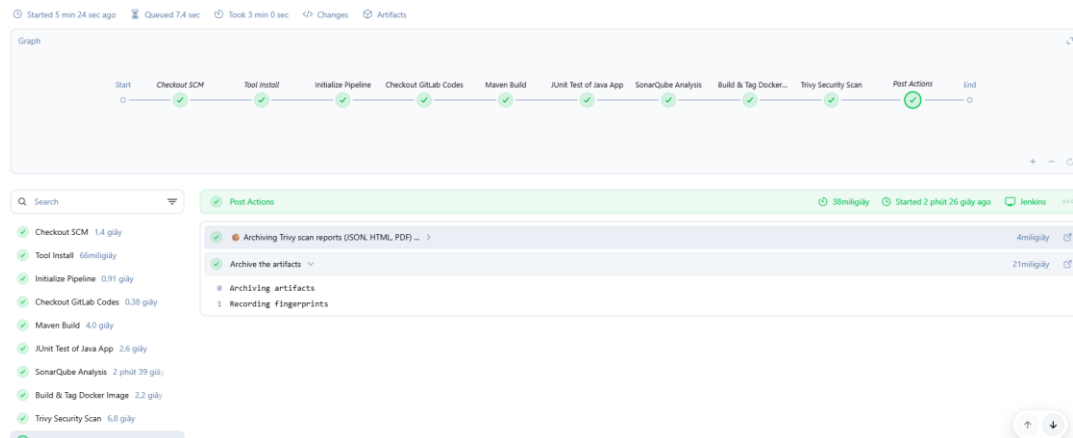
Hình 3.11. Lưu trữ báo cáo và artifacts

Kết quả mong đợi:

Các file artifacts được lưu trữ trong Jenkins, có thể truy cập cho việc phân tích hoặc audit.

3.3. Đánh giá kết quả triển khai.

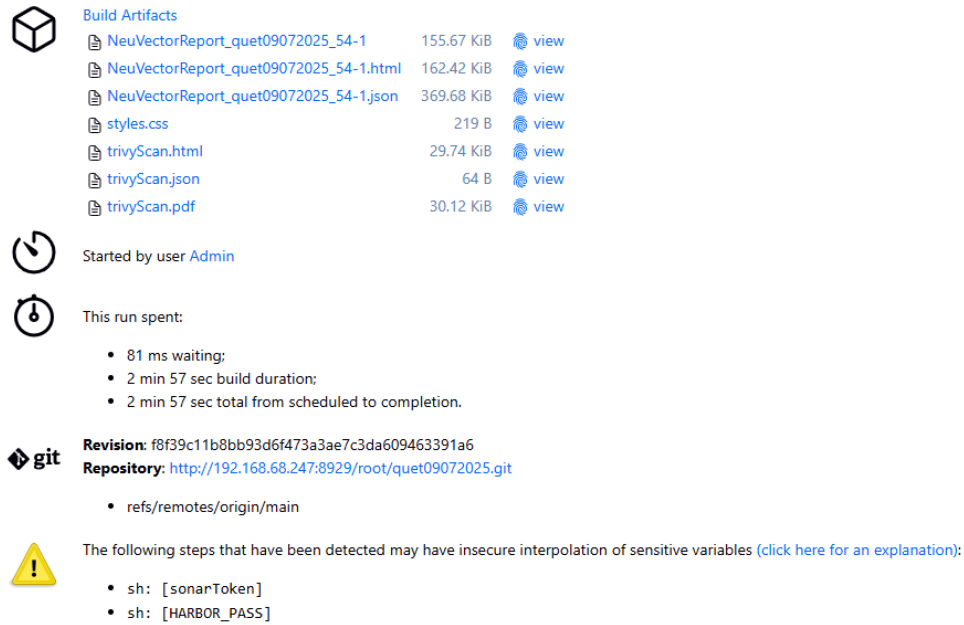
Sau quá trình xây dựng, tích hợp và triển khai hệ thống tự động hóa theo mô hình CI/CD - DevOps, đề tài đã đạt được nhiều kết quả quan trọng cả về mặt kỹ thuật lẫn ý nghĩa thực tiễn, đặc biệt phù hợp với yêu cầu triển khai các hệ thống công nghệ thông tin trong lực lượng Công an nhân dân nói chung và Trường Đại học Kỹ thuật - Hậu cần CAND nói riêng.



Hình 3.12. Kết quả đánh giá triển khai.

3.3.1. Kết quả đạt được về mặt kỹ thuật

Hệ thống CI/CD được xây dựng đã đáp ứng đầy đủ các yêu cầu đề ra, bảo đảm tự động hóa xuyên suốt toàn bộ vòng đời phát triển phần mềm, từ quản lý mã nguồn, kiểm thử, phân tích bảo mật, đóng gói, triển khai đến giám sát vận hành. Các công cụ như GitLab, Jenkins, SonarQube, Nexus, Docker và Trivy được tích hợp đồng bộ, hình thành một pipeline hoàn chỉnh, ổn định và có khả năng mở rộng.



Build Artifacts

NeuVectorReport_quet09072025_54-1	155.67 KiB	view
NeuVectorReport_quet09072025_54-1.html	162.42 KiB	view
NeuVectorReport_quet09072025_54-1.json	369.68 KiB	view
styles.css	219 B	view
trivyScan.html	29.74 KiB	view
trivyScan.json	64 B	view
trivyScan.pdf	30.12 KiB	view

Started by user [Admin](#)

This run spent:

- 81 ms waiting;
- 2 min 57 sec build duration;
- 2 min 57 sec total from scheduled to completion.

git **Revision:** f8f39c11b8bb93d6f473a3ae7c3da609463391a6
Repository: <http://192.168.68.247:8929/root/quet09072025.git>

- refs/remotes/origin/main

The following steps that have been detected may have insecure interpolation of sensitive variables ([click here for an explanation](#)):

- sh: [sonarToken]
- sh: [HARBOR_PASS]

Hình 3.13. Kết quả đạt được về mặt kỹ thuật

So với phương thức triển khai thủ công truyền thống, hệ thống tự động hóa cho thấy hiệu quả rõ rệt về thời gian, độ chính xác và mức độ an toàn. Thời gian triển khai một phiên bản phần mềm được rút ngắn đáng kể, các lỗi phát sinh trong quá trình build và triển khai được phát hiện sớm ngay từ giai đoạn phát triển, hạn chế tối đa các rủi ro khi đưa hệ thống vào môi trường vận hành thực tế.

3.3.2. Đánh giá định lượng kết quả triển khai

Dựa trên quá trình thử nghiệm và mô phỏng triển khai, có thể đưa ra một số chỉ số định lượng giả mang tính tham khảo như sau:

Bảng 3.1. Đánh giá định lượng kết quả triển khai

Tiêu chí đánh giá	Triển khai thủ công truyền thống	Triển khai CI/CD – DevOps	Mức cải thiện
Thời gian triển khai 1 phiên bản	2 - 3 ngày	30 - 60 phút	Giảm ~80-90%
Tỷ lệ lỗi khi triển khai	8 - 12%	< 2%	Giảm ~75-85%
Thời gian phát hiện lỗi bảo mật	Sau khi vận hành	Ngay trong pipeline	Phát hiện sớm 100%
Mức độ phụ thuộc thao tác thủ công	Cao	Rất thấp	Tự động hóa >90%

Tiêu chí đánh giá	Triển khai thủ công truyền thống	Triển khai CI/CD – DevOps	Mức cải thiện
Khả năng truy vết, kiểm soát phiên bản	Hạn chế	Toàn diện, theo log	Tăng mạnh

Các kết quả trên cho thấy mô hình CI/CD không chỉ giúp tối ưu hiệu suất kỹ thuật mà còn nâng cao đáng kể độ tin cậy và an toàn của hệ thống, đặc biệt quan trọng đối với các hệ thống nghiệp vụ có yêu cầu cao về bảo mật và tính liên tục.

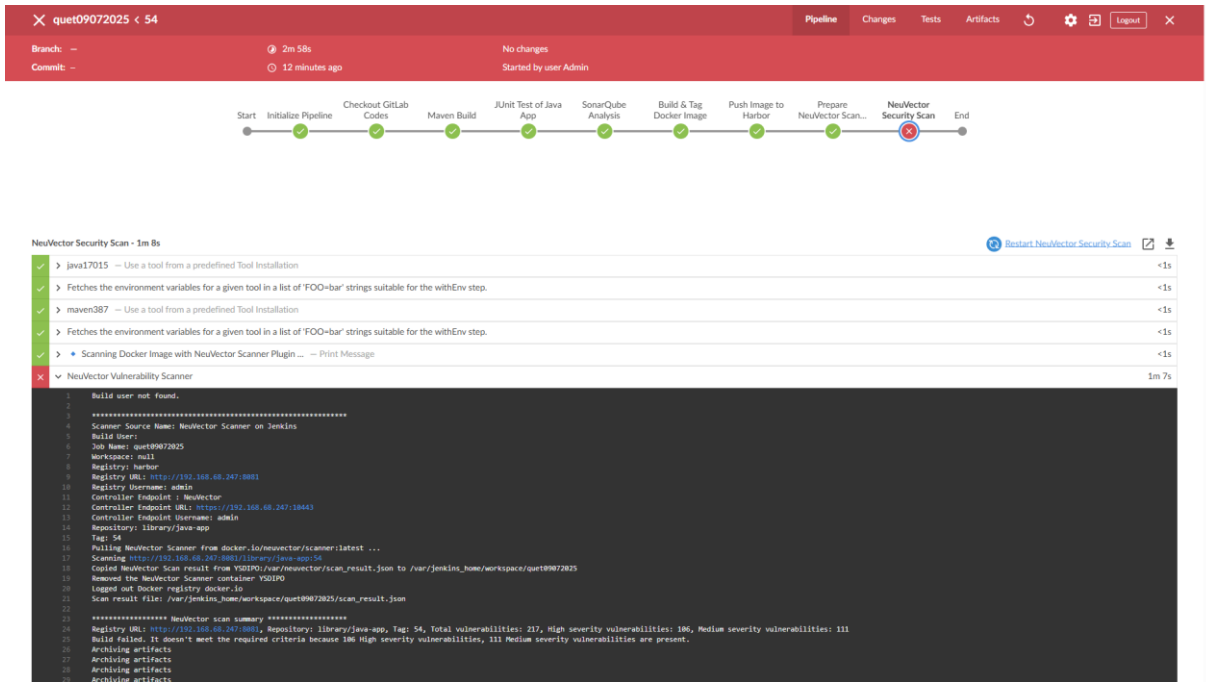
3.3.3. So sánh với các mô hình triển khai hệ thống khác

So với các mô hình triển khai phân tán, thiếu chuẩn hóa hoặc phụ thuộc nhiều vào kinh nghiệm cá nhân, mô hình CI/CD - DevOps thể hiện ưu thế vượt trội về tính đồng bộ, khả năng mở rộng và kiểm soát rủi ro.

Bảng 3.2. So sánh với các mô hình triển khai hệ thống khác

Mô hình triển khai	Ưu điểm	Hạn chế
Triển khai thủ công	Dễ thực hiện ban đầu	Dễ lỗi, khó mở rộng, rủi ro cao
DevOps bán tự động	Cải thiện một phần	Chưa đồng bộ, thiếu kiểm soát bảo mật
CI/CD - DevSecOps	Tự động hóa toàn trình, an toàn cao	Đòi hỏi đầu tư ban đầu và chuẩn hóa

Kết quả phân tích cho thấy mô hình CI/CD tích hợp DevSecOps là lựa chọn phù hợp và tất yếu đối với các hệ thống công nghệ thông tin quy mô lớn, liên thông dữ liệu đa ngành, đặc biệt trong bối cảnh triển khai các nền tảng dùng chung như Cơ sở dữ liệu quốc gia, định danh điện tử, dịch vụ công trực tuyến.



Hình 3.14. Kết quả ưu điểm của sản phẩm so với các mô hình khác

3.3.4. Định hướng phát triển trong thời gian tới

Từ các kết quả đạt được, có thể định hướng phát triển hệ thống theo các hướng sau:

Mở rộng mô hình CI/CD theo hướng DevSecOps và AIOps, tích hợp sâu các công cụ phân tích thông minh và trí tuệ nhân tạo.

Chuẩn hóa pipeline dùng chung trong toàn lực lượng Công an, tiến tới xây dựng nền tảng triển khai tập trung.

Kết nối hệ thống CI/CD với các nền tảng quản lý nghiệp vụ theo cơ chế “một cửa sổ”, nâng cao hiệu quả điều hành và giám sát.

Tăng cường đánh giá an ninh, an toàn thông tin, đáp ứng yêu cầu bảo vệ dữ liệu ở mức cao nhất.

3.4. Kiến nghị đề xuất giải pháp nâng cao trong quá trình tự động hóa hiện nay.

3.4.1. Hoàn thiện mô hình DevSecOps, bảo đảm an ninh ngay từ khâu thiết kế

Trong điều kiện các hệ thống công nghệ thông tin ngày càng giữ vai trò trung tâm trong công tác quản lý nhà nước về an ninh, trật tự, việc bảo đảm an ninh, an toàn hệ thống không thể chỉ dừng lại ở khâu vận hành hay xử lý sự cố, mà phải được đặt ra ngay từ giai đoạn thiết kế, phát triển và triển khai phần mềm. Thực tiễn cho thấy, mô hình DevOps truyền thống, dù mang lại hiệu quả cao về tốc độ và tính linh hoạt trong phát triển phần mềm, vẫn tiềm ẩn nguy cơ nếu yếu tố bảo mật chưa được tích hợp đầy đủ, đồng bộ và có hệ thống.

Trên cơ sở đó, việc chuyển đổi sang mô hình DevSecOps là yêu cầu mang tính tất yếu, phù hợp với đặc thù các hệ thống nghiệp vụ của lực lượng Công an nhân dân. DevSecOps không chỉ là việc bổ sung thêm các công cụ bảo mật, mà là sự thay đổi căn bản về tư duy: coi an ninh, an toàn thông tin là một cấu phần không thể tách rời trong toàn bộ vòng đời phát triển phần mềm, từ khâu phân tích yêu cầu, thiết kế kiến trúc, viết mã, kiểm thử, triển khai cho đến vận hành và giám sát hệ thống.

Việc tích hợp các công cụ rà quét bảo mật mã nguồn (SAST), phân tích động ứng dụng (DAST), kiểm tra thư viện phụ thuộc và thành phần mã nguồn mở, phân tích lỗ hổng container, cùng các cơ chế kiểm soát cấu hình an toàn ngay trong pipeline CI/CD cho phép phát hiện sớm, ngăn chặn kịp thời các nguy cơ tiềm ẩn. Nhờ đó, các lỗ hổng bảo mật có thể được xử lý ngay từ giai đoạn phát triển, trước khi hệ thống được đưa vào khai thác thực tế, giúp giảm đáng kể chi phí khắc phục sự cố, hạn chế rủi ro gián đoạn dịch vụ và tránh nguy cơ bị khai thác trái phép.

Trong bối cảnh đó, việc áp dụng mô hình DevSecOps cho phép hình thành một “lá chắn an ninh nhiều lớp”, kiểm soát chặt chẽ xuyên suốt từ mã nguồn, quy trình triển khai, hạ tầng kỹ thuật đến dữ liệu vận hành. Mô hình này giúp bảo đảm các hệ thống thông tin trọng yếu vận hành an toàn, ổn định và tin cậy, đồng thời đáp ứng yêu cầu vừa phát triển nhanh, vừa bảo đảm an ninh ở mức cao. Đây cũng là định hướng phù hợp với quan điểm đào tạo, nghiên cứu và ứng dụng khoa học - công nghệ của Trường Đại học Kỹ thuật - Hậu cần CAND trong việc xây dựng, triển khai các hệ thống công nghệ thông tin phục vụ hiệu quả nhiệm vụ bảo vệ an ninh quốc gia và giữ gìn trật tự, an toàn xã hội trong tình hình mới.

Không chỉ dừng lại ở khía cạnh kỹ thuật, DevSecOps còn góp phần nâng cao kỷ luật quy trình, tăng cường trách nhiệm của từng cá nhân, từng khâu trong

quá trình phát triển phần mềm. Việc tự động hóa kiểm soát an ninh trong pipeline giúp minh bạch hóa quy trình, loại bỏ sự phụ thuộc vào đánh giá chủ quan, đồng thời tạo ra cơ sở dữ liệu phục vụ công tác kiểm tra, giám sát và đánh giá chất lượng hệ thống. Đây chính là nền tảng quan trọng để xây dựng các hệ thống công nghệ thông tin của lực lượng Công an nhân dân theo hướng chính quy, hiện đại, an toàn và bền vững.

Từ những phân tích trên có thể khẳng định, việc hoàn thiện và triển khai mô hình DevSecOps không chỉ là một giải pháp kỹ thuật đơn thuần mà còn là định hướng chiến lược lâu dài, góp phần nâng cao năng lực bảo đảm an ninh thông tin, phục vụ hiệu quả công tác quản lý và đáp ứng yêu cầu chuyển đổi số quốc gia trong tình hình mới.

3.4.2. Triển khai tự động hóa toàn quy trình, hạn chế tối đa các thao tác thủ công

Trong bối cảnh chuyển đổi số mạnh mẽ hiện nay, quá trình phát triển và triển khai phần mềm cần được tổ chức theo hướng tự động hóa toàn trình, bao phủ đầy đủ các khâu từ tiếp nhận yêu cầu nghiệp vụ, quản lý và kiểm soát mã nguồn, kiểm thử tự động, phân tích chất lượng và an ninh mã nguồn, đóng gói, triển khai, giám sát vận hành đến cập nhật, nâng cấp phiên bản. Việc hạn chế tối đa các thao tác thủ công không chỉ giúp rút ngắn thời gian triển khai, tiết kiệm nguồn nhân lực mà còn góp phần giảm thiểu rủi ro, sai sót phát sinh do yếu tố con người - một trong những nguyên nhân phổ biến dẫn đến sự cố kỹ thuật và mất an toàn thông tin.

Đối với các hệ thống nghiệp vụ phục vụ lực lượng Công an, đặc biệt là các ứng dụng triển khai tại Công an cơ sở, yêu cầu đặt ra không chỉ là triển khai nhanh mà còn phải bảo đảm tính chính xác, thống nhất và an toàn ở mức cao. Việc xây dựng các pipeline CI/CD chuẩn, được thiết kế thống nhất và dùng chung trong toàn lực lượng, sẽ tạo nền tảng để các đơn vị triển khai ứng dụng một cách đồng bộ, giảm phụ thuộc vào năng lực cá nhân và nâng cao hiệu quả quản lý, vận hành hệ thống.

Tại Trường Đại học Kỹ thuật - Hậu cần CAND, mô hình tự động hóa toàn trình có ý nghĩa đặc biệt quan trọng trong công tác đào tạo và nghiên cứu. Việc tổ chức các phòng thí nghiệm, môi trường thực hành theo mô hình CI/CD tự động giúp học viên tiếp cận trực tiếp với quy trình triển khai hiện đại, gắn lý thuyết với thực tiễn công tác sau này tại các đơn vị nghiệp vụ. Đồng thời, đây cũng là cơ sở để nghiên cứu, hoàn thiện các mô hình triển khai tự động phù hợp với đặc thù hệ thống thông tin của lực lượng Công an, góp phần nâng cao năng lực làm chủ công nghệ và chủ động bảo đảm an ninh, an toàn thông tin trong quá trình vận hành các hệ thống trọng yếu.

3.4.3. Gắn mô hình DevOps với cơ chế “một cửa” trong triển khai và quản lý hệ thống

Trong quá trình chuyển đổi số toàn diện của lực lượng Công an, việc gắn mô hình DevOps với cơ chế “một cửa” trong triển khai và quản lý hệ thống công nghệ thông tin là giải pháp mang tính đột phá, góp phần đổi mới phương thức tổ chức, điều hành và giám sát các hoạt động kỹ thuật. Theo đó, cần xây dựng một nền tảng quản lý thống nhất, nơi toàn bộ quy trình từ tiếp nhận yêu cầu nghiệp vụ, phân tích - thiết kế hệ thống, phát triển phần mềm, kiểm thử, triển khai, vận hành đến hỗ trợ kỹ thuật được thực hiện tập trung, xuyên suốt và đồng bộ.

Mô hình “một cửa số” kết hợp DevOps giúp cán bộ kỹ thuật dễ dàng theo dõi tiến độ triển khai, đánh giá chất lượng sản phẩm, kiểm soát trạng thái vận hành của hệ thống theo thời gian thực. Thông qua đó, nâng cao tính minh bạch, rõ trách nhiệm, rõ tiến độ trong quá trình tổ chức thực hiện, hạn chế tình trạng chồng chéo, phân tán và phụ thuộc quá nhiều vào cá nhân. Đồng thời, việc tích hợp các công cụ quản lý yêu cầu, quản lý mã nguồn, pipeline CI/CD, giám sát và hỗ trợ kỹ thuật trên cùng một nền tảng giúp rút ngắn đáng kể chu trình xử lý công việc, nâng cao hiệu suất và chất lượng triển khai.

Đối với các ứng dụng phục vụ người dân và doanh nghiệp trong lĩnh vực quản lý hành chính về trật tự xã hội, mô hình “một cửa số” gắn với DevOps không chỉ nâng cao hiệu quả nội bộ mà còn trực tiếp cải thiện chất lượng cung cấp dịch vụ công. Thời gian xử lý hồ sơ được rút ngắn, quy trình được chuẩn hóa và minh bạch hơn, góp phần nâng cao mức độ hài lòng của người dân, phù hợp với mục tiêu xây dựng nền hành chính hiện đại, lấy người dân làm trung tâm phục vụ.

3.4.4. Chuẩn hóa hạ tầng và nâng cao năng lực đội ngũ cán bộ kỹ thuật

Bên cạnh việc hoàn thiện mô hình và quy trình triển khai, chuẩn hóa hạ tầng kỹ thuật và nâng cao chất lượng nguồn nhân lực là yếu tố mang tính quyết định để bảo đảm tính bền vững của quá trình tự động hóa. Hạ tầng công nghệ thông tin cần được quy hoạch, đầu tư theo hướng tập trung, hiện đại, bảo đảm khả năng mở rộng, an toàn, an ninh thông tin và đáp ứng yêu cầu triển khai các hệ thống nghiệp vụ quy mô lớn, liên thông đa ngành.

Song song với đó, cần chú trọng đào tạo, bồi dưỡng đội ngũ cán bộ kỹ thuật theo hướng đa nhiệm, vừa nắm vững nghiệp vụ chuyên ngành Công an, vừa làm chủ các công nghệ DevOps, DevSecOps, điện toán đám mây, container hóa và tự động hóa triển khai. Việc xây dựng bộ tiêu chuẩn kỹ thuật, quy trình CI/CD và các pipeline mẫu dùng chung trong toàn lực lượng sẽ giúp thống nhất cách làm, giảm phụ thuộc vào kinh nghiệm cá nhân, đồng thời nâng cao khả năng chia sẻ, kế thừa và mở rộng hệ thống.

Đây chính là nền tảng quan trọng để bảo đảm các hệ thống công nghệ thông tin của lực lượng Công an được triển khai đồng bộ, an toàn, hiệu quả và vận hành ổn định trong dài hạn, đáp ứng yêu cầu ngày càng cao của công tác quản lý nhà nước về an ninh, trật tự.

3.5. Kết luận chương 3.

Chương 3 đã hoàn thiện việc xây dựng và triển khai thành công hệ thống CI/CD tích hợp kiểm soát chất lượng và rà quét bảo mật mã nguồn. Toàn bộ quy trình từ quản lý mã nguồn, build, kiểm thử, phân tích tĩnh bằng SonarQube đến quét lỗ hổng container bằng Trivy được tự động hóa trong một pipeline thống nhất, vận hành ổn định trong môi trường thực nghiệm.

Kết quả cho thấy hệ thống có khả năng phát hiện sớm lỗi lập trình, vi phạm tiêu chuẩn mã nguồn và các lỗ hổng bảo mật nghiêm trọng, đồng thời chủ động ngăn chặn việc phát hành khi mức rủi ro vượt ngưỡng cho phép. Cơ chế quản lý artifact, lưu trữ báo cáo và phân quyền truy cập góp phần bảo đảm tính toàn vẹn, khả năng truy vết và an toàn cho vòng đời phát triển phần mềm.

Sau tất cả, với những gì đã làm được chương 3 đã khẳng định mô hình đề xuất là khả thi, hiệu quả và phù hợp để áp dụng trong môi trường đào tạo cũng như thực tiễn phát triển phần mềm, tạo nền tảng vững chắc cho việc mở rộng DevSecOps trong các hệ thống sau này.

KẾT LUẬN

Quá trình nghiên cứu và triển khai đề tài “Nghiên cứu xây dựng hệ thống phần mềm tự động hóa CI/CD trong quản lý và rà quét mã nguồn bảo mật tại Trường Đại học Kỹ thuật - Hậu cần CAND” đã đạt được những kết quả quan trọng cả về mặt lý thuyết và thực tiễn.

Về mặt lý thuyết, đề tài đã nghiên cứu, hệ thống hóa và làm rõ các kiến thức cốt lõi liên quan đến quy trình tích hợp - triển khai liên tục (CI/CD), tự động hóa quy trình phát triển phần mềm, và các phương pháp rà quét mã nguồn bảo mật trong mô hình DevSecOps. Đồng thời, đề án cũng làm rõ vai trò của các công cụ như GitLab CI, Jenkins, SonarQube, Trivy, Harbor và GitLab Runner trong việc xây dựng một chu trình tự động hóa hoàn chỉnh, đảm bảo tính an toàn, minh bạch và hiệu quả trong quản lý mã nguồn.

Trong hướng phát triển tiếp theo, hệ thống sẽ được mở rộng và tối ưu hơn nhằm tích hợp với các hệ thống quản lý chung của Nhà trường, hướng tới mô hình hệ sinh thái DevSecOps nội bộ đồng bộ và an toàn. Ngoài ra, đề án sẽ tiếp tục nghiên cứu việc ứng dụng trí tuệ nhân tạo (AI) và phân tích tự động để hỗ trợ phát hiện bất thường trong mã nguồn, tối ưu hiệu năng pipeline, và đề xuất biện pháp khắc phục tự động. Song song đó, hệ thống có thể được mở rộng thêm các tính năng tự động hóa kiểm tra cấu hình bảo mật (Security as Code), giúp đảm bảo an toàn ngay từ giai đoạn phát triển.

Kết quả đạt được đến thời điểm hiện tại minh chứng cho tính khả thi của việc ứng dụng mô hình CI/CD và DevSecOps trong môi trường Nhà trường mà còn là cơ sở quan trọng để Nhà trường tiếp tục triển khai các giải pháp chuyển đổi số, tự động hóa quy trình quản lý phần mềm và bảo mật thông tin. Đề tài đã góp phần khẳng định vai trò của công nghệ tự động hóa trong việc nâng cao hiệu quả quản lý, giảm thiểu rủi ro an ninh và tạo tiền đề cho việc hình thành một mô hình quản lý an toàn, hiện đại, bền vững và phù hợp với định hướng chuyển đổi số của lực lượng Công an nhân dân.

DANH MỤC TÀI LIỆU THAM KHẢO

Tiếng Việt

- [1] Nguyễn Văn Cường, *Kỹ thuật DevOps và triển khai CI/CD trong quản lý hệ thống phần mềm*, NXB Bách khoa Hà Nội, 2023.
- [2] Trần Văn Bình, *Kỹ thuật lập trình bảo mật cho hệ thống Web trong các cơ quan nhà nước*, NXB Lao động, 2023.
- [3] Lê Xuân Trường, *Xây dựng hệ thống phần mềm quản lý trong lĩnh vực an ninh*, NXB Khoa học và Kỹ thuật, 2023.
- [4] Nguyễn Thanh Bình, *Quản lý và kiểm thử phần mềm tự động trong môi trường DevOps*, NXB Thông tin và Truyền thông, 2022.
- [5] Bộ Thông tin và Truyền thông, *Hướng dẫn triển khai an toàn DevOps trong cơ quan nhà nước*, Hà Nội, 2022.
- [6] Viện Chiến lược Thông tin và Truyền thông, *Báo cáo xu hướng ứng dụng CI/CD và DevSecOps trong phát triển phần mềm tại Việt Nam*, Hà Nội, 2023.

Tiếng Anh

- [7] Jez Humble & David Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*, Addison-Wesley, 2010.
- [8] Gene Kim, Jez Humble, Patrick Debois, John Willis, *The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations*, IT Revolution Press, 2021.
- [9] Kief Morris, *Infrastructure as Code: Managing Servers in the Cloud*, O'Reilly Media, 2020.
- [10] Glenn Wilson, *DevSecOps: A Leader's Guide to Producing Secure Software Without Compromising Flow, Feedback, and Continuous Improvement*, Packt Publishing, 2023.
- [11] John Ferguson Smart, *Jenkins: The Definitive Guide*, O'Reilly Media, 2022.
- [12] O. Chapple, *Practical GitLab CI/CD: A Hands-On Guide to Implementing Pipelines*, Apress, 2022.
- [13] R. M. P. Teixeira, *Full-Stack Web Development with MongoDB, Express, Vue, and Node: Implementing CI/CD with Docker and Jenkins*, Packt Publishing, 2021.

[14] OWASP Foundation, *OWASP Secure Coding Practices Quick Reference Guide*, Version 2.0, 2022.

[15] IBM Security, *DevSecOps and Automated Security Testing: Integrating Security into CI/CD Pipelines*, IBM Press, 2022.

[16] Red Hat, *Automating DevOps Pipelines with Jenkins, GitLab CI/CD, and OpenShift*, Red Hat Publications, 2023.

[17] GitLab Inc., *GitLab Documentation: CI/CD Pipelines and Secure DevOps Workflow*, GitLab Docs, 2024.

[19] S. Greger, *Web Application Security: A Beginner's Guide for Law Enforcement IT Systems*, O'Reilly Media, 2020.

[20] B. Traversy, *Modern Web Development with Node.js, Vue.js, and MongoDB*, Packt Publishing, 2021.